Friedrich-Alexander-Universität

Pattern Recognition Lab

Continual Learning for Promptable Universal 3D Medical Image Segmentation in Few-Shot Settings

Master's Thesis in Computer Science

submitted
by

Shashank Shashank

born 25.03.1998 in India

Written at

Lehrstuhl für Mustererkennung (Informatik 5)
Department Informatik
Friedrich-Alexander-Universität Erlangen-Nürnberg.

Advisor: Prof. Dr.-Ing. habil. Andreas Maier, Dr.-Ing. Tomás Arias Vergara

Started: 15.05.2026

Finished: 15.11.2026

Ich versichere, dass ich die Arbeit ohne fremde Hilfe und ohne Benutzung anderer als der angegebenen Quellen angefertigt habe und dass die Arbeit in gleicher oder ähnlicher Form noch keiner anderen Prüfungsbehörde vorgelegen hat und von dieser als Teil einer Prüfungsleistung angenommen wurde. Alle Ausführungen, die wörtlich oder sinngemäß übernommen wurden, sind als solche gekennzeichnet.

Die Richtlinien des Lehrstuhls für Examensarbeiten habe ich gelesen und anerkannt, insbesondere die Regelung des Nutzungsrechts in Punkt 2.5

Erlangen, den ..... Unterschrift

Übersicht

Abstract

Contents

Notation **xi**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Problem Statement . . . 2
1.3 Research Question and Objectives . . . 4
1.4 Approach and Scope . . . 5
1.5 Contributions . . . 6
1.6 Thesis Outline . . . 7

2 State of the Art **9**
2.1 Text-Promptable Medical Image Segmentation . . . 9
2.2 Few-Shot Adaptation of Segmentation Models . . . 10
2.3 Continual Learning for Pretrained Vision–Language Models . . . 12
2.4 Continual Learning for Medical Image Segmentation . . . 15
2.4.1 Networks Without a Foundation Model . . . 15
2.4.2 Foundation Models Without Text Prompts . . . 17
2.4.3 Text-Promptable Foundation Models . . . 18
2.5 Summary and Research Gap . . . 18

3 Theoretical Background **25**
3.1 Free-Text Promptable Volumetric Segmentation . . . 25
3.1.1 Encoder–Decoder Networks for Volumetric Segmentation . . . 26
3.1.2 Attention . . . 26
3.1.3 VoxTell . . . 27
3.2 Continual Learning . . . 29
3.2.1 Problem Setting and Scenarios . . . 29

3.2.2 Low-Rank Adaptation . . . 30
3.2.3 Knowledge Distillation and ZSCL . . . 30
3.2.4 Prompt Learning and CPE-CLIP . . . 31
3.3 Evaluation Metrics . . . 32

4 Data Description **37**
4.1 SKM-TEA . . . 37
4.2 AeroPath . . . 39
4.3 Data Splits and Few-Shot Subsets . . . 40
4.4 Preprocessing . . . 40

5 Method **43**
5.1 Continual-Learning Setting . . . 43
5.2 Evaluation Protocol . . . 44
5.3 Adapting VoxTell to One Task . . . 47
5.4 Continual-Learning Strategies . . . 50
5.4.1 Sequential Fine-Tuning . . . 50
5.4.2 Task-Specific and Shared LoRA . . . 51
5.4.3 Replay . . . 52
5.4.4 Distillation-Based Strategy . . . 53
5.4.5 Prompt-Tuning Strategy . . . 55
5.5 Computational Complexity . . . 56

6 Experiments and Results **59**
6.1 Experimental Setup . . . 59
6.1.1 Overview of the Experiments . . . 59
6.1.2 Data, Task Orders and Few-Shot Settings . . . 60
6.1.3 Training and Evaluation Configuration . . . 60
6.1.4 Metrics and Statistical Analysis . . . 61
6.2 Comparison of the Strategies . . . 63
6.3 LoRA Rank . . . 68
6.4 Diagnosis of the Patellar Failure . . . 70
6.5 Amount of Labelled Data . . . 71
6.6 Task Order . . . 74
6.7 Computational Cost . . . 76

7 Discussion **85**
7.1 Answer to the Research Question 85
7.2 Training Budget as a Common Factor 89
7.3 Limitations and Threats to Validity 91
7.4 Generalisation and Extensions 93

8 Summary **97**

A Additional Experimental Details and Results **101**
A.1 Training and Test Volumes 101
A.2 Evaluation Matrices of All Runs 101

List of Abbreviations **117**

List of Figures **119**

List of Tables **121**

Bibliography **123**

Notation

Scalars are written in italic, vectors in bold lowercase and matrices in bold uppercase letters. The section in the right column is where the symbol is first defined.

Symbol	Meaning	Sec.
Tasks and data		
T	number of tasks in the task sequence ($T = 6$)	5.1
t; j	task index; stage index ($j = 0$: pretrained model, $j = t$: after training on task t)	3.2
k	number of labelled training volumes per task in a few-shot setting	1.2
$\mathcal{D}_t^{\text{tr}}, \mathcal{D}_t^{\text{val}}, \mathcal{D}_t^{\text{te}}$	training, validation and test set of task t	5.1
$\mathcal{D}_c^{\text{tr}}, \mathcal{D}_c^{\text{te}}$	training and test set of the control task	5.1
p; p_t; p_c	text prompt; text prompt of task t; text prompt of the control task	3.1
$\boldsymbol{X}$	image volume or patch	3.1
$\boldsymbol{Y}$; $\hat{\boldsymbol{Y}}$	binary label mask; binary prediction	3.1
Ω; v	voxel set of a volume or patch; voxel	3.1
y_v; z_v; $\hat{y}_v$	label, logit and predicted foreground probability of voxel v	3.1
Model		
f_θ; θ	segmentation model; its parameters (for VoxTell all except those of the frozen text encoder)	3.1, 3.1.3
$\boldsymbol{Z}$	output logits of the model for a volume or patch	3.1
σ	logistic function	3.1
θ_0	pretrained parameters	3.2
θ_t	parameters after stage t, carried forward to the next task	3.2

Symbol	Meaning	Sec.
$\tilde{\theta}_t$	parameters evaluated after stage t	5.2
P	number of parameters of the segmentation model	5.5
s	resolution stage ($s = 0$: full resolution, $s = 5$: bottleneck)	3.1.1
$\boldsymbol{F}_s$; C_s	encoder features of stage s; their number of channels	3.1.1
$\boldsymbol{U}$; $\boldsymbol{S}$	query vectors and source vectors of an attention block (rows)	3.1.2
n_q; n_s; n_h	number of queries, of sources and of attention heads	3.1.2
d; d_h	model dimension; dimension per attention head	3.1.2
$\boldsymbol{W}_q, \boldsymbol{W}_k, \boldsymbol{W}_v, \boldsymbol{W}_o$	query, key, value and output projections of an attention block	3.1.2
$\boldsymbol{x}$	input of a linear projection $\boldsymbol{x} \mapsto \boldsymbol{W}\boldsymbol{x}$	3.1.2
f_{text}; $\boldsymbol{q}$	frozen text encoder; text query $\boldsymbol{q} = f_{\text{text}}(p)$	3.1.3
L_{dec}	number of prompt-decoder layers ($L_{\text{dec}} = 6$)	3.1.3
G; $\boldsymbol{E}_s$	number of guidance vectors ($G = 32$); text-guidance matrix of stage s	3.1.3
$\boldsymbol{H}_s$; $\boldsymbol{H}'_s$	decoder features of stage s after and before the fusion with the text guidance	3.1.3
Training		
$\mathcal{S}$	continual-learning strategy	5.2
ϕ	parameters trained by a strategy	5.3
$\mathcal{L}_{\text{seg}}$	segmentation loss	3.1
ϵ	smoothing constant of the Dice loss	5.3
E; N_t; U	epochs per task; training samples per epoch of task t; iterations per task, $U = E \cdot N_t$	5.3
η_0; μ; ω	initial learning rate; momentum; weight decay	5.3
π_{fg}	probability that a training patch is centred on a voxel of the target structure	5.3
ℓ; ℓ^*; θ^{best}	validation loss of an epoch; lowest validation loss so far; parameters with the lowest validation loss	5.3
κ; $\kappa_{\mathcal{S}}$	cost of one forward pass on a patch; cost of the backward pass of strategy $\mathcal{S}$	5.5
Strategies		

Symbol	Meaning	Sec.
$\boldsymbol{W}_0$	frozen pretrained weight adapted by LoRA	3.2.2
$\boldsymbol{A}$, $\boldsymbol{B}$; r	LoRA factors; rank	3.2.2
d_{in}; d_{out}	input and output dimension of an adapted weight	3.2.2
α; γ_r	LoRA scaling hyperparameter; scaling factor $\gamma_r = \alpha/\sqrt{r}$	3.2.2
$\mathcal{M}_t$; ρ; $M_{\max}$	replay memory after task t; replay fraction; capacity of the memory	5.4.3
λ; $\mathcal{L}_{\text{KD}}$	weight of the distillation loss; distillation loss	3.2.3
$\hat{\theta}_\nu$; ν; I_{we}	weight ensemble after ν updates; number of ensemble updates; interval between updates in iterations	3.2.3
$\boldsymbol{e}_g$	g-th guidance vector of $\boldsymbol{E}_4$	5.4.4
$\boldsymbol{f}_v$; Ω_4	stage-4 encoder feature at voxel v; voxels of stage 4	5.4.4
$q_{g,v}$; $\bar{q}_{g,v}$	distribution over Ω_4 for guidance vector g of the student; of the teacher	5.4.4
β_t; $\mathcal{C}_t$	gradient scale of task (session) t in prompt tuning; set of classes of session t in CPE-CLIP	3.2.4
$\boldsymbol{V}_l$; m; L_p	learned prompt tokens of prompt-decoder layer l; tokens per layer; number of prompted layers	5.4.5
h_ψ; ψ	meta-net; its parameters	5.4.5
$\bar{\boldsymbol{f}}$	mean of the projected stage-4 image features, input of the meta-net	5.4.5
Evaluation		
$\text{DSC}(\hat{\boldsymbol{Y}}, \boldsymbol{Y})$	Dice similarity coefficient	3.3
$a_{i,j}$	entry of the evaluation matrix: mean DSC on task i after stage j	3.3
c_j	mean DSC on the control task after stage j	3.3
N_{boot}	number of bootstrap draws	6.1.4

Chapter 1

Introduction

1.1 Motivation

Medical image segmentation assigns every voxel of an image either to a structure, such as an organ or a tissue, or to the background. The voxels assigned to a structure form its mask. Masks can be drawn by hand, but this is laborious, because in a three-dimensional (3D) volume the structure has to be outlined on every slice in which it appears. Different readers also draw them differently [Des+21].

Automatic segmentation with deep learning has commonly meant training one model for each dataset and its target structures, for example with nnU-Net [Ise+21]. Universal segmentation models replace these per-dataset models with one network that segments many structures. For example, the developers of the universal model SAT compared it with 72 nnU-Nets, one trained on each of their datasets [Zha+25d]. The structure to segment is specified by an additional input, the prompt [Zha+25d; Rok+26]. The prompt can be spatial, such as a point or a box marked in the image [Ma+24; Che+23], or text that names the target (Section 2.1).

Text-promptable models differ in the text they accept. SAT is prompted with terms from a fixed vocabulary [Zha+25d]. In contrast, VoxTell accepts both a single anatomical term and a full clinical sentence [Rok+26]. We call models of this kind *free-text promptable*. In both models, the structure to segment is not tied to an output channel of the network. The same weights serve every structure, and the text prompt alone selects the target [Zha+25d, Sec. 4.5; Rok+26, Sec. 3] (Section 3.1.3).

This thesis adapts VoxTell, which was trained on more than 62,000 computed tomography (CT), magnetic resonance imaging (MRI) and positron emission tomography (PET)

volumes. On datasets it was not trained on, it reached a mean Dice similarity coefficient (DSC) of 70.85 % [Rok+26, Table 1], where the DSC measures the overlap between the predicted and the reference mask. On knee MRI from the SKM-TEA dataset [Des+21], its mean DSC was 10.72 %, averaged over four cartilages and two menisci [Rok+26, Suppl. Table 9]. The VoxTell authors attribute this low value to the small share of knee images in the training data and to structures missing from it [Rok+26, Sec. 6]. As future work, they suggest adapting the model with a few labelled examples [Rok+26, Sec. 6].

Automatic segmentation of the knee cartilages and menisci is useful because quantitative biomarkers for osteoarthritis are measured within these structures [Des+21]. One example is the T2 relaxation time, which reflects the degeneration of collagen-rich tissue [Des+21]. These values can only be read out once the structures are outlined. The SKM-TEA authors name manual outlining as one obstacle to the use of quantitative MRI [Des+21].

With few manual outlines available, VoxTell would have to learn the knee structures from a few labelled volumes. New structures may also become relevant over time, and the model would then have to adapt to each of them in turn. Retraining VoxTell from scratch on its training data together with each new structure would be expected to preserve most of what it could segment before. However, training the published model took about six days on 64 NVIDIA A100 GPUs [Rok+26, p. 5].

1.2 Problem Statement

Instead of being retrained from scratch, the model can be adapted to one new structure at a time. We call the learning of one new structure, named by one text prompt, a *task*. Continual learning is the setting in which a model learns from a sequence of tasks, using the data of the current task and limited or no data of earlier tasks [De +22, Sec. 2]. After each task, the model has to segment the new structure while keeping the structures learned earlier in the sequence. It also has to keep what it could segment before the first task, which we call its pretrained ability (Figure 1.1).

The simplest way to learn a task is to fine-tune the model on its data. However, training on later tasks can lower the performance on earlier ones, an effect known as catastrophic forgetting [De +22, Sec. 1]. For pretrained vision–language models, continual fine-tuning can also lower the zero-shot accuracy on data the model was not trained on [Zhe+23]. In our setting, this corresponds to a loss of pretrained ability. The more the model is allowed to change, the better it can learn the new structure, but the more it may lose of what it knew.

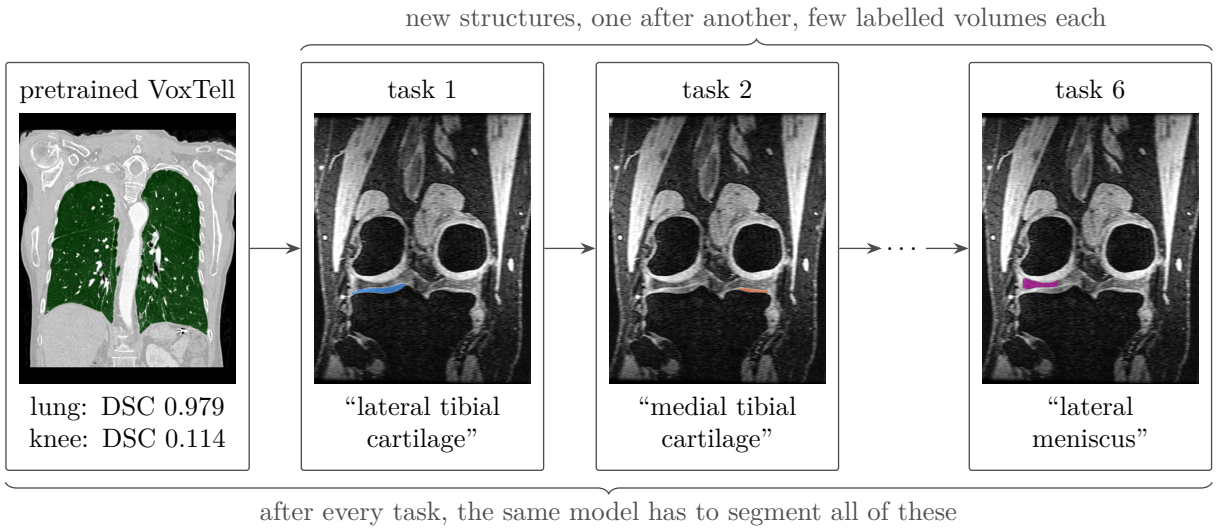


Figure 1.1: Setting of this thesis. The pretrained VoxTell segments the lung well but the knee structures poorly (DSC from our evaluation; for the knee, the mean over its six structures). It is then adapted to the six knee structures one after another, each named by a text prompt and learned from a few labelled volumes. Tasks 3 to 5 are not shown. After every task, the same model has to segment all of these structures, with the lung standing for its pretrained ability. Images: example volumes of Figure 4.1 with their reference masks.

De Lange et al. call this conflict the stability–plasticity dilemma, with stability meaning the retention of previous knowledge and plasticity the integration of new knowledge [De+22, Sec. 1].

With few labelled volumes per task, it is not clear in advance how the balance between stability and plasticity shifts. Fewer volumes mean fewer updates when the number of training epochs is fixed. Fewer updates give the model less opportunity to learn the new structure, but they may also change less of what it knew. Here, *few-shot* means training the model on k labelled volumes per task, one shot being one labelled training volume.

Existing continual-learning results do not show how few labelled volumes shift the balance between stability and plasticity, because they were obtained in other settings. The methods for pretrained vision–language models reviewed in Section 2.3 were developed and evaluated on classification or on tasks with text output. In medical image segmentation, continual-learning methods have mostly given each new task or class components of its own, such as a decoder or an output head [Ji+23; Guo+25; Zha+23; Ye+25]. VoxTell needs no such components, because a new structure is named by a new text prompt and segmented with the same weights.

In our literature search, we found no work that adapts a pretrained free-text promptable 3D segmentation model to a sequence of new structures with few labelled volumes per task (Section 2.5). The closest works use a two-dimensional (2D) model with full training sets [Gao26; Gao+26b; Gao+26a] or a 3D model without text prompts [Che+26b], and none of them measures how much of the pretrained ability is kept. Other works measure it for CLIP [Zha+24] and for segmentation models prompted with points or boxes [Wan+26b; Wan+24], and our control task builds on this idea.

1.3 Research Question and Objectives

This thesis addresses the following research question:

> **How do different continual-learning strategies trade off adaptation performance and catastrophic forgetting when adapting free-text promptable universal 3D medical image segmentation models under few-shot conditions?**

By *adaptation performance* we mean how well the model segments a new structure right after it has been trained on it. In the research question, *forgetting* covers the loss on earlier tasks of the sequence as well as the loss of pretrained ability. Few-shot conditions are $k = 5$, 10 and 20 labelled volumes per task, which we compare with the full training split of 108 volumes per task. Section 1.4 states which strategies were run with few volumes.

We pursue four objectives:

(1) Develop a few-shot continual-learning evaluation protocol for free-text promptable universal 3D medical image segmentation (Section 5.2).

(2) Compare continual-learning strategies under this protocol in terms of their adaptation and forgetting (Section 6.2).

(3) Investigate how the amount of labelled training data influences the evaluated strategies, separating the effect of fewer volumes from that of fewer updates (Section 6.5).

(4) Analyse how the task order, the low-rank adaptation (LoRA) rank and the components of the strategies (replay, weight ensemble and gradient scaling, Section 1.4) influence adaptation and forgetting (Sections 6.2, 6.3 and 6.6).

1.4 Approach and Scope

We study the research question with VoxTell [Rok+26] as the pretrained model and six structures of the SKM-TEA knee MRI dataset [Des+21] as tasks. These are the patellar, femoral, medial tibial and lateral tibial cartilage and the medial and lateral meniscus. The tasks are trained in two task orders, an original order chosen at random and its reverse, to see whether the results depend on the order. The lung in the chest CT volumes of AeroPath [Stø+23] serves as a control task. It is not trained on and is used only to measure how much of the pretrained ability is kept. After each task, the model is tested on all six tasks and on the control task (Section 5.2).

The baseline is sequential fine-tuning, which trains all parameters on each task and adds nothing to reduce forgetting. Three strategies are based on LoRA [Hu+22]. LoRA keeps the pretrained weights frozen and trains only a low-rank update of them, called the adapter. The number of parameters of the adapter grows with its rank r. Task-specific LoRA trains one adapter of rank $r = 4$ per task and selects it by the task identity at inference. Shared LoRA trains one adapter of rank $r = 1024$ for all tasks, and shared LoRA with replay adds a memory of training samples from earlier tasks. The distillation-based strategy (adapted from ZSCL [Zhe+23]) trains all parameters and distils the features of the frozen pretrained model into the adapted one. Like ZSCL, it also keeps a weight ensemble, the running mean of the pretrained parameters and of parameter snapshots taken during training, which we evaluate separately. The prompt-tuning strategy (adapted from CPE-CLIP [DAl+23]) freezes the pretrained weights and trains learned prompt tokens. It also trains a meta-net, a small network that shifts the encoded text prompt. The meta-net is our addition, inspired by CoCoOp [Zho+22a]. As in CPE-CLIP, the gradients of the tokens and the meta-net are multiplied by a factor that decreases with the task index, which we call gradient scaling [DAl+23, p. 5, Eqs. 12–13]. Together, the strategies cover replay, regularisation and parameter-efficient adaptation, the three families of continual-learning methods for vision–language models [Liu+26b, Sec. 4]. All strategies except task-specific LoRA use one parameter set for all tasks (Section 5.4).

All strategies are trained with the full training split. Sequential fine-tuning, task-specific LoRA and shared LoRA are also trained with $k = 5$, 10 and 20 volumes per task, with one draw of volumes for each k. With the training schedule of our experiments, a task with five volumes is trained with 50 updates, against 1,080 with the full training split. Further runs with $k = 5$ and 1,080 updates per task separate the effect of fewer volumes from that of fewer updates (Section 6.1.2).

defense: the research question names few-shot condi-

From the test results, we compute three primary metrics (Section 3.3). Adaptation is the DSC on each task right after training on it, averaged over the six tasks. Final Forgetting is the mean drop of the DSC on the earlier tasks from its best earlier value to its value after the last task [Cha+18, Sec. 3, Eq. 3]. The control-task drop is the decrease of the DSC on the control task from the pretrained model to the model after the last task. Together, Final Forgetting and the control-task drop measure retention, the performance kept on earlier tasks and on the control task. We also report two secondary metrics. Final Performance is the mean DSC on all tasks after the last task, and Transfer is the mean DSC on each task before it is trained [Zhe+23, Sec. 4]. The pretrained model is evaluated in the same way and serves as a reference. As an upper reference, joint fine-tuning trains the model on all six tasks at once. It was run with the full training split and with each k (Section 5.2).

The study uses one model, one dataset for the tasks, one control task, one fixed text prompt per structure and one draw of training volumes per k. Each configuration was trained once, so we estimate only the variability over the test volumes, using a bootstrap (Section 6.1.4). The conclusions are therefore limited to this setting (Section 7.3). Clinical deployment and studies with clinical users are outside the scope of this thesis, but we report the training cost of the strategies (Section 6.7).

1.5 Contributions

The contributions of this thesis are:

(1) An evaluation protocol for few-shot continual adaptation of a free-text promptable 3D segmentation model, which measures adaptation, forgetting within the task sequence and the loss of pretrained ability on a held-out control task, with the pretrained model and joint fine-tuning as reference points (Sections 5.1 and 5.2).

(2) A systematic comparison of sequential fine-tuning and five strategies from the three families of continual learning for vision–language models, adapted to VoxTell, under one shared training configuration, with the full training split and, for three of them, with 5, 10 and 20 labelled volumes per task (Sections 5.4, 6.2 and 6.5).

(3) An analysis of the factors that affect continual adaptation in this setting: the amount of labelled data separated from the number of updates, the task order, the LoRA rank and the components of the strategies (replay, weight ensemble, gradient scaling),

together with a diagnosis of why task-specific LoRA did not learn patellar cartilage (Sections 6.2 to 6.6).

Sequential fine-tuning, LoRA, replay [De +22, Sec. 3.1], ZSCL and CPE-CLIP are existing methods. The work of this thesis is their application to VoxTell with the changes described in Section 5.4, the evaluation protocol and all experiments.

1.6 Thesis Outline

Chapter 2 reviews text-promptable medical image segmentation and few-shot adaptation of segmentation models, as well as continual learning for pretrained vision–language models and for medical image segmentation. It ends with the gap that this thesis addresses. Chapter 3 introduces VoxTell, states the continual-learning problem in formal terms, describes the published methods on which the strategies are based and defines the evaluation metrics. Chapter 4 describes the SKM-TEA and AeroPath data, the tasks and the data splits. Chapter 5 defines the evaluation protocol, the adaptation of VoxTell to one task and the strategies with their complexity. Chapter 6 gives the experimental setup and the results. It compares the strategies and reports the experiments on the LoRA rank, the amount of labelled data, the task order and the strategy components, as well as the training cost. Chapter 7 discusses the results across the experiments, answers the research question and states the limitations, and Chapter 8 summarises the thesis. The appendix lists the training and test volumes (Section A.1) and gives the evaluation matrices of all runs (Section A.2).

Chapter 2

State of the Art

This thesis adapts a pretrained free-text promptable 3D segmentation model to a sequence of new target structures, each with few labelled volumes. The related work therefore comes from four areas: text-promptable medical image segmentation, few-shot adaptation of segmentation models, continual learning for pretrained vision–language models, and continual learning for medical image segmentation. The chapter reviews them in this order and closes with a summary of all reviewed works and the gap that this thesis addresses.

2.1 Text-Promptable Medical Image Segmentation

Promptable segmentation models differ in how the target is selected: either by a spatial prompt marked in the image or by a text prompt that names the target. MedSAM and SAM-Med2D, two 2D adaptations of the Segment Anything Model (SAM) [Kir+23] to medical images, take spatial prompts. MedSAM is prompted with bounding boxes [Ma+24], and SAM-Med2D with boxes, points and masks [Che+23]. SegVol, a 3D model for CT, accepts both kinds of prompt [Du+24]. Besides points and boxes, it takes category names, which it inserts into a fixed template and embeds with a frozen CLIP text encoder [Rad+21; Du+24]. It was first pretrained on about 96,000 CT volumes without using labels and then fine-tuned on about 6,000 labelled volumes covering more than 200 categories [Du+24]. Other work keeps the target class fixed and uses text as additional input when labels are scarce, as LViT does with radiologist-verified descriptions of lung infections, which it needs at inference as well [Li+24].

Several text-promptable models select their target from a fixed vocabulary. The CLIP-Driven Universal Model feeds the CLIP embedding of a template such as “A computerized

tomography of a [CLS]" to a controller that generates the output convolutions of each class [Liu+23a]. It was trained on 3,410 CT scans from 14 public datasets with 25 organs and 6 tumour types [Liu+23a]. SAT extends text prompting to CT, MRI and PET volumes: a text encoder trained on medical knowledge embeds terms from a vocabulary of 497 classes, and a query decoder matches them with the features of a 3D U-Net [Zha+25d]. Free-form input such as a radiology report has to be translated into this vocabulary by a separate language model first [Zha+25d]. On 72 datasets with 22,186 scans, SAT's larger variant, SAT-Pro, reached an average DSC of 78.81 % over all classes, which the authors report as comparable to the 80.54 % of 72 dataset-specific nnU-Nets [Zha+25d]. BiomedParse [Zha+25c] and MedSAM-3 [Liu+26a] also take text prompts, but they segment 2D images. On 3D data, SAT-Pro was better than BiomedParse on 29 of 30 categories, and the SAT authors attribute BiomedParse's false positives to its slice-wise design [Zha+25d].

VoxTell, the model adapted in this thesis, accepts a text prompt that may be a single anatomical term or a full clinical sentence [Rok+26]. It was trained on more than 62,000 CT, MRI and PET volumes from 158 public datasets with 1,087 anatomical and pathological concepts [Rok+26, p. 4]. Unlike SAT, it fuses the text with the image features at every decoder resolution [Rok+26, p. 4] (Section 3.1.3). On eleven datasets that were not used for training, its mean zero-shot DSC was 70.85 %, against 51.23 % for SAT, but on the knee MRI of SKM-TEA [Des+21] it reached only 10.72 % [Rok+26].

The VoxTell authors report robustness to synonyms, rephrasings and typos [Rok+26]. SEER measured how five text-promptable models, four of them 3D, respond to free-text requests that a language model generated in eight clinical styles [Zha+26]. Among these models, VoxTell kept the highest DSC under free-text requests, on average and in the worst case, whereas the DSC of SAT on the pelvic-bone dataset PENGWIN fell from 96.05 % with the class-name prompt to 0.01 % [Zha+26]. Even for VoxTell, the worst rephrasing lowered the DSC on PENGWIN from 97.59 % with the class-name prompt to 79.34 % [Zha+26].

2.2 Few-Shot Adaptation of Segmentation Models

Each task in this thesis comes with only a few labelled training volumes, so this section reviews how pretrained models have been adapted to a single target with few labels. In the review by Dissanayake et al., a few-shot model has to segment a new class from one or a few labelled examples, and it can do so in two ways [Dis+25, Sec. 2.2]. In the first, the model receives the labelled examples as input together with the image to segment, so

a new class often needs no change to its weights [Dis+25, Sec. 2.2, 2.2.1]. UniverSeg is an example of this approach for 2D medical images [But+23]. In the second, the weights of a trained model are updated on the labelled examples, for instance by fine-tuning [Dis+25, Sec. 2.2]. This thesis follows the second approach: a pretrained model is fine-tuned on a few labelled volumes of each task, and one labelled training volume counts as one shot (Section 5.1).

Silva-Rodríguez et al. fine-tuned a Swin UNETR, pretrained on partially labelled CT scans, to one organ of TotalSegmentator at a time with 1, 5 or 10 labelled volumes [Sil+25]. This range overlaps with the 5, 10 and 20 labelled volumes per task used in this thesis (Section 6.1.2). On nine organs that were already among the pretraining classes, LoRA [Hu+22] in the encoder and full fine-tuning differed by less than one percentage point of mean DSC at 5 and 10 shots, and only LoRA improved from 5 to 10 shots [Sil+25]. On 16 structures that were new to the model, parameter-efficient methods in the encoder alone reached at most 40.46 % at 10 shots, below full fine-tuning (52.38 %) and fine-tuning the decoder together with LoRA in the encoder (57.39 %) [Sil+25]. SEA-PEFT uses the same pretrained model and organs and selects the type, position and rank of the adapters automatically during training [Ly+26].

For vision–language models, CLIP-LoRA adds rank-2 LoRA modules to the query, key and value projections of every layer of both CLIP encoders [Zan+24]. It uses one set of hyperparameters for eleven 2D image classification datasets with 1 to 16 shots per class [Zan+24]. With a ViT-B/16 image encoder, its average top-1 accuracy was the best of the compared methods at every number of shots, for example 72.5 % against 70.9 % at one shot [Zan+24]. In the authors' ablation, the value or output projection was the best single matrix to adapt, and adapting more matrices or using a higher rank could lower the accuracy [Zan+24]. The authors note that choosing the adapted matrices and the rank is difficult [Zan+24], and we examine the rank in our setting in Section 6.3. Like CLIP-LoRA, we apply LoRA to attention projections, but only to the query and value projections of the cross-attention in the prompt decoder of VoxTell (Section 5.4.2). CoOp and CoCoOp adapt a frozen CLIP in another way, by training context vectors in front of the class name [Zho+22b; Zho+22a, Sec. 3.2–3.3] (Section 3.2.4). The meta-net of our prompt-tuning strategy is inspired by CoCoOp (Section 5.4.5).

The works above adapt a model to one target at a time, each time starting from the same pretrained weights. The two 3D studies evaluate the adapted model only on that target, so they do not show how much of its earlier ability the model loses through the

adaptation [Sil+25; Ly+26]. In this thesis, the same model is trained on the tasks one after another, so each adaptation can also affect the tasks trained before it. Among the works found in our search (Section 2.5), only Chen et al. adapt a pretrained 3D medical model to a sequence of tasks with few labelled examples each, one of them a segmentation task [Che+26b] (Section 2.4.2).

2.3 Continual Learning for Pretrained Vision–Language Models

VoxTell combines a frozen text encoder with an image encoder and lets the encoded text prompt attend to the image features, which makes it a vision–language model (Section 3.1.3). Methods that add tasks to such a model while keeping its text-conditioned abilities have been developed for pretrained vision–language models such as CLIP [Rad+21], and this section reviews them. They were developed for 2D images and evaluated on classification or on tasks with text output.

Continual-learning scenarios differ in whether the task identity of an input is given at inference [vdVen+25] (Section 3.2.1). Independently of the scenario, De Lange et al. divide continual-learning methods into three families according to how they preserve information about earlier tasks [De +22, Sec. 3]. Replay methods reuse stored or generated samples of earlier tasks while learning a new one [De +22, Sec. 3.1]. Regularisation methods add a loss term, either distillation from an earlier model or a penalty on changes to important parameters [De +22, Sec. 3.2]. Parameter-isolation methods assign different parameters to different tasks, which typically requires the task identity at prediction time [De +22, Sec. 3.3]. For vision–language models, Liu et al. group methods into multimodal replay, cross-modal regularisation and parameter-efficient adaptation, and note that many methods combine these families [Liu+26b, Sec. 4].

The multimodal replay method reviewed here avoids storing real images by generating the replayed data. GIFT fine-tunes all parameters of CLIP on each task together with 1,000 images that Stable Diffusion generates from class names of earlier tasks and of ImageNet [Wu+25]. On these images, it distils the image–text similarities of the model from before the task [Wu+25]. Replay of stored real samples, which our replay strategy uses (Section 5.4.3), is reviewed for medical image segmentation in Section 2.4.1, where Bera et al. also compare how the stored data are selected.

ZSCL, a cross-modal regularisation method, showed that continual fine-tuning of CLIP also lowers its zero-shot accuracy on datasets that it has not been trained on yet [Zhe+23]. Its benchmark trains eleven classification datasets as tasks in two task orders, Order I and Order II, and reports Transfer, the mean accuracy on each task before the task is trained, and Last accuracy, the mean accuracy on all tasks after the last one (Section 3.3) [Zhe+23]. To keep the zero-shot accuracy, ZSCL fine-tunes all parameters and distils the image–text similarity distributions of the frozen initial model on a reference dataset, by default 100,000 ImageNet images, and additionally constrains the weights during training [Zhe+23] (Section 3.2.3). On Order I, this raised Transfer from 44.6 % for sequential fine-tuning to 68.1 %, close to the 69.4 % of the zero-shot model, and Last accuracy from 77.3 % to 83.6 % [Zhe+23]. In the authors' ablation, distilling on the images of the current task instead of a reference dataset kept Transfer at the same level (56.7 % compared with 56.8 %) but lowered Last accuracy from 83.0 % to 80.2 % [Zhe+23]. The authors name the need for a reference dataset as the limitation of ZSCL [Zhe+23]. The distillation-based strategy of this thesis is adapted from ZSCL (Section 5.4.4). SPU measures the loss of zero-shot accuracy outside the task sequence, on a control set, the ImageNet validation set, by comparing the adapted model with the frozen pretrained model [Zha+24]. Our control task follows this idea (Section 3.3).

PRIMED transfers the distillation on reference data of ZSCL to medical 2D image classification with BiomedCLIP [Che+26c]. It was evaluated on two task sequences that include X-ray, pathology, fundus, skin, cell and endoscopy images, with the task identity given at inference [Che+26c]. Instead of a fixed reference dataset, it retrieves image–caption pairs related to the tasks from 18 million PubMed figures and distils on them [Che+26c]. Unlike ZSCL, PRIMED distils from the model of the previous task, and in its ablation this teacher gave a higher Last accuracy than the initial model (82.1 % compared with 78.2 %) [Che+26c]. The ablation of ZSCL had found the opposite, a Last accuracy of 83.0 % with the initial model and 80.7 % with the previous one [Zhe+23, Table 1d].

Parameter-efficient methods freeze most of the pretrained model and train a small set of added parameters [Liu+26b, Sec. 4.3]. Liu et al. note that tasks sharing the low-rank space of one LoRA (Section 3.2.2) can still interfere [Liu+26b, Sec. 4.3]. Some methods therefore keep the updates of new tasks orthogonal to the subspaces of earlier tasks [Liu+26b, Sec. 4.3]. LiLoRA, for continual instruction tuning of the multimodal language model LLaVA, shares one LoRA factor across tasks and splits the other into a shared and a task-specific part [Che+26a]. In its Single-type setting, one LoRA trained sequentially

on all tasks reached an average accuracy of only 46.21 %, against 88.13 % for a separate LoRA per task [Che+26a]. LiLoRA came close to the separate LoRAs with less than half their added parameters [Che+26a]. MSLoRA-CR adds a LoRA branch per task to the frozen medical model LLaVA-Med and sums all branches learned so far [Zha+25a]. It was evaluated on 2D question answering, classification and report generation, with the task identity given at inference, and scored higher over all task metrics than a separate LoRA per task [Zha+25a]. MoE-Adapters adds a mixture of LoRA experts with one router per task to a frozen CLIP [Yu+24]. At inference, one autoencoder per task selects the router, and the authors observed that this selection assigned samples of one task to none of the tasks [Yu+24]. A separate LoRA per task and one LoRA shared by all tasks are the two LoRA strategies compared in this thesis (Section 5.4.2).

Other methods train learned prompt tokens, which are vectors added to the input of a frozen model. S-Prompts learns separate prompt tokens for each domain and selects them at inference through the nearest of the image-feature centroids stored per domain [Wan+22]. CoLeCLIP learns one prompt token per task in the frozen image encoder and keeps a single vocabulary of class text embeddings for all tasks [Li+25]. CPE-CLIP targets few-shot class-incremental learning with five examples per new class [DAl+23] (Section 3.2.4). It learns prompt tokens in both CLIP encoders, couples them through a learned projection and scales their gradients down as more classes have been seen, training about 400,000 parameters [DAl+23]. In its ablation on CUB200, a fine-grained bird classification dataset, removing the gradient scaling lowered the accuracy more than removing any other component [DAl+23, Sec. 5.5, Fig. 2]. The prompt-tuning strategy of this thesis is adapted from CPE-CLIP (Section 5.4.5).

These methods do not carry over directly to VoxTell. ZSCL, GIFT and PRIMED distil distributions over the similarities between an image and a set of texts [Zhe+23; Wu+25; Che+26c]. VoxTell predicts one mask per text prompt and does not compute such distributions. CPE-CLIP places prompt tokens in transformer encoders [DAl+23], whereas the image encoder of VoxTell is convolutional (Section 3.1.3). Section 5.4 describes how our strategies adapt these methods.

2.4 Continual Learning for Medical Image Segmentation

Unlike the methods of Section 2.3, which classify images or generate text, the methods reviewed in this section segment medical images. In continual segmentation, a structure that is foreground in the masks of one task can be labelled as background in the masks of another, which is known as background shift [Cer+20]. In the multi-organ CT sequences of Liu et al., for example, the liver is foreground in the first dataset and background in the later ones [Liu+22]. A foundation model is pretrained on large and diverse data and then adapted to many downstream tasks. Section 2.4.1 reviews networks that do not build on a foundation model, and Sections 2.4.2 and 2.4.3 review foundation models without and with text prompts.

cite: definition of foundation model

2.4.1 Networks Without a Foundation Model

Early work on continual learning for medical image segmentation applied regularisation and distillation. Baweja et al. used elastic weight consolidation (EWC) [Kir+17] for brain MRI [Baw+18], and Ozdemir et al. combined distillation from the previous model with a small set of stored examples for shoulder MRI [Ozd+18]. ILT, a method from natural-image segmentation, distils both the output and the encoder features of the previous model [Mic+19]. Several of the works reviewed below use it as a baseline [Ji+23; Liu+24].

In natural-image segmentation, MiB was the first method to explicitly address background shift, by treating old classes as possibly present in the background of a new task [Cer+20]. Like ILT, it is a baseline in several of the medical works reviewed below [Ji+23; Gon+23]. In medical image segmentation, P. Liu et al. used a marginal loss for partially labelled multi-organ CT [Liu+22], and X. Liu et al. used complementary pseudo-labels for structures added one after another in 2D brain-tumour MRI [Liu+23b]. Our masks also label the structures of the other tasks as background, but each mask is the target for one text prompt only (Section 3.1.3), so no voxel receives different labels for the same text prompt. We therefore do not adopt these methods.

Replay methods for segmentation either store selected parts of earlier data or generate the replayed data. Bera et al. keep crops around the target instead of full volumes and select them by the fraction of target voxels and by training difficulty [Ber+23]. On a sequence of prostate, spleen and hippocampus segmentation, twelve such crops gave an average DSC of $83.3 \pm 1.3\,\%$, against $78.3 \pm 1.3\,\%$ for three randomly chosen full volumes [Ber+23]. C^{2}GR

trains a universal segmentation network from scratch on 20 tasks from eight modalities, all as 2D images or slices, and replays image–mask pairs generated by a diffusion model [Li+26]. Each task is specified by a fixed text template that names the modality, the region and the target [Li+26]. After the last task, the mean DSC over all tasks was 83.62 %, against 86.06 % for joint training and 25.23 % for sequential fine-tuning [Li+26].

Parameter-isolation methods keep separate parts of the model for different tasks or structures. Continual Segment freezes an encoder trained on TotalSegmentator and adds one pruned decoder per later dataset [Ji+23], and its extension CL-Net adds decoders for new anatomies [Guo+25]. In an extension of the CLIP-Driven Universal Model (Section 2.1), Zhang et al. share the encoder and decoder across all classes, keep only the output heads class-specific and train the earlier classes on pseudo-labels from the previous model [Zha+23; Liu+24]. The weights of each head are generated by an multilayer perceptron (MLP) of its own from the pooled image features and the CLIP embedding of the class template [Zha+23]. After 13 BTCV organs followed by the LiTS liver tumour, the mean DSC was 0.817 on the organs and 0.466 on the tumour, against 0.770 and 0.456 with pseudo-labels alone [Zha+23]. Since each class has to be named in advance and adds a head, the model grows with every class [Zha+23]. In our setting, the text prompt names the target structure, and all strategies except task-specific LoRA use one parameter set for all tasks.

Some studies compare existing methods instead of proposing a new one. Lifelong nnU-Net trains an nnU-Net through sequences of hippocampus, prostate and cardiac MRI datasets [Gon+23]. Replay, which the authors call rehearsal, showed the least forgetting, whereas EWC and MiB preserved the performance on earlier tasks at the expense of the new ones [Gon+23]. On the last hippocampus dataset, for example, EWC and MiB reached a DSC of 31.93 % and 20.75 %, compared with 90.92 % for sequential training [Gon+23]. The authors also found the results to depend on the task order and recommend comparing methods over several orders [Gon+23], and Ranem et al. found the difference in forgetting between nnU-Net and a vision-transformer variant to depend on the method used [Ran+22]. The Beyond Forgetting benchmark compares 16 methods on 2D slices for new domains, new structures in the same images and new organs in new images, and reports plasticity and forward transfer besides forgetting [Wan+26a]. It names no single best method, but replay gave the best trade-off between retaining earlier tasks and learning new ones, and parameter isolation avoided forgetting at the cost of a growing model size [Wan+26a]. For the new-domain sequences, the task order affected the results [Wan+26a].

2.4.2 Foundation Models Without Text Prompts

SAM-Med3D-MoE and CA-SAM add task-specific parts to a pretrained SAM-based model, which is prompted with points or boxes, and keep the pretrained model unchanged [Wan+24; Wan+26b]. SAM-Med3D-MoE fine-tunes one copy of the mask decoder of SAM-Med3D for each downstream task or poorly segmented class, separately rather than in a sequence [Wan+24]. A gate selects one of these experts or falls back to the original decoder [Wan+24]. With box prompts, the mean DSC on 15 poorly segmented classes rose from 0.532 to 0.564, whereas on the classes that no expert was trained for it fell from 0.541 to 0.400 [Wan+24, Table 1]. CA-SAM inserts one small alignment network per dataset between the frozen image encoder and the mask decoder of SAM, and one variational autoencoder per task selects the network or passes unfamiliar inputs to the frozen SAM [Wan+26b]. On nine 2D datasets learned in sequence, its final mean intersection over union (IoU) was 76.12 %, against 78.43 % for joint training and 53.81 % for zero-shot SAM [Wan+26b]. On five datasets outside the sequence, its IoU after the last task stayed within 5.3 percentage points of the frozen SAM, whereas Learning without Forgetting [Li+18] on the same alignment network lost 16.7 to 46.5 percentage points [Wan+26b, Table 2]. Both evaluations measure how much pretrained ability is kept outside the adapted tasks, as our control task does.

Two works adapt pretrained 3D models without spatial prompts. MedSeqFT fine-tunes a self-supervised CT model through five segmentation tasks in sequence, with a new head per task [Ye+25]. It adds distillation on stored samples, which are chosen by their pretraining loss, and a LoRA-based refinement that is merged into the weights [Ye+25]. Its mean DSC over the five tasks was 68.3 %, against 65.3 % for fine-tuning a separate copy of the pretrained model per task [Ye+25]. The paper reports no results on earlier tasks after later ones and no measure of forgetting [Ye+25]. Chen et al. freeze a 3D brain-MRI foundation model and train one LoRA module and one head per task, selected by the task identity at inference [Che+26b]. The two tasks, tumour segmentation and brain-age regression, each have 16 or 32 labelled samples [Che+26b]. With 32 samples, the tumour DSC of this design was 0.60 ± 0.08 and unaffected by the second task, whereas sequential fine-tuning reached 0.80 ± 0.02 and fell to 0.16 ± 0.19 after the second task [Che+26b].

2.4.3 Text-Promptable Foundation Models

Three recent works by Gao and colleagues are the closest to this thesis: they train a text-promptable segmentation model on a sequence of medical segmentation tasks [Gao26; Gao+26b; Gao+26a]. All three use the 2D model CLIPSeg, treat each dataset or dataset subset as a task, mix endoscopy, dermoscopy, ultrasound and X-ray images, and train on the full training sets [Gao26; Gao+26b; Gao+26a]. MedCRP-CL and MedPEFT-CL take their text prompts from earlier work on medical vision–language segmentation, which includes descriptive sentences per image [Gao26; Gao+26a]. PA-EWC uses templated phrases that range from class names to combinations of visual, spatial and medical descriptions [Gao+26b].

MedCRP-CL clusters the tasks by the CLIP embeddings of their text prompts and trains one LoRA module per cluster, with EWC within each cluster [Gao26]. On 16 tasks, its mean DSC after the sequence was 73.3 % with 4.1 % forgetting, compared with 65.3 % and 7.1 % for MoE-Adapters and 48.0 % and 28.3 % for sequential fine-tuning [Gao26]. PA-EWC weights the EWC penalty of different parameter groups by the complexity of the text prompts [Gao+26b]. On five tasks, it reports a higher mean DSC and less forgetting than ZSCL and MoE-Adapters, although the averages in its summary table do not match its per-task results [Gao+26b, Tables 3 and 5]. MedPEFT-CL adds LoRA to both CLIPSeg encoders, reuses the adapter of an earlier task when the text embeddings of the two tasks are similar, and replays the hardest 15 % of each task's training samples [Gao+26a]. On a sequence of ten tasks, its mean DSC was 78.19 %, against 74.15 % for MoE-Adapters and 81.02 % for one model per task [Gao+26a]. For MedCRP-CL and MedPEFT-CL, the papers do not state which adapter is used at test time, so it is unclear whether the adapter is selected by the task identity or by the text prompt [Gao26; Gao+26a].

2.5 Summary and Research Gap

Text-promptable 3D models such as SAT and VoxTell are trained once on many structures, and VoxTell's zero-shot results on the knee MRI of SKM-TEA show that this generalisation does not extend to the knee structures (Section 2.1). Few-shot fine-tuning adapts 3D segmentation models to one new target with few labelled volumes and evaluates only that target (Section 2.2). Continual-learning methods for pretrained vision–language models protect the pretrained representation while tasks are added and were evaluated on classification or on tasks with text output rather than on segmentation (Section 2.3). Among

Table 2.1: Setting of this thesis compared with the closest works. Pretrained model: whether the task sequence starts from a foundation model (Section 2.4). Few-shot: labelled samples per task (no: full training sets). Control task: a held-out task evaluated after the sequence to measure the loss of pretrained ability (Section 1.4). [a]Only for sequential fine-tuning, task-specific LoRA and shared LoRA.

Work	Text prompt	3D	Pretrained model	Few-shot	Control task
Zhang et al., Liu et al. [Zha+23; Liu+24]	Fixed CLIP class names	yes	no (Swin UNETR)	no	no
MedSeqFT [Ye+25]	none	yes	yes (VoCo, UniMiSS+)	no	no
Chen et al. [Che+26b]	none	yes	yes	16, 32, 64	no
MedCRP-CL [Gao26]	Descriptive sentences	no	yes (CLIPSeg)	no	no
PA-EWC [Gao+26b]	Templated phrases	no	yes (CLIPSeg)	no	no
MedPEFT-CL [Gao+26a]	Descriptive sentences	no	yes (CLIPSeg)	no	no
This thesis	Free text, one fixed prompt per task	yes	yes (VoxTell)	5, 10, 20[a]	yes (AeroPath)

the continual-learning works for medical image segmentation, only Chen et al. adapt with few labelled samples, and those conditioned on text are either 2D or limited to a fixed set of class names (Section 2.4).

In our search[1] we found no work that adapts a pretrained free-text promptable 3D segmentation model to a sequence of new structures with few labelled volumes per task. The closest works each lack part of this setting: MedCRP-CL, PA-EWC and MedPEFT-CL adapt the 2D model CLIPSeg with full training sets (Section 2.4.3), and Chen et al. adapt a 3D model with few samples but without text prompts and with the task identity given at inference (Section 2.4.2). None of these four works measures how much of the pretrained model's ability outside the task sequence is kept at the end of the sequence. SPU measures such a loss for CLIP (Section 2.3), and CA-SAM and SAM-Med3D-MoE for models prompted with points or boxes (Section 2.4.2).

Unlike the closest works in Table 2.1, which each propose a new method, this thesis adapts existing methods to VoxTell and compares the resulting strategies, as Lifelong nnU-Net and the Beyond Forgetting benchmark compare existing methods for networks without a foundation model (Section 2.4.1). Table 2.1 contrasts our setting with the closest works.

defense: meta-net and gradient scaling a new method? (tracker)

[1]Keyword queries in OpenAlex, PubMed and arXiv and a search of the works citing VoxTell and SAT, run on 23 October 2026; the 461 hits contained 332 unique titles, of which 43 passed a keyword filter on title or abstract; these 43 and the 23 works citing VoxTell were screened by title and abstract.

Table 2.2 summarises the data, approach and best reported result of all works reviewed in this chapter.

Table 2.2: Works reviewed in this chapter, grouped by section. Works cited only for a definition, a method or a single finding are not listed. Results are given as reported by the authors, in the paper's metric, with the main comparison in parentheses; settings are named in the paper's own terms. FT: fine-tuning; seq.: sequential; TIL, DIL, CIL: task-, domain- and class-incremental learning (Section 3.2); MTIL: the eleven-dataset benchmark of ZSCL; FSCIL: few-shot class-incremental learning.

Work	Data	Approach and setting	Metric: best reported result
Text-promptable medical image segmentation (Section 2.1)			
SegVol [Du+24]	25 CT datasets; 3D	Points, boxes, class names in a template; not continual	Median DSC, AMOS22: 0.8593, box + text (SAM-Med2D 0.6668)
CLIP-Driven Universal Model [Liu+23a]	14 CT datasets; 3D	Fixed class names in a CLIP template; not continual	DSC, BTCV: 86.13 % (Swin UNETR 82.06 %)
SAT [Zha+25d]	72 CT, MRI, PET datasets; 3D	Terms from a fixed vocabulary; not continual	Mean DSC, all classes: 78.81 % (72 nnU-Nets 80.54 %)
VoxTell [Rok+26]	>62,000 CT, MRI, PET volumes; 3D	Free text; not continual	Mean zero-shot DSC, 11 unseen datasets: 70.85 % (SAT 51.23 %); SKM-TEA 10.72 %
Few-shot adaptation of segmentation models (Section 2.2)			
Silva-Rodríguez et al. [Sil+25]	TotalSegmentator, FLARE'22; 3D	Parameter-efficient vs. full FT; one target at a time; 1, 5, 10 (30) volumes	Mean DSC, 9 organs, 10 shots: 77.79 %, LoRA (full FT 76.93 %)
CLIP-LoRA [Zan+24]	11 classification datasets; 2D	Rank-2 LoRA in both CLIP encoders; 1–16 shots per class; not continual	Top-1 accuracy, mean: 72.5 % (1 shot), 83.0 % (16 shots)
Continual learning for pretrained vision–language models (Section 2.3)			
GIFT [Wu+25]	MTIL; 2D	Replay of generated images, full FT; MTIL, CIL	Transfer / Last, Order I: 69.3 / 86.0 % (ZSCL 68.1 / 83.6 %)
ZSCL [Zhe+23]	MTIL; 2D	Distillation on reference data, weight averaging, full FT; MTIL	Transfer / Last, Order I: 68.1 / 83.6 % (seq. FT 44.6 / 77.3 %)
PRIMED [Che+26c]	HieraMedTransfer, MedXtreme; 2D	Distillation on retrieved image–caption pairs; TIL, DIL	Transfer / Last, Order I: 58.3 / 82.1 % (ZSCL 57.7 / 77.6 %)
LiLoRA [Che+26a]	CVIT benchmark; 2D	Shared and task-specific LoRA parts; continual instruction tuning	Average accuracy, Single-type: 84.88 % (separate LoRAs 88.13 %, seq. LoRA 46.21 %)

continued on next page

Table 2.2 (continued)

Work	Data	Approach and setting	Metric: best reported result
MSLoRA-CR [Zha+25a]	Nine medical datasets; 2D	One LoRA per task, branches summed; TIL	Sum of task metrics: 995.21 (separate LoRAs 970.29)
CPE-CLIP [DAl+23]	CIFAR100, miniImageNet, CUB200; 2D	Prompt tokens in both encoders, gradient scaling; FSCIL	Average accuracy, CIFAR100: 83.42 % (zero-shot 70.58 %)
Continual learning for medical image segmentation (Section 2.4)			
Bera et al. [Ber+23]	Prostate, spleen, hippocampus datasets; 3D	Replay of selected crops; DIL, "task incremental"	Average DSC, three organs: 83.3 ± 1.3 % (random replay 78.3 ± 1.3 %)
C^2GR [Li+26]	20 tasks, 8 modalities; 2D	Diffusion replay, fixed task template, trained from scratch; TIL	Mean DSC, final: 83.62 % (joint 86.06 %)
Zhang et al., Liu et al. [Zha+23; Liu+24]	Private CT, BTCV, LiTS; 3D	Fixed CLIP class names, head per class, pseudo-labels; class-incremental	DSC, BTCV organs / LiTS tumour: 0.817 / 0.466 (PLOP 0.799 / 0.362)
Lifelong nnU-Net [Gon+23]	Hippocampus, prostate, cardiac MRI; 3D	Five methods compared; DIL	Rehearsal forgets least (DSC)
Beyond Forgetting [Wan+26a]	Prostate MRI, MMWHS, four organs; 2D	16 methods; Domain-, Class-, Organ-CL	Mean final DSC, Domain-CL: 0.789, PNN (joint 0.830)
SAM-Med3D-MoE [Wan+24]	SPPIN2023, poorly segmented classes; 3D	Points or boxes; decoder experts with a gate; not sequential	DSC, box prompts: poorly segmented 0.564 (0.532); without expert 0.400 (0.541)
CA-SAM [Wan+26b]	Nine datasets; 2D	Boxes; alignment layer per task, autoencoder router	Final IoU: 76.12 % (joint 78.43 %)
MedSeqFT [Ye+25]	Five CT, five MRI tasks; 3D	Seq. FT with distillation and LoRA refinement	Mean DSC, CT tasks: 68.3 % (parallel FT 65.3 %)
Chen et al. [Che+26b]	BraTS, IXI; 3D	LoRA and head per task, task identity given; 16, 32, 64 samples per task; task-incremental	DSC, tumour, 32 samples: 0.60 ± 0.08 (seq. FT 0.16 ± 0.19 after task 2)
MedCRP-CL [Gao26]	16 tasks, four modalities; 2D	CLIPSeg; LoRA per prompt cluster, EWC	Mean DSC / forgetting: 73.3 / 4.1 % (MoE-Adapters 65.3 / 7.1 %)

continued on next page

Table 2.2 (continued)

Work	Data	Approach and setting	Metric: best reported result
PA-EWC [Gao+26b]	Kvasir, ISIC, CheXlocalize, BUSI, CAMUS; 2D	CLIPSeg; EWC weighted by prompt complexity	Mean DSC: 75.34 % (ZSCL 73.02 %)[a]
MedPEFT-CL [Gao+26a]	Ten datasets; 2D	CLIPSeg; LoRA reused by prompt similarity, replay, EWC	Mean DSC: 78.19 % (MoE-Adapters 74.15 %)

[a] Summary-table value; it does not match the paper's own per-task results.

Chapter 3

Theoretical Background

This chapter introduces the concepts on which Chapter 5 builds. Section 3.1 describes volumetric segmentation networks, attention and the free-text promptable model VoxTell. Section 3.2 states the continual-learning problem in the notation used throughout the thesis and describes the published methods from which our strategies are adapted. Section 3.3 defines the evaluation matrix and the metrics with which we measure adaptation and forgetting.

3.1 Free-Text Promptable Volumetric Segmentation

We consider the segmentation of one target structure in a 3D image volume. A single-channel volume $\boldsymbol{X}$ assigns an intensity to every voxel v of its voxel set Ω, and the label mask $\boldsymbol{Y}$ assigns $y_v = 1$ to the voxels of the structure and $y_v = 0$ to all others. For this structure, a segmentation network predicts one logit z_v per voxel. The logistic function σ turns each logit into a foreground probability $\hat{y}_v = \sigma(z_v)$, and thresholding these probabilities gives the binary prediction $\hat{\boldsymbol{Y}}$. In a free-text promptable model, a text prompt p names the target structure, so the structure is not tied to a fixed output channel of the network. One network with parameters θ can then serve all target structures and computes the logit map $\boldsymbol{Z} = f_\theta(\boldsymbol{X}, p)$ with entries z_v. We write $\mathcal{L}_{\text{seg}}$ for the segmentation loss that compares $\boldsymbol{Z}$ with $\boldsymbol{Y}$.

VoxTell implements f_θ with a volumetric encoder–decoder network and attention between the text prompt and the image features. Sections 3.1.1 and 3.1.2 describe these two building blocks, and Section 3.1.3 describes VoxTell itself.

3.1.1 Encoder–Decoder Networks for Volumetric Segmentation

The 3D U-Net [Çiç+16] is an encoder–decoder network for volumetric segmentation. Its encoder computes features at successively lower resolutions in a sequence of stages, and its decoder upsamples them back to the input resolution. Skip connections pass the encoder features of every stage to the decoder stage of the same resolution, so the decoder can use spatial detail that was lost by downsampling. We write $\boldsymbol{F}_s$ for the encoder features of stage s and C_s for their number of channels, with $s = 0$ at the full resolution and larger s at coarser resolutions. The authors of VoxTell regard this design as still the strongest architecture for 3D medical image segmentation [Rok+26, p. 4].

A whole volume and its intermediate features often do not fit into the memory of a GPU, so such networks are trained on patches cut from the volumes. At inference, a sliding window applies the network to overlapping patches, and their predictions are combined into one prediction per volume. The nnU-Net framework configures these steps automatically for a given dataset [Ise+21].

3.1.2 Attention

Attention lets each of a set of query vectors collect information from a set of source vectors, weighting each source by how well it matches the query [Vas+17]. Let the rows of $\boldsymbol{U} \in \mathbb{R}^{n_q \times d}$ be n_q query vectors and the rows of $\boldsymbol{S} \in \mathbb{R}^{n_s \times d}$ be n_s source vectors of dimension d. Multi-head attention projects the query vectors to queries with a learned matrix $\boldsymbol{W}_q \in \mathbb{R}^{d \times d}$ and the source vectors to keys and values with $\boldsymbol{W}_k, \boldsymbol{W}_v \in \mathbb{R}^{d \times d}$, and it splits each projection into n_h heads of dimension $d_h = d/n_h$. We write all projections as maps $\boldsymbol{x} \mapsto \boldsymbol{W}\boldsymbol{x}$ of column vectors, so they appear transposed when applied to the row-wise inputs $\boldsymbol{U}$ and $\boldsymbol{S}$. Let $\boldsymbol{W}_q^{(i)}, \boldsymbol{W}_k^{(i)}, \boldsymbol{W}_v^{(i)} \in \mathbb{R}^{d_h \times d}$ denote the rows of the projection matrices that belong to head i. Head i and the output of multi-head attention are then

$$\begin{aligned} \mathrm{head}_i &= \mathrm{softmax}\left(\frac{\boldsymbol{U}\boldsymbol{W}_q^{(i)\top} \left(\boldsymbol{S}\boldsymbol{W}_k^{(i)\top}\right)^\top}{\sqrt{d_h}} \right) \boldsymbol{S}\boldsymbol{W}_v^{(i)\top}, \\ \mathrm{MHA}(\boldsymbol{U}, \boldsymbol{S}) &= \left[\mathrm{head}_1, \ldots, \mathrm{head}_{n_h} \right] \boldsymbol{W}_o^\top, \end{aligned} \tag{3.1}$$

where the softmax is taken over the n_s sources for each query, $[\cdot]$ concatenates the heads along the feature dimension and $\boldsymbol{W}_o \in \mathbb{R}^{d \times d}$ is the output projection. Each query vector thus receives a weighted mean of the values, and the weights are given by the similarity

of its query to every key. The result does not depend on the order of the sources, so a positional encoding is usually added to them [Vas+17].

In self-attention, the query vectors and the source vectors are the same ($\boldsymbol{S} = \boldsymbol{U}$). In cross-attention, they come from different inputs, for example a text embedding and image features. Keys and values are computed from each source vector separately, so further rows appended to $\boldsymbol{S}$ give every query additional sources to attend to without any change to the projections. The prompt-tuning strategy (Section 5.4.5) relies on this property.

A transformer decoder layer combines attention blocks with a feed-forward network, two linear layers with a nonlinearity between them that are applied to every query separately [Vas+17]. Each block is wrapped in a residual connection and a layer normalisation. In the original transformer, each decoder layer applies self-attention to its queries and then cross-attention to the output of the transformer encoder [Vas+17].

3.1.3 VoxTell

VoxTell is a vision–language model that maps a 3D volume and a free-text prompt to a segmentation mask of the structure that the prompt describes. The prompt can range from a single word to a clinical sentence [Rok+26, p. 1]. The model has four parts (Figure 3.1): a frozen text encoder, a convolutional image encoder, a transformer prompt decoder that relates the text to the image, and a convolutional decoder into which the text is fused at every resolution [Rok+26, Sec. 3]. Section 5.3 describes where the version used in this thesis differs from the paper.

VoxTell is built on the nnU-Net framework [Rok+26, Suppl. p. 1], and its image encoder is the residual encoder of the nnU-Net configuration ResEnc L [Ise+24, Sec. 3.1]. The encoder has six stages ($s = 0, \ldots, 5$) with 32, 64, 128, 256, 320 and 320 channels [Rok+26, Suppl. p. 1]. For a patch of 192^3 voxels, the coarsest stage has 6^3 voxels. The text encoder is the embedding model Qwen3-Embedding-4B [Zha+25b], which maps the text prompt p to an embedding $\boldsymbol{q} = f_{\text{text}}(p)$ [Rok+26, Eq. 2]. Before encoding, a fixed instruction is placed in front of the prompt. The instruction asks the encoder to retrieve the anatomical entity and location that the prompt describes [Rok+26, Suppl. p. 1]. The text encoder is frozen, so the parameters θ of VoxTell comprise all parameters except those of f_{text}.

The prompt decoder consists of $L_{\text{dec}} = 6$ transformer decoder layers of the kind described in Section 3.1.2, with eight heads and dimension $d = 2048$ [Rok+26, Sec. 3]. Its query is the text embedding $\boldsymbol{q}$, and its keys and values are the features $\boldsymbol{F}_5$ of the coarsest stage, the bottleneck [Rok+26, Eq. 3]. For each decoder stage s, a two-layer perceptron

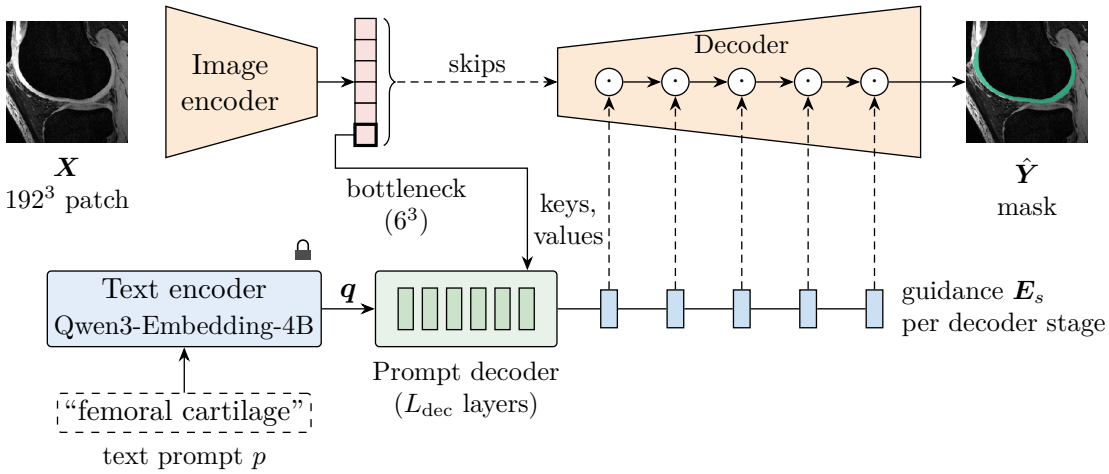


Figure 3.1: Architecture of VoxTell (adapted from [Rok+26, Fig. 2]). Each of the six stages of the image encoder passes its features to the decoder stage of the same resolution (skips). The text encoder embeds the text prompt into the query $\boldsymbol{q}$ and is frozen (lock). In the prompt decoder, $\boldsymbol{q}$ attends to the bottleneck features. The output of the prompt decoder is mapped to a guidance matrix $\boldsymbol{E}_s$ for each of the five decoder stages and fused into that stage (circles, Equation (3.2)). The input is a knee MRI patch from SKM-TEA with the text prompt "femoral cartilage". For illustration, the output shows the reference mask, not a prediction.

maps the output of the prompt decoder to a text-guidance matrix $\boldsymbol{E}_s \in \mathbb{R}^{G \times C_s}$ [Rok+26, Sec. 3, Suppl. p. 1]. Its $G = 32$ rows are guidance vectors with the channel number C_s of that stage. The guidance depends on both the prompt and the image, because the query attends to the image features before it is mapped to the guidance.

The decoder upsamples the features stage by stage and fuses the guidance into every stage [Rok+26, Eqs. 4–6]. Starting from $\boldsymbol{H}_5 = \boldsymbol{F}_5$, it computes for $s = 4, \ldots, 1$

$$\boldsymbol{H}'_s = \text{ConvBlock}\left(\left[\text{Up}(\boldsymbol{H}_{s+1});\, \boldsymbol{F}_s\right]\right), \qquad \boldsymbol{H}_s = \left[\boldsymbol{H}'_s;\, \boldsymbol{E}_s \odot \boldsymbol{H}'_s\right], \tag{3.2}$$

where Up upsamples to the resolution of stage s, $[\cdot\,;\cdot]$ concatenates along the channel dimension, and $\boldsymbol{E}_s \odot \boldsymbol{H}'_s$ denotes the G maps of dot products between each guidance vector and the C_s-dimensional feature vector at every voxel. Each map thus shows how well the features at every voxel match one guidance vector, and the G maps enter all following convolutions as additional channels. At the full resolution, $s = 0$, the decoder features are combined with the guidance once more to give the output logits $\boldsymbol{Z}$.

VoxTell was trained with the sum of the binary cross-entropy and the Dice loss [Rok+26, Sec. 3]. With deep supervision, the loss was applied to the outputs of all five decoder

stages, with weights from 1 at the full resolution down to 1/16 as in nnU-Net [Rok+26, Eq. 7, Suppl. p. 1]. Training used patches of 192^3 voxels, of which 85 % were sampled to contain foreground [Rok+26, Suppl. p. 1]. It used the data augmentation of nnU-Net without left–right mirroring and SGD with an initial learning rate of 10^{-4} and polynomial decay. Each training image was paired with three prompts, two naming structures present in the image and one naming an absent structure, so that the model also learns to return an empty mask. Each prompt was the default name of its concept with probability 0.25 and a rephrased variant otherwise [Rok+26, Suppl. p. 1].

3.2 Continual Learning

This section states the continual-learning problem in the notation of this thesis and defines three common continual-learning scenarios. It then describes the published methods on which our strategies are based, as they were originally proposed. These are low-rank adaptation, knowledge distillation with ZSCL and prompt learning with CoCoOp and CPE-CLIP. Section 5.4 presents the corresponding strategies in the same order and explains how we adapted these methods to VoxTell and where our versions differ from the originals.

3.2.1 Problem Setting and Scenarios

In continual learning, a model is trained on a sequence of tasks $t = 1, \ldots, T$, one after the other. Training on task t starts from the parameters θ_{t-1} reached after the previous task, uses only the training data $\mathcal{D}_t$ of task t and yields the parameters θ_t. In our setting, the sequence starts from the pretrained parameters θ_0, and each task has its own text prompt p_t. We index the resulting models by the stage j of the sequence: stage $j = 0$ is the pretrained model, and stage $j = t$ is the model after training on task t.

Van de Ven et al. distinguish three continual-learning scenarios by whether the task identity of an input is given at inference and, if it is not, whether the model has to determine it [vdVen+25]. In task-incremental learning, the task identity is given at inference. In domain-incremental learning, the possible outputs stay the same while the input data changes, and the model is not told the domain of an input. In class-incremental learning, the model has to distinguish all classes seen so far, including classes it never saw together in training, and thereby also has to infer the task [vdVen+25].

3.2.2 Low-Rank Adaptation

Low-rank adaptation (LoRA) keeps a pretrained weight matrix $\boldsymbol{W}_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ frozen and learns an additive update of low rank [Hu+22, Sec. 4.1],

$$\boldsymbol{W} = \boldsymbol{W}_0 + \gamma_r \, \boldsymbol{B}\boldsymbol{A}, \qquad \boldsymbol{B} \in \mathbb{R}^{d_{\text{out}} \times r}, \quad \boldsymbol{A} \in \mathbb{R}^{r \times d_{\text{in}}}, \quad r \ll \min(d_{\text{in}}, d_{\text{out}}), \tag{3.3}$$

where only $\boldsymbol{A}$ and $\boldsymbol{B}$ are trained and γ_r is a scaling factor. $\boldsymbol{A}$ is initialised randomly and $\boldsymbol{B}$ with zeros, so the update is zero at the start and training begins from the pretrained model. After training, $\gamma_r \boldsymbol{B}\boldsymbol{A}$ can be added to $\boldsymbol{W}_0$, so the adapted model has no additional inference cost, and switching to another task only requires exchanging the update [Hu+22, Sec. 4.1]. Hu et al. applied LoRA to the projection matrices of self-attention (Equation (3.1)) and, comparing choices under a fixed number of trainable parameters, found that adapting the query and value projections together gave the best performance overall [Hu+22, Sec. 4.2, 7.1, Table 5].

In the original formulation, $\gamma_r = \alpha/r$ with a constant hyperparameter α [Hu+22, Sec. 4.1]. Kalajdzievski showed that this factor makes the gradients of the adapter collapse as the rank grows, so that large ranks learn no better than small ones, and proved that only a factor proportional to $1/\sqrt{r}$ keeps the outputs and gradients of the adapter stable for all ranks [Kal23, Sec. 2.1, 3]. The resulting rank-stabilised LoRA (rsLoRA) sets

$$\gamma_r = \frac{\alpha}{\sqrt{r}}. \tag{3.4}$$

In fine-tuning experiments with a large language model and ranks up to 2048, rsLoRA improved with increasing rank while standard LoRA did not, and the gradient norms showed the same pattern with SGD as with AdamW [Kal23, Sec. 4].

3.2.3 Knowledge Distillation and ZSCL

In knowledge distillation, a student model is trained to reproduce the output distribution of a teacher model, where a temperature in the softmax softens the distribution [Hin+15, Sec. 2]. In continual learning, distillation serves as a regulariser. Learning without Forgetting (LwF) trains on the data of the new task only and uses the outputs of the model before this task as soft labels for the earlier tasks [Li+18] [De +22, Sec. 3.2.1]. The teacher is thus the previous model, and the distillation data are those of the current task.

ZSCL changes two choices of LwF to preserve CLIP's zero-shot transfer during continual learning [Zhe+23, Sec. 1]. The teacher is the initial pretrained model instead of the previous one, and the distillation data are a separate reference dataset of unlabelled public images and arbitrary texts instead of the current task's data [Zhe+23, Sec. 3.2]. For each image, a softmax turns CLIP's similarities to a set of texts into a distribution, and the loss compares the distributions of student and teacher by a cross-entropy [Zhe+23, Sec. 3.2, Eqs. 2–3]. A second term does the same for each text, with a distribution over the images of a batch, and both terms are added to the task loss with a weight λ [Zhe+23, Sec. 3.2, Eq. 4]. In addition, ZSCL limits the change of the parameters themselves. The full method adds a weight-consolidation loss, a penalty on the squared distance between the parameters and a reference set of parameters [Zhe+23, Sec. 3.3, Eq. 5, Table 2]. Every I_{we} iterations, it also updates a weight ensemble, the running mean of the parameters along the training trajectory, starting from θ_0,

$$\hat{\theta}_\nu = \frac{1}{\nu+1}\,\theta + \frac{\nu}{\nu+1}\,\hat{\theta}_{\nu-1}, \qquad \hat{\theta}_0 = \theta_0, \tag{3.5}$$

where θ are the current parameters and ν counts the ensemble updates [Zhe+23, Sec. 3.3, Eq. 7].

3.2.4 Prompt Learning and CPE-CLIP

Prompt learning adapts a frozen vision–language model by training a few input vectors instead of its weights. CoOp replaces the hand-written context words of a text prompt, such as "a photo of a", by learnable context vectors in front of the class name [Zho+22b] [Zho+22a, Sec. 3.2]. CoCoOp adds a small network, the meta-net, which computes a token from the image features and adds it to every context vector, so that the context vectors depend on the input image [Zho+22a, Sec. 3.3]. Its authors report that these image-dependent context vectors generalise better to classes not seen in training [Zho+22a, Sec. 4.1].

CPE-CLIP applies prompt learning to few-shot class-incremental learning [DAl+23, Sec. 1, 3]. In this setting, a base session with many classes and examples is followed by sessions that each add a few classes with few examples, and the task identity is not given at evaluation [DAl+23, Sec. 1, 3]. CPE-CLIP keeps CLIP frozen and learns prompt tokens in the layers of the language encoder up to a given depth, where each layer replaces the prompt tokens it receives by its own learned ones [DAl+23, Sec. 4, Eqs. 2–4]. The prompt tokens of the vision encoder are derived from those of the language encoder by a learned

linear projection [DAl+23, Sec. 4, Eqs. 6–8]. To limit forgetting, CPE-CLIP scales the gradients of the prompt tokens and of the projection in session t by

$$\beta_t = \frac{|\mathcal{C}_t|}{\sum_{\tau=0}^{t} |\mathcal{C}_\tau|}, \tag{3.6}$$

where $\mathcal{C}_t$ is the set of classes of session t, so that the prompt tokens change less as more classes have been learned [DAl+23, Sec. 4, Eqs. 11–13].

3.3 Evaluation Metrics

This section defines the segmentation metric and the continual-learning metrics computed from it.

The overlap between a binary prediction $\hat{\boldsymbol{Y}}$ and the label mask $\boldsymbol{Y}$ is measured with the DSC [Dic45],

$$\mathrm{DSC}(\hat{\boldsymbol{Y}}, \boldsymbol{Y}) = \frac{2\,|\hat{\boldsymbol{Y}} \cap \boldsymbol{Y}|}{|\hat{\boldsymbol{Y}}| + |\boldsymbol{Y}|}, \tag{3.7}$$

where $|\cdot|$ counts the foreground voxels of a mask and $\hat{\boldsymbol{Y}} \cap \boldsymbol{Y}$ denotes the voxels that are foreground in both. The DSC is 0 for disjoint masks and 1 for identical ones. The DSC is normalised by the sizes of the two masks and not by the size of the volume, so the many background voxels do not inflate it. This matters for structures as small as the knee cartilages and menisci (Section 4.1).

Evaluating the model on every task after every stage (Section 3.2.1) shows how training on later tasks affects earlier ones [Zhe+23, Fig. 4b]. We write $a_{i,j}$ for the mean DSC on the test set of task i after stage j, with $i = 1, \ldots, T$ and $j = 0, \ldots, T$. Together, the values $a_{i,j}$ form the evaluation matrix (Figure 3.2). The continual-learning metrics computed from this matrix were originally defined for classification accuracy, but Chaudhry et al. note that a task-specific measure such as the intersection over union of a segmentation can take the place of accuracy without changing the definitions [Cha+18, Sec. 3]. The diagonal entries $a_{i,i}$ show how well task i is learned when it is trained. The entries with $j > i$ show how much of this performance is kept after later tasks, and the entries with $j < i$ show the performance on task i before it is trained. Entries $a_{i,j} < a_{i,i}$ with $j > i$ therefore indicate catastrophic forgetting of task i (Section 1.2). The matrix does not show whether the model loses pretrained abilities outside the task sequence, because its rows cover only the tasks of the sequence. We therefore also evaluate the model on a control task, a held-out

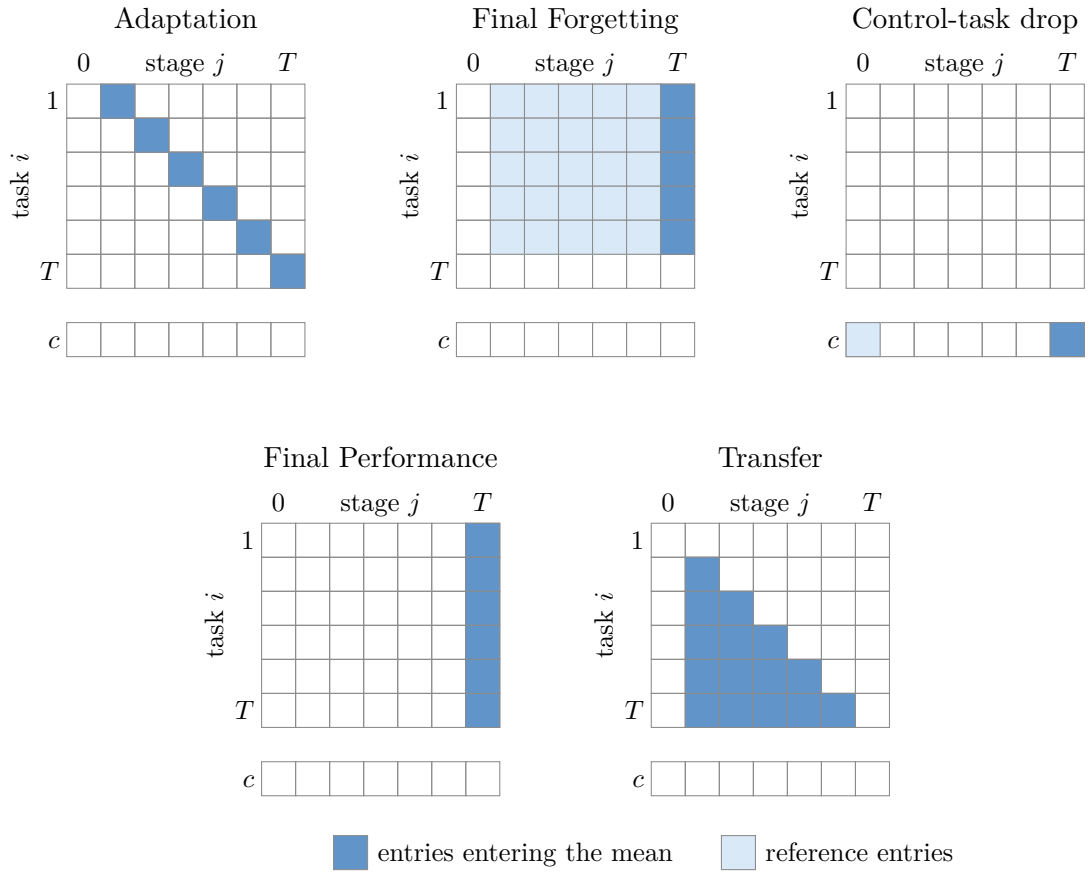


Figure 3.2: Entries of the evaluation matrix used by each metric (Equations (3.8) to (3.12)). Rows are the tasks $i = 1, \ldots, T$, and columns are the stages $j = 0, \ldots, T$, with stage 0 the pretrained model. The separate row c is the control task. Dark cells enter the mean of a metric. Light cells are the reference from which the dark cell of the same row is subtracted, namely their maximum for Final Forgetting and the pretrained value c_0 for the control-task drop.

task that is never trained on and is used only to measure the loss of pretrained ability (Section 4.2). We write c_j for its mean DSC after stage j.

We summarise the evaluation matrix with four metrics from the literature and the control task with a fifth. Figure 3.2 shows which entries each metric uses.

Adaptation. Adaptation measures how well each task is learned when it is trained,

$$\text{Adaptation} = \frac{1}{T} \sum_{i=1}^{T} a_{i,i}. \tag{3.8}$$

Each diagonal entry $a_{i,i}$ also appears in the intransigence measure of Chaudhry et al., which subtracts it from the performance on task i of a reference model trained on the data of tasks $1, \ldots, i$ together [Cha+18, Sec. 3, Eq. 4].

Final Forgetting. Final Forgetting is the average forgetting of Chaudhry et al. after the last task [Cha+18, Sec. 3, Eq. 3]. For each task before the last, it subtracts the performance at the end from the best performance at any earlier stage,

$$\text{Final Forgetting} = \frac{1}{T-1} \sum_{i=1}^{T-1} \Big(\max_{j \in \{1,\ldots,T-1\}} a_{i,j} - a_{i,T} \Big). \tag{3.9}$$

Taking the best earlier performance instead of the value right after training also counts gains that a task made while later tasks were trained and lost again by the end [Cha+18, Sec. 3]. Lower values are better, and a negative value means that training on later tasks improved earlier ones, which Chaudhry et al. call positive backward transfer [Cha+18, Sec. 3].

Control-task drop. The control-task drop follows SPU, which uses a control set to measure how much of a pretrained model's general ability is lost during continual learning [Zha+24, Sec. 5.1, Eq. 6]. SPU computes this loss as the accuracy of the frozen pretrained model on the control set minus that of the continually trained model [Zha+24, Sec. 5.1, Eq. 6]. We compute the drop with the DSC, between the pretrained model and the model after the last stage of a task sequence,

$$\text{Control-task drop} = c_0 - c_T, \tag{3.10}$$

so a positive value is a loss and lower values are better. SPU samples its control set from the pretraining distribution [Zha+24, Sec. 3]. Our control task instead comes from a dataset that VoxTell was not trained on, and its target structure is well represented in VoxTell's training data (Section 4.2).

Final Performance. Final Performance is the metric Last of ZSCL, the mean performance on all tasks after the last stage [Zhe+23, Sec. 4, Fig. 4b],

$$\text{Final Performance} = \frac{1}{T} \sum_{i=1}^{T} a_{i,T}. \tag{3.11}$$

A common reference for Final Performance is joint training, in which one model is trained on the data of all tasks at once. De Lange et al. describe joint training as a soft upper bound for continual learning [De +22, Sec. 5].

Transfer. Transfer is the metric of the same name in ZSCL [Zhe+23, Sec. 4, Fig. 4b]. For each task i, it averages the performance at the stages $1, \ldots, i-1$ before the task is trained, and it then averages these values over the tasks,

$$\text{Transfer} = \frac{1}{T-1} \sum_{i=2}^{T} \frac{1}{i-1} \sum_{j=1}^{i-1} a_{i,j}. \tag{3.12}$$

As in ZSCL, whose evaluation matrix has no entries for the model before the first training step, stage $j = 0$ is not included, so the first task does not enter Transfer. When the task sequence starts from a pretrained model, as here, an entry $a_{i,j}$ with $j < i$ reflects both the ability of the pretrained model on structure i and the effect of training on tasks $1, \ldots, j$, and Transfer does not separate the two. The pretrained-model reference row (Section 5.2) shows the first part alone.

Chapter 4

Data Description

This chapter describes the two public datasets used in this thesis. SKM-TEA [Des+21] provides the knee MRI volumes from which the six continual-learning tasks are built, and AeroPath [Stø+23] provides the chest CT volumes of the control task. Table 4.1 summarises both datasets. The chapter then describes how the volumes are split, how the few-shot training subsets are drawn, how the data are preprocessed, and which properties of the data limit the conclusions that can be drawn from them.

4.1 SKM-TEA

The Stanford Knee MRI with Multi-Task Evaluation (SKM-TEA) dataset contains knee MRI scans of 155 patients acquired at Stanford Healthcare [Des+21]. We chose SKM-TEA because VoxTell performs poorly on it (Section 1.1). SKM-TEA is distributed under the terms of the Stanford University School of Medicine and of the Lower Extremity Radiographs dataset [Des+21, App. A.5]. We signed the Stanford AIMI research-use agreement before downloading the data.

All scans were acquired with the 3D quantitative double-echo steady-state (qDESS) sequence on one of two 3 T GE MR750 scanners, with patient consent and approval of the Stanford University Institutional Review Board [Des+21]. qDESS acquires two inherently registered 3D echoes with different contrasts [Des+21]. SKM-TEA is distributed in two benchmark tracks, one based on raw k-space data and one based on the scanner-generated Digital Imaging and Communications in Medicine (DICOM) images [Des+21]. We use the DICOM track, which stores the images and segmentations of each scan in one Hierarchical Data Format version 5 (HDF5) file [Stab]. We use only the first of the two echoes, as

Table 4.1: Datasets used in this thesis. Values computed from the data and the official SKM-TEA annotation files (version 1.0.0), except the rows on imaging, source and access, which are taken from the dataset publications [Des+21; Stø+23].

	SKM-TEA	AeroPath
Role in this thesis	six continual-learning tasks	control task
Imaging	MRI, 3D qDESS, echo 1	CT angiography
Source	Stanford Healthcare	St. Olavs Hospital
Volumes (patients)	155 (155)	27 (27)
With pathology label[a]	130 of 155	–
Split (train/val/test)	108/23/24	19/4/4
Volume size (voxels)	512 × 512 × 144–168	487–512 × 407–512 × 241–843
Voxel size (mm)	0.3125 × 0.3125 × 0.8[b]	0.56–0.82 × 0.56–0.82 × 0.50–1.25
Access	github.com/StanfordMIMI/skm-tea	github.com/raidionics/AeroPath

[a] At least one pathology bounding box in the official SKM-TEA annotation files (cartilage lesion, effusion, meniscal tear, ligament tear).
[b] From the official annotation files; the NIfTI files used in this thesis store an identity affine (1 mm).

the dataset authors do in the segmentation part of their tutorial [Staa]. The dataset also provides versioned annotation files that list the scans of each split defined by the dataset authors, with metadata such as the voxel size [Stab]. All 155 volumes have a voxel size of $0.3125 \times 0.3125 \times 0.8$ mm (Table 4.1).

Six tissues were segmented manually on all DICOM images: patellar cartilage, femoral cartilage, medial and lateral tibial cartilage, and medial and lateral meniscus [Des+21]. The segmentations were drawn slice by slice in the sagittal plane and then corrected for volumetric consistency in the axial and coronal planes [Des+21, App. A.4]. Each scan was labelled by one of two researchers with three to four years of experience in knee MRI interpretation, supervised by two board-certified musculoskeletal radiologists [Des+21, App. A.4]. The dataset therefore contains no information on the agreement between annotators. The dataset also provides bounding boxes for sixteen pathology categories, which we do not use [Des+21]. According to the annotation files, 130 of the 155 scans contain at least one of these pathologies.

Each of the six segmented structures defines one task. The text prompt of a task is the name of its structure in lower case: “patellar cartilage”, “femoral cartilage”, “medial tibial cartilage”, “lateral tibial cartilage”, “medial meniscus” and “lateral meniscus”. The same text prompt is used for training and evaluation, and its wording is not varied. Of the six

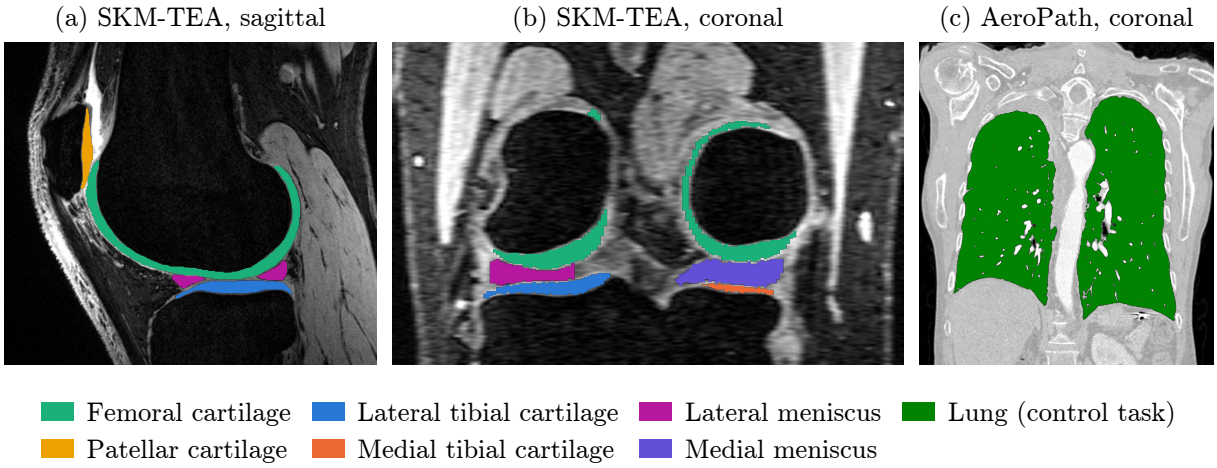


Figure 4.1: Example volumes with their labels. (a) Sagittal and (b) coronal slice of SKM-TEA volume MTR_005, cropped to the knee joint; each slice shows only the structures it intersects. In (a), anterior is on the left. In (b), the lateral side is on the left, and the image is coarser along the left–right axis, the slice direction. (c) Coronal slice of AeroPath volume 1 with the lung label of the control task. Both example volumes belong to the test split.

structures, the four cartilages are among VoxTell's training labels, whereas the two menisci are not [Rok+26, Suppl. Table 10]. Since all six structures are labelled in every volume, the six tasks share the same 155 volumes and differ only in the target structure. The structures are small, with median volumes from 1.90 cm^3 (lateral meniscus) to 10.99 cm^3 (femoral cartilage), and lie next to each other in the knee joint (Figure 4.1). The order in which the tasks are trained is part of the experimental setup and is given in Section 6.1.2.

4.2 AeroPath

AeroPath contains 27 contrast-enhanced chest CT scans (computed tomography angiography (CTA)) of patients undergoing diagnostic tests for lung cancer, acquired with a thorax lung protocol at St. Olavs Hospital in Trondheim, Norway [Stø+23]. Data collection was approved by the regional ethics committee, and the patients gave written informed consent [Stø+23]. The dataset is distributed under the Creative Commons Attribution 4.0 licence [Hof+23]. AeroPath provides the control task (Section 5.1), for which we use only the lung label of each volume, with the text prompt "lung". Neither the publication nor the dataset repository or the Zenodo record documents how the lung labels were created, so it is unknown whether they are manual or automatic references. AeroPath is not part

of VoxTell's training data [Rok+26, Suppl. Table 7], whereas "lung" is a VoxTell training label in 3,427 training cases [Rok+26, Suppl. Table 10].

4.3 Data Splits and Few-Shot Subsets

We split each dataset once, at random. After shuffling the volumes with a fixed random seed (42), we assign the first 70 % of the shuffled list to the training split, the next 15 % to the validation split and the remaining volumes to the test split. This gives 108 training, 23 validation and 24 test volumes for SKM-TEA and 19, 4 and 4 volumes for AeroPath. Of the AeroPath volumes, we use only the 4 test volumes, to evaluate the control task. Because the six tasks share the same volumes, all tasks and all experiments in this thesis use the same split. Each SKM-TEA patient contributes exactly one scan, so no patient appears in more than one split. The validation volumes are used to monitor training and, in an additional evaluation, to select the checkpoint with the lowest validation loss (Section 5.3). All reported results are computed on the test volumes.

SKM-TEA also defines an official split of 86 training, 33 validation and 36 test scans, whose test set consists of the 36 patients who received additional arthroscopic surgery [Des+21]. We use our own random split instead, which, unlike the official split, does not allow a comparison with published SKM-TEA results. For comparing continual-learning strategies with each other, it is sufficient that all strategies use the same split.

For the few-shot experiments, we reduce the training split to its first k volumes in the order of the shuffled list, where $k \in \{5, 10, 20\}$ is the number of labelled training volumes per task. The subsets are therefore nested, with the 5 volumes contained in the 10 and the 10 contained in the 20. For each k, all six tasks use the same k volumes. Only one subset is drawn for each k, so we cannot measure how much the results depend on which volumes are chosen. Runs with the full training split use all 108 training volumes. The validation and test volumes are the same for every k and for the full training split.

4.4 Preprocessing

We convert each SKM-TEA HDF5 file into two Neuroimaging Informatics Technology Initiative (NIfTI) files, one for the first echo and one for the labels. In the HDF5 file, the segmentation is stored as six binary channels, one per tissue. We merge them into a single label map, in which unlabelled voxels take the value 0 and labelled voxels the number of

their channel in the order of the dataset documentation [Stab]: 1 for patellar cartilage, 2 for femoral cartilage, 3 for medial tibial cartilage, 4 for lateral tibial cartilage, 5 for medial meniscus and 6 for lateral meniscus. The binary target mask of a task consists of the voxels with that task's label value. The NIfTI files are written with an identity affine, so they store neither the voxel size nor the orientation of the scan. Images and labels come from the same voxel grid, so they stay aligned. The AeroPath volumes and labels are used in their original NIfTI format.

When a volume is loaded, it is reoriented according to its affine and cropped to the bounding box of its non-zero voxels. The reorientation leaves the SKM-TEA volumes unchanged because their affine is the identity. According to the annotation files, 18 of the 155 SKM-TEA scans store the left–right axis in the opposite direction to the other 137. The intensities of each volume are then normalised to zero mean and unit standard deviation over the cropped volume, with the z-score normalisation of nnU-Net [Ise+21]. The same normalisation is applied to the MRI and the CT volumes. The volumes are not resampled, so the network receives each volume on its stored voxel grid, and the voxel size in the affine has no effect on the input. This preprocessing follows the configuration distributed with the VoxTell checkpoint, which uses z-score normalisation for all inputs, including CT, and a preprocessing plan without resampling.[1] Section 5.3 describes how patches are sampled during training and how predictions are assembled during evaluation.

[1] File `plans.json` of the checkpoint `voxtell_v1.1` in the Hugging Face repository `mrokuss/VoxTell`: normalisation scheme `ZScoreNormalization`, no target spacing, image reader `NibabelIOWithReorient`.

Chapter 5

Method

This chapter describes how we address the research question and the reasons for the main design choices. Section 5.1 defines the continual-learning setting, and Section 5.2 defines the protocol that measures adaptation and forgetting. Section 5.3 describes how VoxTell is adapted to a single task, Section 5.4 describes the strategies, each a variant of this adaptation procedure, and Section 5.5 compares their computational cost. The concepts used in this chapter are introduced in Chapter 3. Chapter 6 gives the values of all parameters and describes the experiments.

5.1 Continual-Learning Setting

We consider a sequence of $T = 6$ tasks built from SKM-TEA. Each task $t \in \{1, \dots, T\}$ is defined by one knee structure, its text prompt p_t and the binary masks of that structure (Section 4.1). We write $\mathcal{D}_t^{\text{tr}}$, $\mathcal{D}_t^{\text{val}}$ and $\mathcal{D}_t^{\text{te}}$ for the training, validation and test sets of task t, each consisting of the volumes of the corresponding split (Section 4.3) paired with the masks of structure t. Because the six tasks share the same volumes (Section 4.1), catastrophic forgetting shows up here as a loss of segmentation performance on earlier structures in the same images.

In every strategy, the text prompt names the target structure during training and at inference. Because each task has one fixed text prompt, the prompt also identifies the task. The model therefore never has to determine which of the learned structures is meant, as class-incremental learning would require. Nor is the setting domain-incremental, because the images stay the same and only the target changes from task to task. Of the three scenarios (Section 3.2.1), our setting is therefore closest to task-incremental learning. The

strategies differ in whether they also use the task identity to select parameters. Only task-specific LoRA (Section 5.4.2) receives the task identity t at inference and uses it to select the parameters trained for that task, which we call task-aware inference. All other strategies use one parameter set for all tasks.

The protocol has two variables that are set per run. The first is the task order: a run trains all tasks in one fixed order. Comparing runs that differ only in the task order shows whether the results depend on the order in which the structures are learned.

The second variable is the amount of labelled data. In a few-shot setting, every task is trained on the first k volumes of its training split (Section 4.3) instead of on the full split. We call k the number of shots, that is, the number of labelled training volumes per task. Within a run, k is the same for all tasks, and the validation and test sets do not depend on k. The task orders and values of k that we used, and the reasons for them, are given with the experiments in Section 6.1.2.

In addition to the six tasks, we evaluate a control task that is not part of the task sequence and is never trained on: segmentation of the lung in the AeroPath test volumes $\mathcal{D}_c^{\text{te}}$ with the text prompt $p_c =$ "lung". It measures how much of VoxTell's pretrained ability on structures outside the task sequence is lost during adaptation, following the control set of SPU [Zha+24]. We chose this task for two reasons. First, lung is a concept included in the training data of VoxTell, while AeroPath itself was not used for training (Section 4.2). The pretrained model segments the lung with a mean DSC of 0.979 on our four test volumes (Figure 6.3), so a loss of ability is clearly visible. Second, AeroPath is publicly available.

5.2 Evaluation Protocol

The protocol trains a strategy on the T tasks in sequence and evaluates the model after every stage j (Section 3.2.1) on every task and on the control task (Algorithm 1). After each stage j, we record the mean DSC $a_{i,j}$ on the test set $\mathcal{D}_i^{\text{te}}$ of every task $i \in \{1, \ldots, T\}$ and the mean DSC c_j on the control task. The values $a_{i,j}$ form the evaluation matrix with T rows and $T+1$ columns (Section 3.3), and the values c_j form an additional control row. We evaluate all tasks at every stage, including tasks that have not been trained yet ($i > j$), because these entries show how training on one structure affects the segmentation of structures that come later in the sequence. All metrics of this thesis are computed from these two quantities. Their definitions are given in Section 3.3, and Section 6.1.4 explains which of them we report and why.

Each entry is the per-volume DSC between the binary prediction and the label mask, averaged over the volumes of one test set. The predictions are obtained by sliding-window inference (Section 5.3). All entries of all runs use the same test volumes: the 24 SKM-TEA test volumes for the tasks and the 4 AeroPath test volumes for the control task (Section 4.3). Differences between entries, and between runs, therefore never come from different test data. We also keep the per-volume values, from which Section 6.1.4 estimates the variability of the metrics over test volumes.

Which parameters are evaluated at stage j depends on the strategy. All strategies except task-specific LoRA evaluate one parameter set, obtained after training on task j, on every task and on the control task. Task-specific LoRA holds one adapter per trained task and uses the task identity to choose among them (Section 5.4.2): a task $i \leq j$ is evaluated with the adapter trained on task i, a task $i > j$ with the adapter of the current task j, and the control task with the pretrained model without any adapter. As a consequence, the entries of an earlier task do not change once its adapter is trained, and the control task always gives the pretrained result. Low forgetting and the absence of a control-task drop are thus properties of this evaluation for task-specific LoRA, not measured effects.

defense: task-specific LoRA's forgetting and control-task drop are zero by construction; state this wherever its results are compared

As a reference, we apply the same metrics to the pretrained model itself, treating it as if it had been evaluated unchanged after every stage, that is, with $a_{i,j} = a_{i,0}$ and $c_j = c_0$ for all j. This pretrained-model reference shows whether a strategy moved away from the pretrained model at all and in which direction. Because the order of the columns matters for some metrics, we compute this reference once for each task order.

As an upper reference, we use joint fine-tuning, which trains one model on the training sets of all six tasks at once, in a single stage. Each training sample keeps the text prompt of its own structure. All other settings are those of sequential fine-tuning (Section 5.3), so the joint model receives the same number of updates as sequential fine-tuning over the whole sequence. After its single stage, it is evaluated on the test set of every task and on the control task. Without a task sequence, Adaptation, Final Forgetting and Transfer are not defined for it, so we compare it with the strategies on Final Performance, overall and per structure, and on the control-task drop. We do not compute the intransigence measure of Chaudhry et al. [Cha+18], because its reference for task i is a model trained jointly on tasks $1, \ldots, i$. Our joint model is trained on all six tasks and therefore provides this reference only for $i = T$.

Algorithm 1 distinguishes the parameters θ_j that are carried forward to the next task from the parameters $\tilde{\theta}_j$ that are evaluated. In the main evaluation, both are the param-

Algorithm 1 Continual adaptation and evaluation protocol. ADAPT adapts the model to one task (Algorithm 2); SELECT returns the parameters a strategy evaluates on a given task (the parameters of the stage for all strategies except task-specific LoRA, see text); EVALUATE returns the mean DSC over a test set.

Input: pretrained parameters θ_0; strategy $\mathcal{S}$; tasks $t = 1, \ldots, T$ in the chosen task order, each with text prompt p_t and sets $\mathcal{D}_t^{\text{tr}}, \mathcal{D}_t^{\text{val}}, \mathcal{D}_t^{\text{te}}$; control task with p_c and $\mathcal{D}_c^{\text{te}}$
Output: evaluation matrix $(a_{i,j})$ with $i = 1, \ldots, T$, $j = 0, \ldots, T$; control row (c_j)

$\tilde{\theta}_0 \leftarrow \theta_0$
for $j = 0$ **to** T **do**
  if $j > 0$ **then**
    $(\theta_j, \tilde{\theta}_j) \leftarrow \text{ADAPT}(\theta_{j-1}, j, \mathcal{S})$ {carried-forward and evaluated parameters}
  end if
  for $i = 1$ **to** T **do**
    $a_{i,j} \leftarrow \text{EVALUATE}(\text{SELECT}(\mathcal{S}, \tilde{\theta}_{0:j}, i), p_i, \mathcal{D}_i^{\text{te}})$
  end for
  $c_j \leftarrow \text{EVALUATE}(\text{SELECT}(\mathcal{S}, \tilde{\theta}_{0:j}, c), p_c, \mathcal{D}_c^{\text{te}})$
end for

eters after the last epoch of task j. The distinction matters only for the two additional evaluations, which use the checkpoint with the lowest validation loss (Section 5.3) and, for the distillation-based strategy, the weight ensemble (Section 5.4.4).

The protocol costs T calls of ADAPT (Section 5.3) plus the evaluations. With $T + 1$ stages, each evaluated on T task test sets and the control test set, the protocol runs sliding-window inference on $(T + 1)\,(T\,|\mathcal{D}^{\text{te}}| + |\mathcal{D}_c^{\text{te}}|)$ volumes, where $|\mathcal{D}^{\text{te}}|$ is the size of a task test set. For our sets, this gives $7 \cdot (6 \cdot 24 + 4) = 1036$ volume inferences per run, and the same number again for each additional evaluation. The evaluation time thus grows quadratically with the number of tasks, while the training time grows linearly. Evaluation needs memory for one parameter set and the prediction of one volume at a time; the stored results are the $(T + 1)^2$ means plus the per-volume values. The measured evaluation time per volume and per run is reported in Section 6.7.

5.3 Adapting VoxTell to One Task

All strategies adapt VoxTell to a task with the same procedure, ADAPT in Algorithm 2. A strategy changes this procedure at three defined points, which we describe in Section 5.4; this section describes the procedure itself.

We start every run from the released VoxTell checkpoint (version 1.1) [Rok+26], whose architecture is introduced in Section 3.1.3. We use VoxTell because it accepts free-text prompts and reached the highest mean zero-shot DSC in the comparison of its authors, ahead of SAT (Section 2.1). Figure 5.1 shows the model as it is used here, together with the parts that the strategies change. Two properties of the implementation matter for what follows and differ from the description in the VoxTell paper. First, the prompt decoder takes its keys and values not from the bottleneck but from the output of encoder stage 4, which has 12^3 voxels for a 192^3 input patch. Second, each of the L_{dec} prompt-decoder layers applies only a cross-attention block and a feed-forward block, each preceded by layer normalisation and followed by a residual connection. Every layer also contains a self-attention module, but the configuration of the released model never executes it, so its parameters are never updated. Before the first layer, the text embedding and the stage-4 features are projected to the model dimension d of the prompt decoder. After the last layer, one multilayer perceptron per decoder stage maps the output to the guidance $\boldsymbol{E}_s$ of that stage, $s = 0, \ldots, 4$. At stages 4 to 1, the guidance is split into G vectors whose dot products with the decoder features at every voxel are appended as G new channels; at stage 0, the full resolution, a single dot product yields the output logits. The text query $\boldsymbol{q}$ that enters the prompt decoder is the embedding of the text prompt computed by the text encoder Qwen3-Embedding-4B [Zha+25b], which stays frozen in every strategy.

Training uses patches of 192^3 voxels, the patch size of VoxTell. In every epoch, each training volume contributes one patch: with probability π_{fg} the patch is centred on a voxel drawn at random from the target structure, otherwise it is placed uniformly at random in the volume, and volumes smaller than the patch are padded with zeros. Each patch is paired with the text prompt of its task only. The batch size is one, so that a training step fits into the memory of one GPU and a full-data run fits into the 24-hour job limit of the cluster. The same time limit bounds the number of epochs E per task (Tables 6.2 and 6.10). To keep the experiments simple, we add neither prompts of absent structures nor data augmentation, unlike the pretraining of VoxTell (Section 3.1.3). We also compute the loss on the full-resolution output only, without deep supervision.

author: check the reason; full-data training took 19 to 37 hours, so several runs needed a second 24-hour job

The loss is the sum of the binary cross-entropy and the Dice loss, the combination VoxTell was pretrained with [Rok+26]. For a patch with voxel set Ω, output logits $\boldsymbol{Z}$ and label mask $\boldsymbol{Y}$, and with $\hat{y}_v = \sigma(z_v)$, where σ is the logistic function, the loss is

$$\mathcal{L}_{\text{seg}}(\boldsymbol{Z}, \boldsymbol{Y}) = -\frac{1}{|\Omega|} \sum_{v \in \Omega} \left[y_v \log \hat{y}_v + (1 - y_v) \log(1 - \hat{y}_v) \right] + 1 - \frac{2 \sum_{v \in \Omega} \hat{y}_v y_v + \epsilon}{\sum_{v \in \Omega} \hat{y}_v + \sum_{v \in \Omega} y_v + \epsilon}, \quad (5.1)$$

with a small constant $\epsilon = 10^{-5}$ that keeps the Dice term defined for empty masks. The Dice term is computed from the probabilities $\hat{y}_v$ instead of the binary prediction, so that it is differentiable.

The trainable parameters are optimised with stochastic gradient descent with Nesterov momentum μ and weight decay ω. The learning rate follows a polynomial schedule with exponent one: it decays linearly from η_0 to zero over the $U = E \cdot N_t$ iterations of a task and is updated after every iteration. Here, N_t is the number of training samples of task t per epoch. The optimiser, its momentum and the learning-rate schedule are created anew for every task. The optimiser (SGD), the initial learning rate, the patch size and the foreground fraction follow the pretraining of VoxTell [Rok+26, Suppl. p. 1]. The paper does not state the momentum, the weight decay or the exponent of the polynomial schedule. For momentum and weight decay, we use the defaults of nnU-Net, on which VoxTell builds. For the schedule, we use exponent 1 (linear decay), close to nnU-Net's default of 0.9. The values of all parameters are listed in Tables 6.2 and 6.3. All strategies are trained with these same settings, and neither the learning rate nor the number of epochs was tuned per strategy. Our results therefore compare the strategies under one shared training configuration, not each strategy at its best setting.

cite: nnU-Net v2 code, nnUNetTrainer defaults (momentum 0.99, Nesterov, weight decay 3e-5)

defense: no per-strategy tuning of learning rate and epochs; Only-Patellar diagnosis runs (lr 3e-4, longer training) as evidence of LoRA's

After every epoch, we compute the loss $\mathcal{L}_{\text{seg}}$ on the validation set, with one patch per validation volume drawn in the same way as the training patches. The patches are drawn anew for every validation, so the validation loss of two epochs is computed on different patches. In the equal-updates runs of Section 6.5, which have many short epochs per task, only every 24th epoch is validated, the last epoch included. All strategies are evaluated with the parameters after the last epoch, which are also the parameters that the next task starts from, so the evaluated model is the model that continues through the sequence. For all strategies except the two LoRA strategies, an additional evaluation uses the checkpoint of the epoch with the lowest validation loss; for the LoRA strategies, only the adapter after the last epoch is stored. Algorithm 2 therefore returns both parameter sets, and Section 6.1.3 compares the two evaluations. Because the validation loss of an epoch is computed on one random patch per volume, it is a noisy criterion for selecting a checkpoint.

For evaluation, each test volume is cropped and normalised as described in Section 4.4 and segmented with sliding-window inference: patches of 192^3 voxels are placed with a step of half the patch size, their logits are weighted with a Gaussian of standard deviation one eighth of the patch size and averaged, as in nnU-Net [Ise+21]. The averaged logits

Algorithm 2 ADAPT: adapting VoxTell to task t. The steps with subscript $\mathcal{S}$ are where the strategies differ (Section 5.4).

Input: parameters θ_{t-1} carried forward from the previous task; task t with text prompt p_t, $\mathcal{D}_t^{\text{tr}}$ and $\mathcal{D}_t^{\text{val}}$; strategy $\mathcal{S}$; E, η_0, μ, ω, π_{fg}
Output: parameters θ_t after the last epoch; evaluated parameters $\tilde{\theta}_t$

$(\theta, \phi) \leftarrow \textsc{Init}_{\mathcal{S}}(\theta_{t-1}, t)$ {model and trainable parameters ϕ}
$\mathcal{D} \leftarrow \mathcal{D}_t^{\text{tr}} \cup \textsc{ExtraData}_{\mathcal{S}}(t)$; $U \leftarrow E \cdot |\mathcal{D}|$; $u \leftarrow 0$; $\ell^* \leftarrow \infty$
for $e = 1$ **to** E **do**
  for each volume $(\boldsymbol{X}, \boldsymbol{Y}, p)$ of $\mathcal{D}$ in random order **do**
    $(\boldsymbol{X}', \boldsymbol{Y}') \leftarrow \textsc{SamplePatch}(\boldsymbol{X}, \boldsymbol{Y}, \pi_{\text{fg}})$
    $\mathcal{L} \leftarrow \mathcal{L}_{\text{seg}}(f_\theta(\boldsymbol{X}', p), \boldsymbol{Y}') + \textsc{ExtraLoss}_{\mathcal{S}}(\boldsymbol{X}', p, \theta)$
    update ϕ by one SGD step on $\nabla_\phi \mathcal{L}$ with learning rate $\eta_0(1 - u/U)$, momentum μ, weight decay ω; $u \leftarrow u + 1$
  end for
  $\ell \leftarrow$ mean of $\mathcal{L}_{\text{seg}}$ over one sampled patch per volume of $\mathcal{D}_t^{\text{val}}$
  if $\ell < \ell^*$ **then**
    $\ell^* \leftarrow \ell$; $\theta^{\text{best}} \leftarrow \theta$
  end if
end for
$\theta_t \leftarrow \theta$; $\tilde{\theta}_t \leftarrow \theta_t$ {additional evaluation: $\tilde{\theta}_t \leftarrow \theta^{\text{best}}$}
return $(\theta_t, \tilde{\theta}_t)$

are passed through the logistic function and thresholded at 0.5, and the DSC between the binary prediction and the label mask is computed on the cropped volume. No test-time augmentation is used.

Each iteration of Algorithm 2 costs one forward pass through the model and one backward pass through the part of the network that lies between the loss and the trainable parameters, so a task costs $E \cdot |\mathcal{D}|$ such iterations plus $E \cdot |\mathcal{D}_t^{\text{val}}|$ forward passes for validation. The time is therefore linear in the number of epochs and in the number of training volumes. The memory holds the model parameters, one momentum buffer per trainable parameter and the activations of one 192^3 patch; it does not depend on the number of training volumes, because one patch is processed at a time. How these costs differ between the strategies is summarised in Section 5.5, and the measured runtime and memory are reported in Table 6.10.

5.4 Continual-Learning Strategies

We compare five continual-learning strategies with sequential fine-tuning, the baseline, which adds nothing to reduce forgetting. One of the five combines shared LoRA with replay. The five strategies were selected to cover the three families of continual learning for vision–language models distinguished by Liu et al. [Liu+26b, Sec. 4]. Replay represents multimodal replay. The distillation-based strategy represents cross-modal regularisation; its template ZSCL is one of the survey's examples of preserving the structure of the pretrained image–text embedding space. Parameter-efficient adaptation is represented by low-rank adaptation (task-specific and shared LoRA) and by prompt-based methods (the prompt-tuning strategy, adapted from CPE-CLIP, which the survey lists among them). Shared LoRA with replay combines two families, a combination the survey describes as common. Within each family, we started with the method best suited to a pretrained vision–language model or the few-shot setting. Other methods, such as adapter-based or mixture-of-experts adaptation, constraints to orthogonal subspaces [Liu+26b, Sec. 4] or elastic weight consolidation [Kir+17], were not evaluated.

defense: alternatives within each family not evaluated

Every strategy is a version of Algorithm 2, defined by the parameters it trains, by how it starts each task and by data or loss terms it adds. Table 5.1 summarises these choices, and Figure 5.1 shows where in the model each strategy acts.

5.4.1 Sequential Fine-Tuning

Sequential fine-tuning trains all parameters of the model except the frozen text encoder on each task in turn, starting each task from the parameters after the last epoch of the previous one. It adds no data, no loss term and no restriction of the parameters, so it is the usual fine-tuning of a pretrained model, repeated for every task. Comparing the other strategies with it shows the effect of their components. Of the 440,029,541 parameters it sets as trainable, 100,737,024 belong to the unused self-attention modules of the prompt decoder and the layer normalisations before them (Section 5.3) and never receive a gradient, so 339,292,517 parameters are actually updated. Its cost per iteration is one forward and one full backward pass. It needs a momentum buffer for every trained parameter, which makes it the most memory-intensive strategy apart from the distillation-based one (Section 5.5).

Table 5.1: Overview of the continual-learning strategies as instances of Algorithm 2. All strategies are evaluated with the parameters after the last epoch, from which the next task also starts (Section 5.3).

Strategy	Trained parameters ϕ	Start of task t (INIT)	Added data / loss
Sequential fine-tuning	all parameters except the text encoder	parameters of task $t-1$	–
Task-specific LoRA	low-rank factors $\boldsymbol{B}, \boldsymbol{A}$ in $\boldsymbol{W}_q, \boldsymbol{W}_v$ of the cross-attention of all layers	pretrained model with a new adapter	–
Shared LoRA	as task-specific LoRA	adapter of task $t-1$	–
Shared LoRA + replay	as task-specific LoRA	adapter of task $t-1$	replay memory $\mathcal{M}_{t-1}$
Distillation-based	all parameters except the text encoder	parameters of task $t-1$	$\lambda \mathcal{L}_{\mathrm{KD}}$
Prompt tuning	tokens $\boldsymbol{V}_l$ and meta-net h_ψ in layers $l \le L_p$	tokens and meta-net of task $t-1$	–

5.4.2 Task-Specific and Shared LoRA

Both LoRA strategies keep all pretrained weights frozen and train a low-rank update $\gamma_r \boldsymbol{B}\boldsymbol{A}$ (Equation (3.3)) of selected projection matrices $\boldsymbol{W}_0 \in \mathbb{R}^{d \times d}$ of the prompt decoder [Hu+22]. The rank r and the hyperparameter α are set per configuration. The scaling factor is the rank-stabilised $\gamma_r = \alpha/\sqrt{r}$ of rsLoRA (Equation (3.4)) [Kal23]. We use it instead of α/r because the shared adapter is trained with high ranks (Section 6.3), at which rsLoRA showed α/r to slow learning (Section 3.2.2). $\boldsymbol{A}$ is initialised with the Kaiming-uniform scheme instead of the Gaussian of the original LoRA. $\boldsymbol{B}$ starts at zero, so the update is zero at the start and training begins from the pretrained model. Dropout is applied to the input of the low-rank path only.

We apply LoRA to the query and value projections $\boldsymbol{W}_q$ and $\boldsymbol{W}_v$ of the cross-attention in all L_{dec} prompt-decoder layers (Figure 5.1). The cross-attention is where the text query attends to the image features, the step through which the text prompt steers the segmentation in VoxTell. LoRA has also been applied to attention projections of vision–language models before, for instance in CLIP-LoRA [Zan+24]. Our LoRA layers are adapted from

the `loralib` module of the public CLIP-LoRA code[1], including the split of the joint input projection of the attention into separate query, key and value projections, to which the low-rank updates are added. The restriction to the query and value projections follows the finding of Hu et al. for the self-attention of GPT-3 [Hu+22, Sec. 7.1, Table 5] (Section 3.2.2); whether it also holds for the cross-attention of VoxTell was not tested here. The image encoder, the convolutional decoder, the guidance projections, the self-attention modules and the key and output projections of the cross-attention stay unchanged. With two adapted projections per layer and $2dr$ parameters each, a LoRA adapter has $4L_{\text{dec}}\,d\,r = 49{,}152\,r$ trainable parameters for $d = 2048$ and $L_{\text{dec}} = 6$.

The two strategies differ in how the adapters are used across tasks. Task-specific LoRA starts every task from the pretrained model with a new adapter, so each adapter is trained only on its own task and independently of all others. After each task, the adapter is stored, and at inference the task identity selects the adapter (Section 5.2). After T tasks, T adapters are kept. Shared LoRA creates one adapter before the first task and continues training it on every following task, so all tasks and the control task are evaluated with the same adapter. The two strategies were run with different values of α and of the dropout rate, which are listed in Table 6.3; their comparison is therefore one between configurations, not only between a task-specific and a shared adapter.

defense: alpha and dropout differ between the two LoRA variants

For both strategies, an iteration still needs a forward pass through the whole model, but the backward pass only has to reach the adapted projections, through the convolutional decoder, the guidance projections and the prompt decoder; no gradient is needed in the image encoder or the text encoder. Only the adapter parameters need momentum buffers. In our implementation, every training forward pass forms the update as the full matrix $\gamma_r \boldsymbol{BA}$, at $\mathcal{O}(d^2 r)$ operations, and applies it like a second projection. The two adapted projections therefore cost about twice as much as without LoRA, which is still small compared with the convolutions of the image network. For evaluation, the update is merged into $\boldsymbol{W}_0$ once, so inference costs the same as for the pretrained model.

5.4.3 Replay

Replay keeps a memory $\mathcal{M}$ of earlier training samples and adds it to the training data of each new task. With only a few stored samples per task, the model can overfit them [De+22, Sec. 3.1]. Storing data also conflicts with storage limits and privacy constraints [De+22, Sec. 1], which in medical imaging restrict what may be kept from earlier patients. We

[1] https://github.com/MaxZanella/CLIP-LoRA/tree/main/loralib

combine replay with shared LoRA, so that shared LoRA with and without replay can be compared directly. Because the six tasks share the same volumes, a memory entry consists of a volume together with the text prompt and the mask of the task it was drawn from.

The memory starts empty and contains only samples of earlier tasks, so the control task stays held out and nothing from the pretraining of VoxTell is replayed. After each task t, $\max(1, \mathrm{round}(\rho\,|\mathcal{D}_t^{\mathrm{tr}}|))$ samples drawn at random from its training set are added to the memory, up to a capacity of $M_{\max}$ entries, where ρ is the replay fraction. They are not selected by a criterion such as that of iCaRL [Reb+17], which keeps per class the exemplars that best approximate the class mean in feature space [De +22, Sec. 3.1]. During task t, the memory $\mathcal{M}_{t-1}$ is appended to the training set (ExtraData in Algorithm 2). Its entries are sampled and trained like the task's own volumes, each with the text prompt of its own task, so an epoch has $N_t = |\mathcal{D}_t^{\mathrm{tr}}| + |\mathcal{M}_{t-1}|$ iterations. The memory is stored with the state of the run, so a run that is interrupted and resumed continues with the same memory. The replay fraction was set once and not tuned; ρ and $M_{\max}$ are listed in Table 6.3.

defense: reason for random instead of selected replay samples (author)

Replay changes only the data, so the cost of an iteration is that of shared LoRA, and the number of iterations per epoch grows by $|\mathcal{M}_{t-1}|$. The memory stores references to volumes and masks, not copies, so it needs no additional GPU memory.

5.4.4 Distillation-Based Strategy

The distillation-based strategy is adapted from ZSCL [Zhe+23], which protects the zero-shot ability of CLIP by distilling the similarity distributions of the frozen initial model on a reference dataset, by averaging the weights during training and, in the full method, by a weight-consolidation loss (Section 3.2.3).

Our strategy trains all parameters except the text encoder, like sequential fine-tuning, and adds a distillation loss $\lambda\,\mathcal{L}_{\mathrm{KD}}$ on each training patch (ExtraLoss in Algorithm 2). The teacher is a frozen copy of the pretrained model f_{θ_0}, which is not updated during the whole task sequence and receives the same patch and text prompt as the student. The loss compares how the stage-4 guidance relates to the stage-4 image features (Figure 5.1). Let $\boldsymbol{e}_g$, $g = 1, \ldots, G$, be the G guidance vectors of $\boldsymbol{E}_4$ and $\boldsymbol{f}_v$ the encoder features of stage 4 at voxel v of the 12^3 voxels Ω_4 of that stage. For each guidance vector, the student defines a distribution over the voxels,

$$q_{g,v} = \frac{\exp\left(\langle \boldsymbol{e}_g/\|\boldsymbol{e}_g\|, \boldsymbol{f}_v\rangle\right)}{\sum_{v'\in\Omega_4} \exp\left(\langle \boldsymbol{e}_g/\|\boldsymbol{e}_g\|, \boldsymbol{f}_{v'}\rangle\right)}, \qquad \mathcal{L}_{\mathrm{KD}} = \sum_{g=1}^{G} \sum_{v\in\Omega_4} \bar{q}_{g,v} \log \frac{\bar{q}_{g,v}}{q_{g,v}}, \tag{5.2}$$

where $\bar{q}_{g,v}$ is the same distribution computed by the teacher, so $\mathcal{L}_{\mathrm{KD}}$ is the Kullback–Leibler divergence of the student's distributions from the teacher's. The gradient of $\mathcal{L}_{\mathrm{KD}}$ reaches the image encoder up to stage 4, the prompt decoder and the stage-4 guidance projection.

From ZSCL, the strategy keeps two elements. The first is the frozen initial model as teacher, which ZSCL found to work better than the model of the previous task [Zhe+23, Table 1d]. The second is the idea of distilling a similarity distribution between text-derived and image representations, added to the task loss with a weight λ. It differs from ZSCL in six points. First, it distils on the patches of the current task instead of a reference dataset, so it needs no data beyond the task data; in ZSCL's ablation, this choice kept the transfer to unseen tasks at the level of a reference dataset but lowered the average and final accuracy [Zhe+23, Table 1b, p. 7]. Second, the distribution runs over voxels for each guidance vector, not over texts for each image, and only in this one direction; ZSCL additionally distils a text-side distribution. Third, it uses stage 4, the level at which text and image interact in the prompt decoder, which is also the coarsest and cheapest level, with 12^3 voxels per patch. In the segmentation itself, $\boldsymbol{E}_4$ is fused with the decoder features of that resolution, whereas $\mathcal{L}_{\mathrm{KD}}$ pairs it with the encoder features. Fourth, λ is much smaller than ZSCL's $\lambda = 1$ and was not tuned (Table 6.3). Fifth, it has no weight-consolidation loss. Sixth, its task loss is the segmentation loss of Equation (5.1), optimised with SGD instead of AdamW.

The strategy also maintains ZSCL's weight ensemble (Equation (3.5)), which is updated after every I_{we}-th iteration of an epoch and continued across tasks, so the ensemble is the equally weighted mean of θ_0 and all parameter snapshots taken so far. Iterations at the end of an epoch that do not complete an interval of I_{we} are not included. Our interval $I_{\mathrm{we}} = 10$ is smaller than the interval of 100 used by ZSCL (Table 6.3). Training continues from the student, so the ensemble is not carried forward to the next task. We evaluate the ensemble after every task as an additional evaluation, next to the student. The strategy in the main comparison is therefore fine-tuning with feature distillation, after which we name it.

Each iteration adds one forward pass of the teacher without a backward pass. The teacher is kept in CPU memory and copied to the GPU for this forward pass, which adds transfer time. The weight ensemble is a further copy of all parameters on the GPU, so this strategy needs the most memory of all (Table 6.10).

5.4.5 Prompt-Tuning Strategy

The prompt-tuning strategy is adapted from CPE-CLIP [DAl+23], which learns prompt tokens in a frozen CLIP model for few-shot class-incremental learning and scales their gradients down as more classes have been seen (Section 3.2.4).

Our strategy keeps every VoxTell parameter frozen and trains two components in the first L_p of the L_{dec} prompt-decoder layers (Figure 5.1). The first component is a separate set of learned prompt tokens $\boldsymbol{V}_l \in \mathbb{R}^{m \times d}$ for each layer $l = 1, \ldots, L_p$. In layer l, its m tokens are appended to the keys and values of the cross-attention with a positional encoding of zero. We place the tokens in the prompt decoder because VoxTell's image encoder is convolutional and has no token sequence to which they could be appended. The second component is a meta-net h_ψ, a two-layer perceptron with a GELU activation, which adds an offset to the text query before each of these layers,

$$\boldsymbol{q} \leftarrow \boldsymbol{q} + h_\psi\big([\boldsymbol{q}; \bar{\boldsymbol{f}}\,]\big), \tag{5.3}$$

where $\bar{\boldsymbol{f}}$ is the mean of the stage-4 image features over all voxels after their projection to dimension d, and $[\cdot\,;\cdot]$ denotes concatenation. The design of the meta-net was inspired by the conditional prompt learning of CoCoOp [Zho+22a]. The tokens are initialised with the Xavier-uniform scheme. With $L_p \cdot m \cdot d$ token parameters and the meta-net, the strategy has 12,599,296 trainable parameters, of which 12,587,008 belong to the meta-net; the name prompt tuning therefore understates the share of the meta-net. The values of m and L_p were set once and not tuned (Table 6.3).

One set of tokens and one meta-net are trained through all tasks and used for every task at inference, so the strategy needs no task identity. Because the tokens and the meta-net change the keys, values and query for every text prompt, freezing the backbone does not keep the outputs for other text prompts unchanged. The gradients of the tokens and of the meta-net in task t are multiplied by $\max(0.25, \beta_t)$, where CPE-CLIP's scale β_t (Equation (3.6)) reduces to $1/t$ for one new structure per task and no base session. CPE-CLIP applies its scale to the language prompts and to the projection that derives the vision prompts from them [DAl+23, p. 5, Eqs. 12–13]; we correspondingly apply it to the tokens and to the meta-net, which holds almost all of our trainable parameters. In CPE-CLIP's ablation, removing this scaling degraded performance more than removing any other component (Section 2.3). An earlier run in which the scale was not activated

serves as an ablation; it was evaluated only with the checkpoint of the lowest validation loss, so the two runs are compared with that checkpoint (Section 6.2).

The strategy differs from CPE-CLIP in four points. The tokens act on the cross-attention of the prompt decoder instead of on the inputs of a language and a vision encoder. There is no projection between CPE-CLIP's language and vision prompts. The meta-net is added. The task loss is the segmentation loss of Equation (5.1), without a base session. We therefore name the strategy after what it trains, prompt tuning, rather than after CPE-CLIP. An iteration needs a forward pass through the whole model and a backward pass through the convolutional decoder, the guidance projections and the prompt decoder down to its first layer, but not through the encoders. The momentum buffers cover only the 12.6 M trainable parameters, and the m additional keys and values per prompted layer cost little compared with the 12^3 image positions.

5.5 Computational Complexity

All strategies share the forward pass through the whole model and the evaluation of Algorithm 1; they differ in how far the backward pass reaches, in how many parameters need optimiser state, in additional forward passes and in the amount of data per epoch (Table 5.2). Let κ be the cost of one forward pass on a patch and $\kappa_{\mathcal{S}}$ the cost of the backward pass of strategy $\mathcal{S}$. Adapting to task t then costs $\mathcal{O}\big(E\,N_t\,(\kappa+\kappa_{\mathcal{S}}) + E\,|\mathcal{D}_t^{\mathrm{val}}|\,\kappa\big)$, and the distillation-based strategy adds $E\,N_t\,\kappa$ for the teacher. The time is thus linear in the number of epochs and of training samples for every strategy, and replay increases N_t by the size of the memory.

For memory, the main difference is whether the backward pass reaches the image encoder: sequential fine-tuning and the distillation-based strategy store its activations and keep momentum buffers for all parameters, whereas the LoRA strategies and the prompt-tuning strategy need neither. The distillation-based strategy additionally holds the teacher and the weight ensemble. After T tasks, task-specific LoRA stores one adapter per task, so its storage grows linearly with the number of tasks; all other strategies store one parameter set. At inference, the sliding-window forward passes cost about the same for all strategies (Sections 5.4.2 and 5.4.5). The measured training time per epoch, the peak GPU memory and the evaluation time per volume on the NVIDIA A100 used in this work are reported in Table 6.10.

Table 5.2: Computational cost of the strategies relative to sequential fine-tuning. P: parameters of the segmentation model (440,029,541, of which 339,292,517 receive gradients, Section 5.4.1); d, L_{dec}: dimension and number of prompt-decoder layers; r: LoRA rank; m, L_p: tokens per layer and prompted layers; $|\psi|$: parameters of the meta-net. The frozen text encoder is not counted. Measured runtime and memory: Table 6.10.

Strategy	Trained parameters $\lvert\phi\rvert$	Additional cost per training iteration	Additional training memory	Stored after T tasks
Sequential fine-tuning	P	– (reference: full backward pass)	momentum for P	P
Task-specific LoRA	$4L_{\text{dec}}\,d\,r$	backward pass only down to the prompt decoder; forming and applying $\gamma_r \boldsymbol{B}\boldsymbol{A}$, $\mathcal{O}(d^2 r)$ per projection	momentum for $\lvert\phi\rvert$	$P + T \cdot 4L_{\text{dec}}\,d\,r$
Shared LoRA	$4L_{\text{dec}}\,d\,r$	as task-specific LoRA	momentum for $\lvert\phi\rvert$	$P + 4L_{\text{dec}}\,d\,r$
Shared LoRA + replay	$4L_{\text{dec}}\,d\,r$	as shared LoRA; $\lvert\mathcal{M}_{t-1}\rvert$ more iterations per epoch	momentum for $\lvert\phi\rvert$; memory holds references only	$P + 4L_{\text{dec}}\,d\,r$
Distillation-based	P	one forward pass of the teacher, copied from CPU to GPU; $\mathcal{L}_{\text{KD}}$ on $G \cdot 12^3$ similarities	momentum for P; teacher (P, CPU); weight ensemble (P, GPU)	P
Prompt tuning	$L_p\,m\,d + \lvert\psi\rvert$	backward pass only down to the prompt decoder; m more keys and values in L_p layers	momentum for $\lvert\phi\rvert$	$P + L_p\,m\,d + \lvert\psi\rvert$

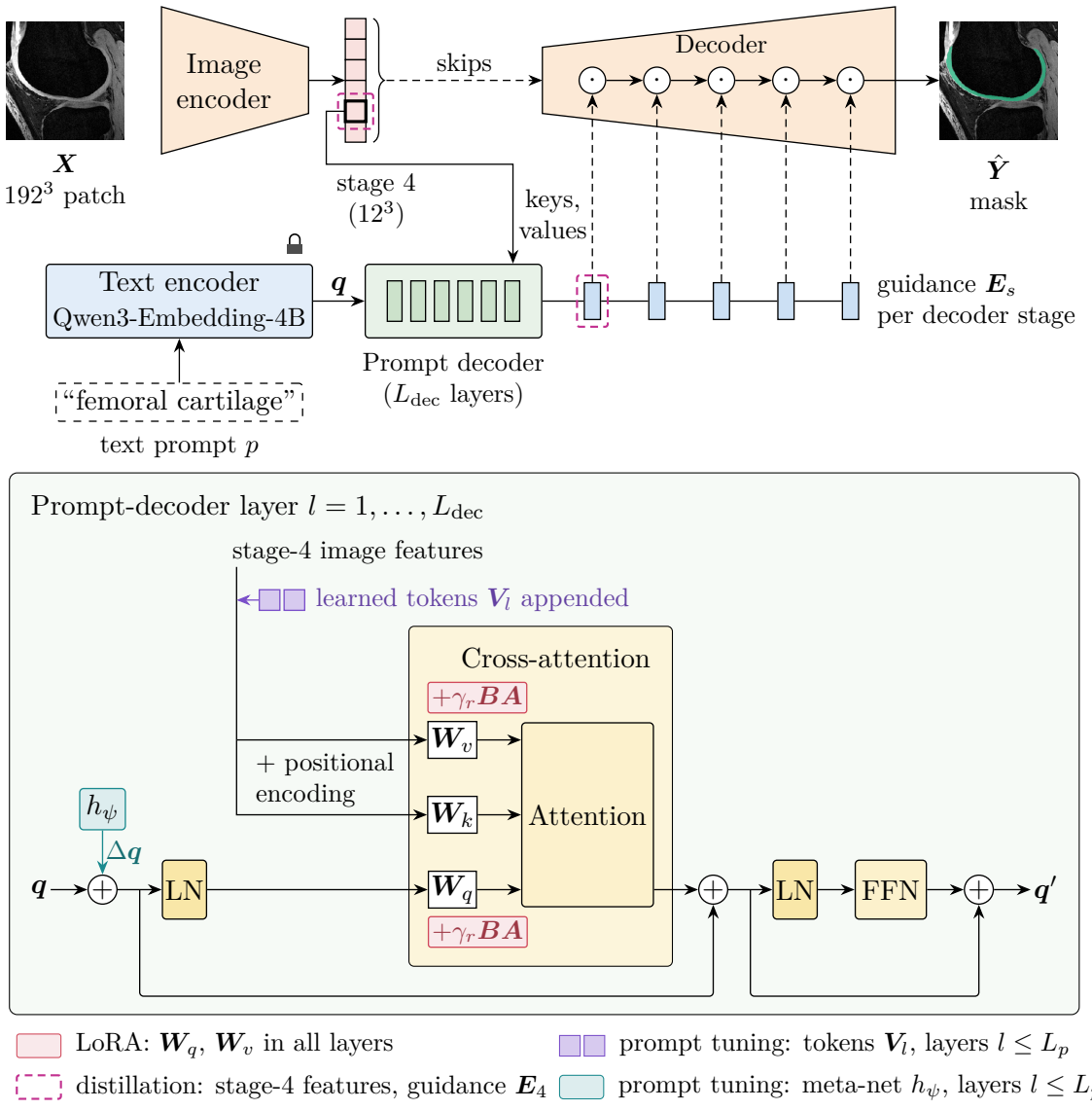


Figure 5.1: VoxTell as used in this thesis and the parts that the continual-learning strategies change (layout adapted from [Rok+26, Fig. 2]). Top: all encoder stages feed the decoder stage of the same resolution; the prompt decoder attends to the stage-4 features, and its output is mapped to the guidance $\boldsymbol{E}_s$ of the five fusion points. The lock marks the frozen text encoder; the output panel shows the reference mask of femoral cartilage (test volume MTR_005) for illustration. Bottom: one prompt-decoder layer as executed; the attention block includes the output projection. The coloured marks are explained in Section 5.4. Sequential fine-tuning and the distillation-based strategy update all parameters except the text encoder; replay changes only the training data.

Chapter 6

Experiments and Results

This chapter reports the experiments with which we answer the research question. Section 6.1 gives the setup: the experiments, the data settings, the values of all parameters and the metrics. Each of the following sections presents one experiment with its result and its conclusion.

6.1 Experimental Setup

6.1.1 Overview of the Experiments

Table 6.1 lists the experiments. Each consists of runs of the protocol of Algorithm 1 that differ in one factor: the strategy, the LoRA rank, the number of training volumes or of updates per task, the task order, the use of replay or of gradient scaling. Joint fine-tuning, trained on all six tasks at once, is the upper reference for the comparison of the strategies and for the amount of labelled data. The comparison of the strategies addresses the second objective of Section 1.3, the experiment on the amount of labelled data the third, and the experiments on rank, task order, replay and gradient scaling the fourth. A diagnostic experiment with task-specific LoRA trained on patellar cartilage alone tests why this strategy did not learn that structure in the task sequence (Section 6.4). In total, we report 38 runs, each trained once; the evaluation matrices of all of them are given in Section A.2. The pretrained model serves as a reference in every experiment (Section 5.2).

Some factors were not varied. We trained two task orders, one draw of training volumes per few-shot setting and one text prompt per structure, and we did not tune the learning

rate or the number of epochs per strategy. How these choices limit the conclusions is discussed in Section 7.3.

6.1.2 Data, Task Orders and Few-Shot Settings

In the original task order, the structures are trained in the sequence lateral tibial cartilage, medial tibial cartilage, femoral cartilage, patellar cartilage, medial meniscus and lateral meniscus; this order was chosen at random. The reverse order serves as a second task order, to check whether the results depend on the order in which the structures are learned. Sequential fine-tuning, task-specific LoRA and shared LoRA were trained in both orders (Table 6.1). We did not train further orders, because every additional order requires a complete run of each strategy, which our time and compute budget did not allow.

In the few-shot settings, every task is trained on the first $k \in \{5, 10, 20\}$ volumes of its training split instead of all 108 (Section 4.3). These values correspond to about 5, 10 and 20 % of the training split (exactly 4.6, 9.3 and 18.5 %). The sets are nested and drawn once; Section A.1 lists the volumes of each set and the test volumes. The number of epochs stays $E = 10$ in all settings, so a task is trained with $10k$ updates, that is 50, 100 or 200, instead of 1,080 with the full split. The few-shot settings therefore reduce both the amount of labelled data and the number of updates. To separate the two, sequential fine-tuning, task-specific LoRA and shared LoRA were also trained with $k = 5$ and $E = 216$ epochs, which gives the 1,080 updates per task of the full split (equal-updates runs), and only every 24th epoch is validated in these runs (Section 5.3). We ran this ablation for $k = 5$ only, the setting with the largest difference in updates, because of the compute it needs; for $k = 10$ and $k = 20$, data and updates remain confounded. Joint fine-tuning was trained with the same k volumes per structure, that is, with $6k$ volumes in its single stage.

6.1.3 Training and Evaluation Configuration

Table 6.2 lists the settings that all runs share, and Table 6.3 the parameters of each strategy. Both tables are generated from the configuration files of the runs; the few values that are defaults of the code, such as the fraction of foreground-centred patches, are checked against the code by the same script. Each run was trained on one NVIDIA A100 GPU with 40 GB of memory on the TinyGPU cluster, where a job may run for at most 24 hours. The software environment comprises PyTorch 2.8.0 with CUDA 12.6 and cuDNN 9.10.2, MONAI 1.5.2, nnU-Net 2.7.0, Transformers 4.57.6, NumPy 2.4.3 and NiBabel 5.4.2. The

runs did not log these versions; they were read from the environment on 22 October 2026, and we cannot rule out that packages were updated between the earlier runs and this date. Training is deterministic for a given configuration, and a run that was continued at a task boundary reproduces an uninterrupted one exactly: evaluated at the same checkpoint, the reruns of sequential fine-tuning with 5, 10 and 20 volumes and in the reverse order reproduce the evaluation matrices of the earlier runs with the same configuration in every entry. A run that was continued inside a task, however, draws its training samples and patches in a different order from that point on, so it is valid but not reproducible from its configuration alone; this concerns five runs of the comparison of the strategies and of the few-shot experiment, in each case from the task in which the job was resumed. Training a full-data run took between 18.1 and 26.4 hours, and 37.1 hours for the distillation-based strategy; runs that reached the 24-hour limit were continued in a second job, and the peak GPU memory was between 20.3 and 37.9 GiB (Section 6.7). These numbers are why we train with a batch size of one and $E = 10$ epochs per task (Section 5.3).

defense: software versions not logged per run; read from the environment on 2026-10-22

For the comparison of the strategies, we use rank $r = 4$ for task-specific LoRA and $r = 1024$ for shared LoRA. We chose these ranks after comparing the runs of the rank experiment, whose results are computed on the same test volumes as all other results; Section 6.3 gives the reasons. The choice therefore favours the two LoRA configurations in the comparison, and we report the run with $r = 8$ for task-specific LoRA as well, so that the alternative is visible.

author: same reason as in Sec. 5.3; with the reruns, several full-data runs needed a second job

All runs are evaluated with the parameters after the last epoch (Section 5.3). In the additional evaluation with the checkpoint of the lowest validation loss, this checkpoint came from the last epoch in only 8 of the 46 trained tasks of the 11 runs with ten epochs per task, and in none of the six tasks of the equal-updates run of sequential fine-tuning. The selected epoch was epoch 8 in 28 tasks, and the same few epochs (2, 6, 8, 9 and 10) recur across runs and tasks. Since the validation patches are drawn with the same seed in every task (Section 5.3), this pattern suggests that the selected epoch depends on which patches are drawn as much as on the model; we did not test this.

defense: main-comparison ranks chosen on test-split results (optimistic bias)

6.1.4 Metrics and Statistical Analysis

We report the five metrics of Section 3.3. Three of them are primary, because the research question asks how strategies trade adaptation against forgetting: Adaptation, Final Forgetting and the control-task drop. The control-task drop matters for a universal model, whose value lies in the many structures it can segment beyond the tasks it is adapted to.

defense: selected epochs

Final Forgetting and the control-task drop together measure retention, within and outside the task sequence; retention is therefore a property we measure with these two metrics, not a metric of its own. Final Performance, the quality of the model that the sequence delivers, and Transfer are secondary. Because the structures that come first differ between task orders, we compare Transfer only within one task order. Backward transfer appears as negative Final Forgetting (Section 3.3) and needs no metric of its own. We do not report the Average Performance of ZSCL, the mean over all entries of stages 1 to T, because it mixes adaptation, retention and transfer into one number that none of our questions asks for.

For task-specific LoRA, Final Forgetting and the control-task drop are zero by construction, because each task is evaluated with its own adapter and the control task with the unchanged pretrained model (Section 5.2). For this strategy, the comparison rests on Adaptation and Final Performance. For joint fine-tuning, which has no task sequence, only Final Performance and the control-task drop are defined (Section 5.2). Every table also contains the pretrained-model reference row of Section 5.2. All values are mean DSC fractions, reported with three decimals.

Each configuration was trained once, so we cannot estimate how results vary with the training randomness or with the choice of training volumes. The only variability we can estimate is that over the test volumes, and we do so with a bootstrap [Efr+93]. In each of $N_{\text{boot}} = 2000$ draws, the 24 SKM-TEA test volumes are resampled with replacement, the same draw is used for all tasks and all stages, and the 4 control-task volumes are resampled separately; every metric is then recomputed from the resampled means. We report the 2.5 and 97.5 percentiles of the resampled values as a 95 % interval [Efr+93] in the figures; the tables show the values on the full test set. With only 4 control-task volumes, the intervals of the control-task drop are coarse.

Two runs are evaluated on the same test volumes, so their errors are correlated, and two overlapping intervals do not show that the runs perform alike. For the difference between two runs, we therefore apply the same draws to both runs and compute the interval of the difference (paired bootstrap). We read these intervals only for comparisons that follow from the design of the experiments in Table 6.1: each strategy against sequential fine-tuning, each few-shot setting against the full training split of the same strategy, each equal-updates run against the same strategy with $k = 5$ and with the full split, the reverse against the original task order of the same strategy, a strategy with a component against the same strategy without it, each additional evaluation against the main evaluation of the same

run, and joint fine-tuning against each strategy in Final Performance and the control-task drop. For the LoRA rank, we describe the trend over the ranks instead of comparing pairs of ranks, and differences between two continual-learning strategies are reported only descriptively. With the runs of this chapter, this gives 34 pairs and 152 comparisons. The intervals are not adjusted for this number of comparisons and describe the uncertainty due to the choice of test volumes only, so we do not use them as significance tests and do not call any difference significant. How large the variation between training runs can be compared with these intervals is shown by two runs of task-specific LoRA that differ only in their random initialisation (Section 6.6).

6.2 Comparison of the Strategies

This experiment asks how the strategies trade adaptation against forgetting when each task is trained on the full training split. We trained sequential fine-tuning, task-specific LoRA ($r = 4$), shared LoRA ($r = 1024$), the prompt-tuning strategy and the distillation-based strategy in the original task order, and shared LoRA with replay as a sixth configuration (Table 6.1). Joint fine-tuning, trained on all six tasks at once, serves as the upper reference (Section 5.2). All runs are evaluated with the parameters after the last epoch of each task. Table 6.4 gives all metrics, Figure 6.1 shows them with their bootstrap intervals, and Figure 6.2 shows the evaluation matrices from which they are computed.

In the primary metrics, the strategies fall into two groups, with the prompt-tuning strategy between them. Sequential fine-tuning and the distillation-based strategy reached the highest Adaptation, 0.781 and 0.747, but forgot most of what they had learned: their Final Forgetting is 0.782 and 0.734, and the DSC on the control task fell from 0.979 to at most 0.0003, a control-task drop of 0.979 for both. The two LoRA strategies adapted less, with 0.485 for task-specific and 0.471 for shared LoRA, and retained much more: task-specific LoRA forgets nothing by construction, and shared LoRA reached a Final Forgetting of 0.289 and a control-task drop of 0.237. Against sequential fine-tuning, the paired differences in Adaptation are -0.296 $[-0.319, -0.273]$ for task-specific and -0.311 $[-0.324, -0.295]$ for shared LoRA, and the difference in Final Forgetting is -0.492 $[-0.509, -0.474]$ for shared LoRA. The prompt-tuning strategy reached an Adaptation of 0.580, a Final Forgetting of 0.510 and a control-task drop of 0.865.

After the last task, sequential fine-tuning, the distillation-based and the prompt-tuning strategy segment little beyond the structure trained last. Their Final Performance of 0.130,

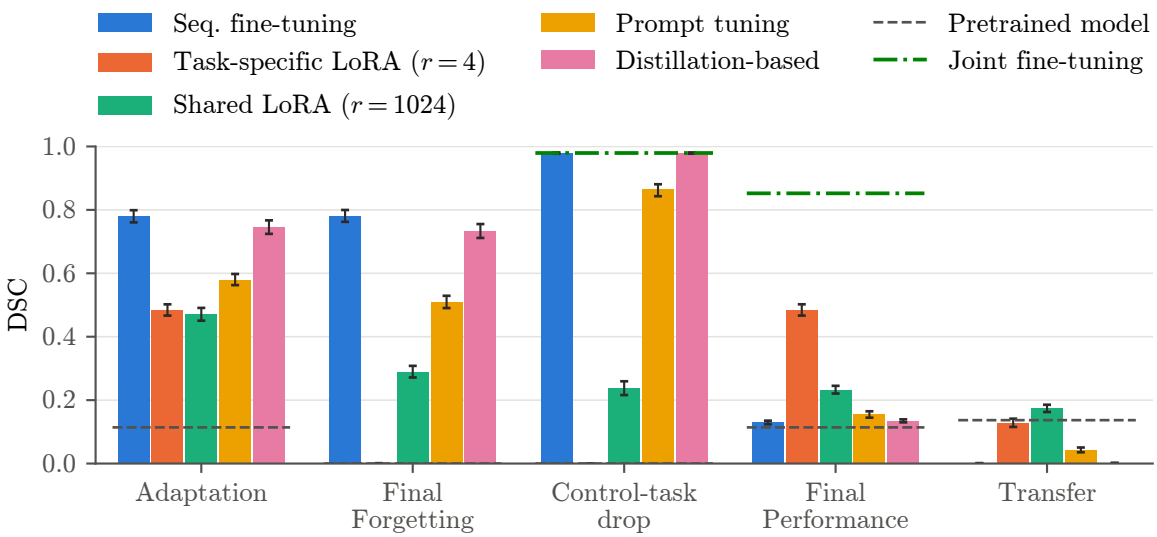


Figure 6.1: Metrics of the five strategies of Table 6.4 without replay, with 95 % bootstrap intervals over the test volumes (Section 6.1.4). Dashed line: pretrained model; dash-dotted line: joint fine-tuning, shown for the two metrics defined for it.

0.135 and 0.155 is only slightly above the 0.114 of the pretrained model, and they lost femoral cartilage, which the pretrained model already segmented with a DSC of 0.684 and which ends at 0.010 or less (Figure 6.2). Outside the diagonal, the matrices of sequential fine-tuning and the distillation-based strategy are close to zero; the only entry above 0.2 is femoral cartilage after the patellar task in the distillation-based strategy (0.25). The prompt-tuning strategy keeps more outside the diagonal, with seven entries between 0.20 and 0.49, the highest of them femoral cartilage after the patellar task. Among the strategies without replay, task-specific LoRA reached the highest Final Performance, 0.485, because each task keeps its own adapter, and shared LoRA reached 0.233.

Figure 6.3 shows when the control task was lost. For sequential fine-tuning and the distillation-based strategy, the DSC on the lung fell to 0.002 or less after the first task and did not recover. The prompt-tuning strategy fell to 0.04 after the first task and then stayed between 0.04 and 0.23. Shared LoRA kept the control task between 0.955 and 0.980 up to the fifth task, and most of its control-task drop arose in the last step (Section 6.6).

The distillation-based strategy forgot slightly less than sequential fine-tuning. Its Final Forgetting was 0.048 lower $[0.039, 0.055]$ at an Adaptation lower by 0.035 $[0.027, 0.041]$, its Final Performance was 0.005 higher $[0.001, 0.009]$, and its control-task drop was the same. These differences are small against the gap to the LoRA strategies. They hold for the evaluated configuration, with a distillation weight $\lambda = 0.01$ that is much smaller

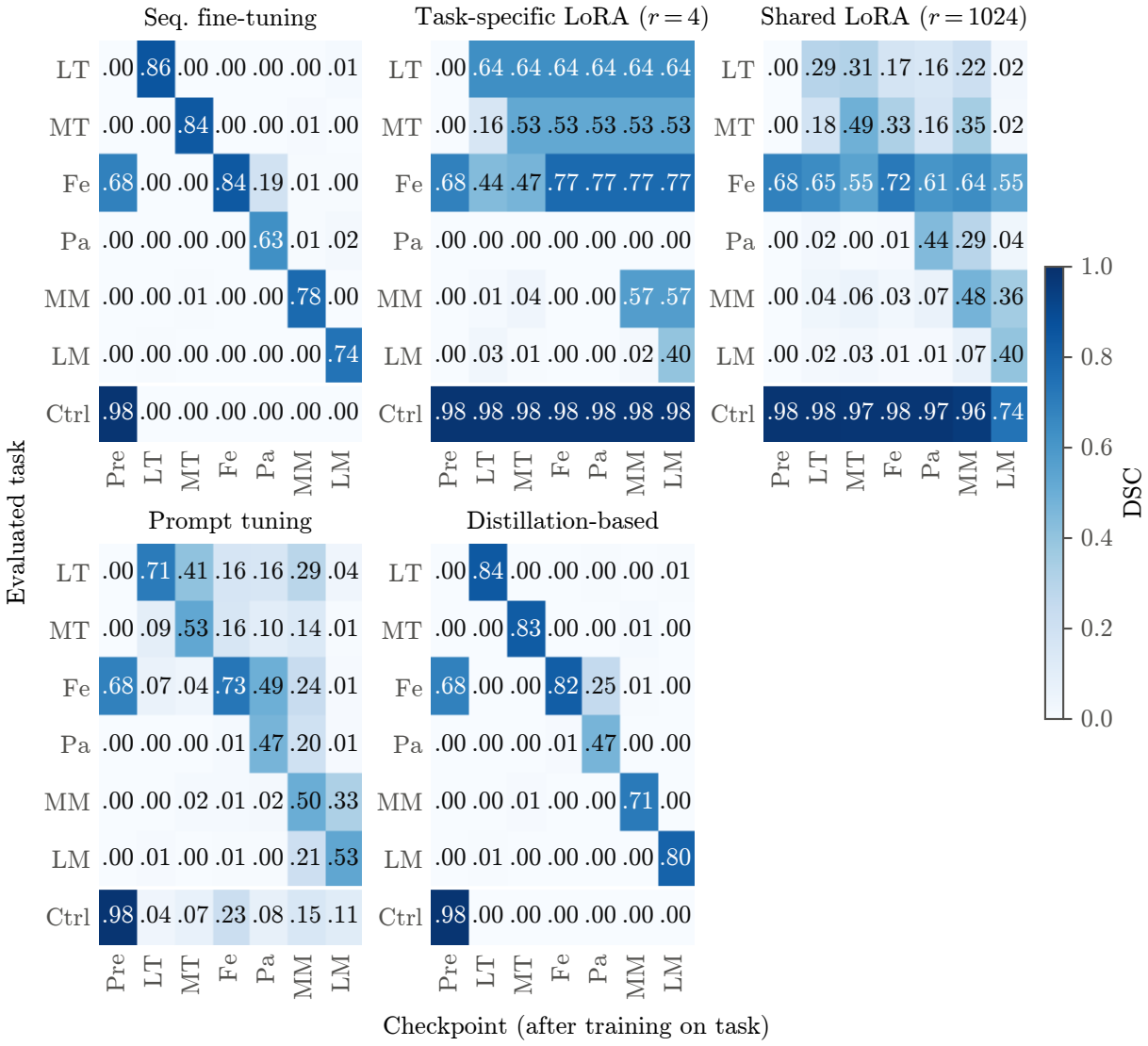


Figure 6.2: Evaluation matrices of the five strategies without replay: mean DSC of each task (rows) at each checkpoint (columns), from the pretrained model (Pre) to the model after the last task. LT, MT: lateral and medial tibial cartilage; Fe: femoral cartilage; Pa: patellar cartilage; MM, LM: medial and lateral meniscus; Ctrl: control task (lung, AeroPath).

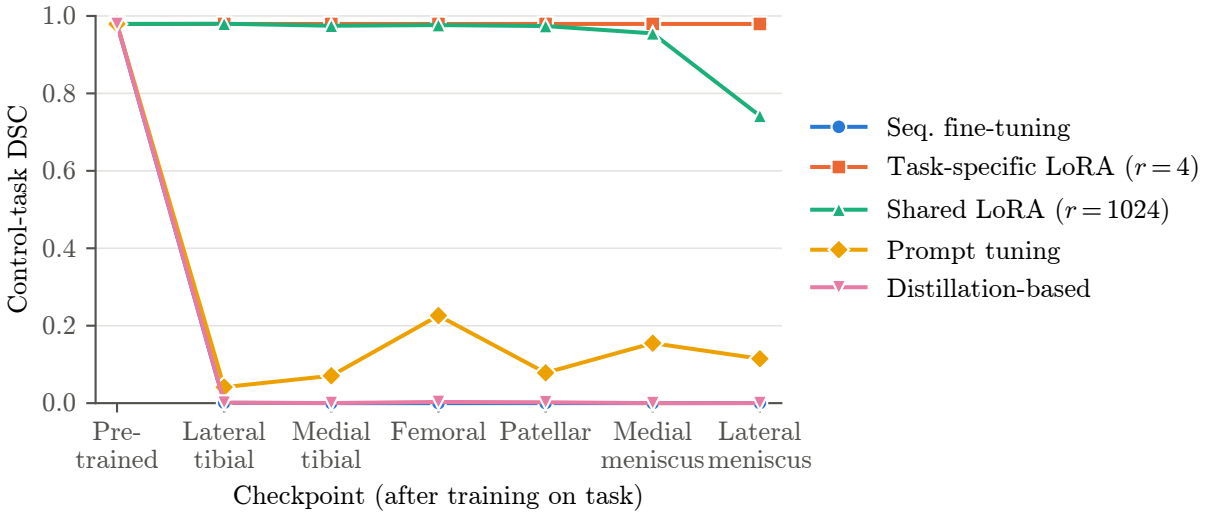


Figure 6.3: DSC on the control task (lung, AeroPath) at each checkpoint of the five strategies without replay. Task-specific LoRA evaluates the control task with the unchanged pretrained model. The curves of sequential fine-tuning and the distillation-based strategy lie on top of each other.

than that of ZSCL and was not tuned, and with distillation on the current task's data (Section 5.4.4); they do not show that feature distillation cannot reduce forgetting more in this setting. The training logs record only the total loss, so we cannot determine how much the distillation term contributed to it.

defense: distillation weight lambda = 0.01 untuned; distillation term not logged separately; result holds only for this configuration

The prompt-tuning strategy keeps all pretrained weights frozen and trains 12.6 million parameters, a quarter of the 50.3 million of shared LoRA (Table 6.10). Against sequential fine-tuning, it forgot less, by 0.272 $[0.253, 0.293]$ in Final Forgetting, at an Adaptation lower by 0.201 $[0.185, 0.218]$, and its control-task drop was lower by 0.114 $[0.097, 0.137]$, but it still lost most of the control task. Because the learned prompt tokens and the meta-net change the keys, values and query for every text prompt, the frozen backbone does not keep the outputs for other text prompts unchanged (Section 5.4.5); whether this explains the loss was not tested (Section 7.1).

In Transfer, the LoRA strategies, with 0.129 and 0.174, stayed close to the pretrained model with 0.137, whereas sequential fine-tuning, the distillation-based and the prompt-tuning strategy reached 0.001, 0.002 and 0.043: before a task was trained, they segmented almost none of it, including femoral cartilage, which the pretrained model segments.

Replay lowered the forgetting of shared LoRA without changing its Adaptation. With replay, Final Forgetting was 0.046 instead of 0.289 (-0.243, $[-0.259, -0.226]$) and Final Performance 0.450 instead of 0.233 ($+0.218$, $[0.203, 0.231]$), while Adaptation differed by

only 0.001 $[-0.002, 0.004]$. Its Final Performance is the highest of all strategies that use one parameter set for all tasks and is 0.034 $[0.011, 0.059]$ below that of task-specific LoRA, which needs the task identity at inference. At the end of the sequence, every structure was still segmented, with a DSC between 0.38 for medial tibial cartilage and 0.70 for femoral cartilage. Replay also lowered the control-task drop, from 0.237 to 0.138 (-0.100, $[-0.103, -0.096]$), although the memory holds no control-task samples (Section 5.4.3). As without replay, most of the drop arose in the last task, in which the DSC on the lung fell from 0.931 to 0.842, against 0.955 to 0.742 without replay. In that task, 55 of the 163 samples of an epoch came from the memory, so the adapter was trained on a mixture of structures; whether this is why it specialised less to the last task, and thereby lost less of the control task, was not tested. Because the drop is concentrated in the last task, this effect may also depend on the task order (Section 6.6).

defense: replay lowers the control-task drop although the memory holds no control samples; explanation (mixture of structures in the last task) not tested; may depend on the task order

Joint fine-tuning reached a Final Performance of 0.852, with a DSC between 0.834 for patellar cartilage and 0.873 for femoral cartilage. This is 0.367 $[0.345, 0.389]$ more than task-specific LoRA and 0.722 $[0.710, 0.734]$ more than sequential fine-tuning, and it shows how far all strategies remain from what the same model reaches when it sees all tasks at once. Joint fine-tuning lost the control task as completely as sequential fine-tuning, with a drop of 0.979, although it receives the same number of updates and is trained without a task sequence. The loss of the control task therefore comes from fine-tuning all parameters on the knee structures, not from training the structures one after another.

Table 6.5 shows the additional evaluations of Section 5.3. For sequential fine-tuning, the checkpoint with the lowest validation loss reached an Adaptation 0.059 lower $[0.053, 0.065]$ and a Final Forgetting 0.070 lower $[0.063, 0.077]$ than the parameters after the last epoch, at the same Final Performance; for the distillation-based and the prompt-tuning strategy, the two evaluations differ by at most 0.005. The checkpoint selection on a noisy validation loss (Section 6.1.3) thus lowers the Adaptation of sequential fine-tuning, and evaluating the last epoch avoids this.

The weight ensemble of the distillation-based strategy halved its Adaptation, from 0.747 to 0.376 (-0.371, $[-0.385, -0.356]$), lowered its Final Performance by 0.011 $[0.007, 0.017]$ and did not change its control-task drop. Its Final Forgetting is lower, 0.354 instead of 0.734, only because it learned less in the first place: right after training, its DSC fell from 0.828 on the first task to 0.219 on the fourth and to 0.002 and 0.000 on the last two. A possible explanation follows from the ensemble's definition: as the equally weighted mean of θ_0 and all snapshots, with 100 snapshots per task, a snapshot taken in the last task

enters with a weight of about 1/600, so the ensemble stays close to the earlier states and hardly follows later tasks. It does not protect the control task, because the student had already lost it after the first task (DSC 0.001) and the ensemble mainly averages such states. ZSCL updates its ensemble every 100 iterations and distils on a reference dataset (Section 5.4.4), so these results apply to our configuration only.

defense: weight ensemble with an interval of 10 and without reference dataset; explanation of the low Adaptation (weights of late snapshots) not tested

The per-task gradient scaling of the prompt-tuning strategy lowered its forgetting. At the best validation checkpoint of both runs, the run with scaling forgot 0.063 less [0.052, 0.074], lost 0.117 less of the control task [0.100, 0.139] and reached a Final Performance higher by 0.020 [0.014, 0.025], at an Adaptation lower by 0.032 [0.025, 0.040]. Without scaling, the DSC on the lung fell to 0.019 after the second task and to 0.000 at the end; with scaling, it stayed between 0.04 and 0.23. Scaling the steps down in later tasks thus traded a little adaptation for retention, but it did not keep the control task.

No strategy combined the Adaptation of sequential fine-tuning with the retention of the LoRA strategies. The strategies that update all parameters reached the highest Adaptation and left a model that segments little beyond the last task and has lost the control task and femoral cartilage, abilities of the pretrained model. Joint fine-tuning shows that the loss of the control task does not require a task sequence: with the same number of updates on all six tasks at once, its DSC on the control task also fell to 0.000, while its Final Performance of 0.852 was higher than that of every strategy. The prompt-tuning strategy lies between the two groups, and gradient scaling moved it towards retention; freezing the backbone alone did not preserve the control task. Retention came with the LoRA strategies, at an Adaptation about 0.30 lower than that of sequential fine-tuning. Part of this gap comes from the training configuration rather than from the strategies, because the LoRA strategies were limited by the ten epochs per task (Section 6.4), so the trade-off is probably less steep than these runs show. If the task identity is available at inference, task-specific LoRA gave the highest Final Performance; without it, shared LoRA with replay came within 0.034 of it and kept most of the control task.

defense: one run per strategy; conclusion drafted from Claude's suggestions accepted by the author

6.3 LoRA Rank

This experiment asks how the rank of the LoRA adapters affects adaptation and forgetting. We trained shared LoRA with seven ranks from $r = 4$ to $r = 2048$ and task-specific LoRA with $r = 4$ and $r = 8$, all on the full training split in the original task order (Table 6.1). In shared LoRA, $\alpha = 32$ is the same for all ranks, so the scaling factor $\gamma_r = \alpha/\sqrt{r}$ falls from

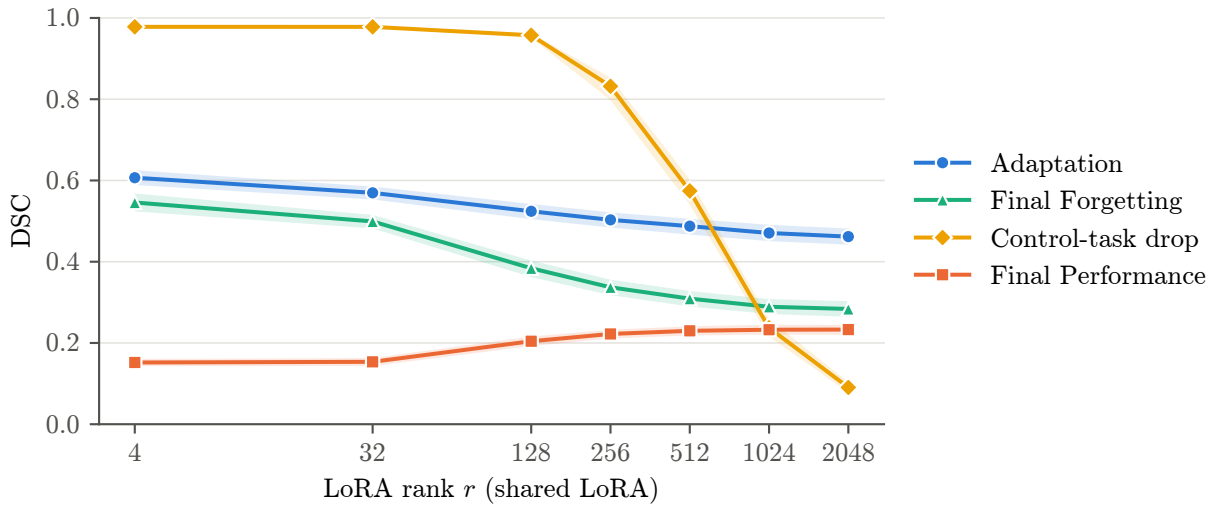


Figure 6.4: Metrics of shared LoRA over the rank r (logarithmic axis). Shaded bands: 95 % bootstrap intervals over the test volumes (Section 6.1.4).

16 at $r = 4$ to 0.71 at $r = 2048$ (Table 6.3). Table 6.6 gives all metrics, and Figure 6.4 shows the trend for shared LoRA.

For shared LoRA, the metrics change monotonically with the rank. From $r = 4$ to $r = 2048$, Adaptation falls from 0.607 to 0.462, while Final Forgetting falls from 0.546 to 0.284 and the control-task drop from 0.978 to 0.091. Final Performance rises from 0.152 to 0.233 and stays at this level from $r = 512$ on, and Transfer rises from 0.045 to about 0.17 from $r = 128$ on. These changes are several times larger than the bootstrap intervals of the individual runs, which are about ± 0.02 (Figure 6.4). The control-task drop changes most: up to $r = 32$, the lung is no longer segmented at all after the last task, and it is largely kept only at the two highest ranks.

The gain in Final Performance comes from two structures. Femoral cartilage, which the pretrained model already segments with a DSC of 0.684, is lost by the end of the sequence at low ranks but kept at 0.55 from $r = 512$ on, whereas lateral tibial, medial tibial and patellar cartilage end below 0.06 at every rank. The fall in Adaptation comes mostly from the first task, lateral tibial cartilage, whose DSC right after training falls from 0.69 at $r = 4$ to 0.28 at $r = 2048$; the other tasks lose at most 0.17. The weak first task recurs in the task-order experiment (Section 6.6).

For task-specific LoRA, $r = 8$ reached a lower Adaptation than $r = 4$ (0.404 against 0.485). The difference comes from lateral tibial cartilage (0.52 against 0.64) and lateral meniscus (0.12 against 0.40), while the other tasks differ by at most 0.05. Patellar cartilage

was not learned at either rank (DSC 0.00 right after training). Each rank was trained once, so we cannot determine whether such differences between single tasks come from the rank or from the training randomness.

Because α is fixed, the rank and the scaling factor change together in the shared-LoRA runs, so this experiment does not separate the effect of the adapter's capacity from that of its scale, and we did not compare γ_r with the scaling α/r of the original LoRA.

defense: rank and γ_r confounded in the shared-LoRA rank ablation (alpha fixed); rsLoRA intends rank-stable training, not tested here

For shared LoRA, the rank sets a trade-off between adaptation and retention: higher ranks forget less, both of the trained tasks and of the control task, at a moderate cost in Adaptation, and Final Performance stops improving from $r = 512$ on. Why higher ranks retain more remains open. rsLoRA chooses the factor $1/\sqrt{r}$ so that the magnitude of the adapter's outputs and gradients does not depend on the rank [Kal23, Sec. 3], so smaller updates at higher rank are not what this scaling is meant to produce; this experiment cannot determine whether capacity, scale or both cause the trend. For the comparison of the strategies we use $r = 1024$. It has half the trainable parameters of $r = 2048$ (50.3 against 100.7 million) and the same Final Performance, and its Adaptation and Final Forgetting differ by less than 0.01. The control-task drop, however, still falls from 0.237 to 0.091 between the two ranks, so the comparison of the strategies understates how much of the control task shared LoRA can keep. For task-specific LoRA, doubling the rank brought no gain in this run.

defense: r = 1024 chosen for parameter count and saturation of Adaptation, Final Forgetting and Final Performance; control

6.4 Diagnosis of the Patellar Failure

Task-specific LoRA did not learn patellar cartilage in the task sequence: the DSC right after training on it was 0.000 at both ranks (Section 6.3), although shared LoRA reached 0.44 to 0.51 on the same task. The pretrained model also segments patellar cartilage with a DSC of 0.000, and task-specific LoRA starts every task from this model, whereas the shared adapter has already been trained on three tasks when it reaches patellar cartilage. To find the cause, we trained task-specific LoRA on patellar cartilage alone, first with the settings of the sequence and then changing one setting at a time: the rank ($r = 16$), the learning rate ($\eta_0 = 3 \cdot 10^{-4}$), the training seed and the number of epochs (50 instead of 10). Each change tests one explanation: too little capacity, too small steps, an unlucky initialisation or too little training. Apart from the changed setting and the single task, the runs use the configuration of the sequence. Table 6.7 gives the results and Figure 6.5 the training loss.

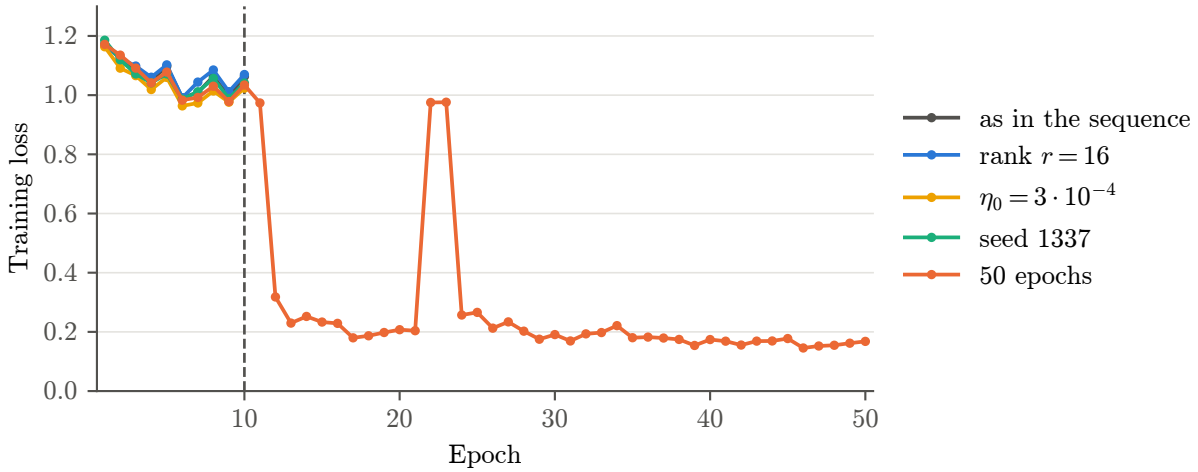


Figure 6.5: Mean training loss per epoch of the runs of Table 6.7. The four runs with 10 epochs lie on top of each other. Dashed line: end of training in the task sequence (epoch 10).

With 10 epochs, every run ended with a DSC of 0.000, whether with the settings of the sequence, a four times larger rank, a three times larger learning rate or another seed. With 50 epochs, the same configuration reached 0.839. The training loss explains the difference: it stays at about 1.0 during the first ten epochs in every run, close to its value for an empty prediction, for which the Dice term alone is 1; only in the 50-epoch run does it drop, from 0.97 at epoch 11 to 0.32 at epoch 12, and it then stays at about 0.2. At epoch 22, the loss of this run jumps back to about 1.0 for two epochs and then returns to its previous level.

Ten epochs were therefore too few for task-specific LoRA to leave the empty prediction on patellar cartilage, while capacity, step size and seed were not the cause in these runs. The 50-epoch run differs from the others in two ways, however: it has five times as many updates, and its learning rate decays over these 5,400 updates, so that at epoch 10 it is still at 80 % of η_0, while in the 10-epoch runs it has reached almost zero. The experiment does not separate these two effects. For the comparison of the strategies, the result means that the Adaptation of task-specific LoRA is limited by the training length of the shared configuration (Section 6.1.3) and not only by the strategy itself.

defense: shared training configuration (10 epochs) understates task-specific LoRA on patellar

6.5 Amount of Labelled Data

This experiment asks how the amount of labelled data affects adaptation and forgetting. Sequential fine-tuning, task-specific LoRA ($r = 4$) and shared LoRA ($r = 1024$) were

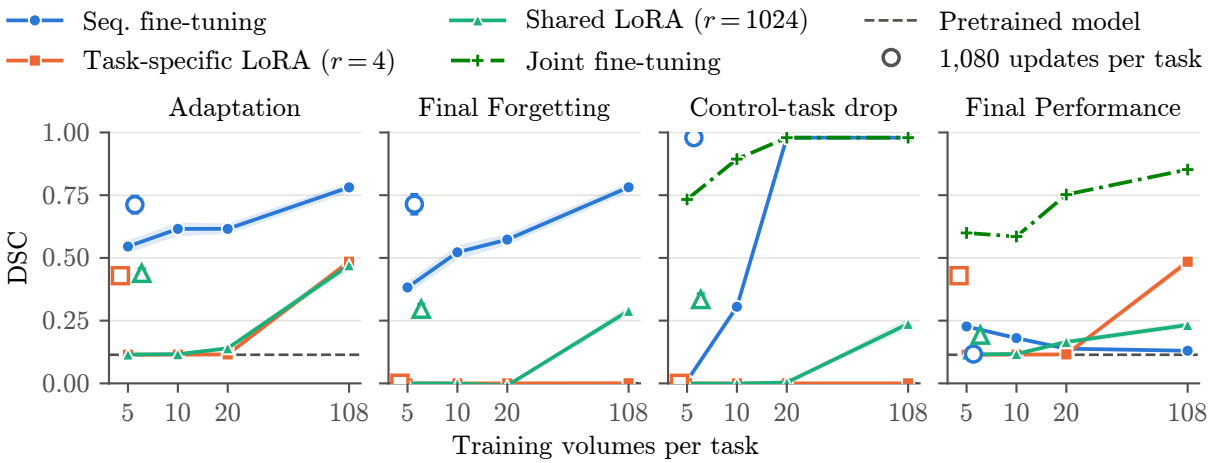


Figure 6.6: Metrics over the number of training volumes per task, with 95 % bootstrap intervals (Section 6.1.4). Filled markers and solid lines: ten epochs per task; hollow markers at 5 volumes: equal-updates runs with 1,080 updates per task, shifted slightly to the side for legibility. Dash-dotted line: joint fine-tuning, shown for the two metrics defined for it. Dashed line: pretrained model.

trained with $k = 5$, 10 and 20 training volumes per task and compared with the same strategy on the full training split (Section 6.1.2). Because the number of epochs is fixed, the few-shot runs also receive fewer updates: 50, 100 and 200 per task instead of 1,080. The equal-updates runs with $k = 5$ and 1,080 updates per task separate the two factors for this setting, and joint fine-tuning with the same k serves as the reference (Section 6.1.2). Table 6.8 gives all metrics and Figure 6.6 the main ones.

Sequential fine-tuning learned every task in all three few-shot settings, and with fewer volumes it forgot less. With $k = 5$, its Adaptation was 0.545, lower by 0.236 $[0.217, 0.255]$ than with the full split, while its Final Forgetting was 0.382 instead of 0.782 and it kept the control task, with a drop of 0.010 instead of 0.979. Its Final Performance was therefore higher than with the full split, 0.227 against 0.130 (difference 0.097, $[0.090, 0.104]$), close to the 0.233 of shared LoRA on the full split (Section 6.2), and it ended with a DSC of 0.42 on femoral cartilage, which the full-split run lost. With $k = 10$ and $k = 20$, the metrics lie between these two: Adaptation 0.616 in both, Final Forgetting 0.522 and 0.573, Final Performance 0.180 and 0.138. The control task was lost gradually with $k = 10$, down to 0.435 after the fifth task and 0.674 at the end (drop 0.305), and within two tasks with $k = 20$ (0.587 after the first, 0.005 after the second), as with the full split after the first task.

With 5 and 10 volumes per task, neither LoRA strategy learned any task. Their Adaptation of 0.114 to 0.116 equals that of the pretrained model (0.114), and the evaluation matrices show why: right after training, every structure except femoral cartilage has a DSC of 0.00, and femoral cartilage stays at the 0.69 to 0.70 that the pretrained model already reaches. Final Forgetting and the control-task drop are therefore zero for both strategies as well. With 20 volumes, task-specific LoRA still did not learn any task, while shared LoRA began to: its Adaptation rose to 0.140 and its Final Performance to 0.165, with DSC values of 0.12 for medial tibial cartilage, 0.09 for patellar cartilage and 0.07 for medial meniscus at the end of the sequence. Its Final Forgetting is slightly negative (-0.009), because these values rose further while later tasks were trained. With the full training split, the two strategies reached an Adaptation of 0.485 and 0.471 (Section 6.3).

The equal-updates runs show that the number of updates, not the number of volumes, explains most of this failure at $k = 5$. With 1,080 updates per task on 5 volumes, task-specific and shared LoRA reached an Adaptation of 0.429 and 0.438, higher by 0.315 $[0.293, 0.334]$ and 0.323 $[0.306, 0.341]$ than with 50 updates and lower by only 0.056 $[0.040, 0.073]$ and 0.032 $[0.025, 0.039]$ than with the full split. Task-specific LoRA again did not learn patellar cartilage (DSC 0.000 right after training), in agreement with Section 6.4, where 1,080 updates were not enough for this structure. With more updates, shared LoRA also forgot again: its Final Forgetting of 0.296 equals that of the full split within $[-0.007, 0.018]$, and its control-task drop of 0.334 is 0.097 higher. Sequential fine-tuning with 1,080 updates on 5 volumes reached an Adaptation of 0.712, 0.167 $[0.142, 0.190]$ more than with 50 updates, and forgot as much as with the full split: Final Forgetting 0.714, control-task drop 0.979 and Final Performance 0.117.

Joint fine-tuning reached a Final Performance of 0.600, 0.585 and 0.753 with 5, 10 and 20 volumes per structure, against 0.852 with the full split. Between $k = 5$ and $k = 10$, no difference can be detected (-0.014, $[-0.037, 0.013]$); with one draw of volumes per setting, we do not interpret this. With 5 volumes, joint fine-tuning ended 0.373 $[0.334, 0.406]$ above sequential fine-tuning and 0.170 $[0.131, 0.205]$ above task-specific LoRA with equal updates. Its control-task drop rose with the amount of training, from 0.733 with $k = 5$ over 0.894 with $k = 10$ to 0.979 with $k = 20$ and with the full split, for 300, 600 and 1,200 updates in total.

With ten epochs per task, sequential fine-tuning was the only strategy that learned the tasks under few-shot conditions, and the LoRA strategies did not move away from the pretrained model within 50 to 200 updates per task. At $k = 5$, the fixed epoch budget

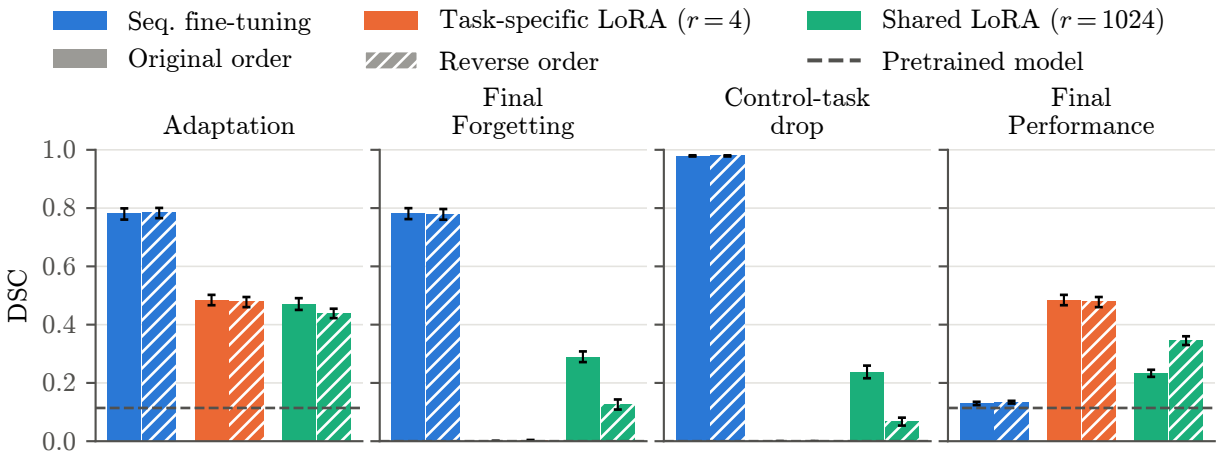


Figure 6.7: Metrics in the original (plain bars) and the reverse task order (hatched bars), with 95 % bootstrap intervals. Dashed line: pretrained model.

explains most of the failure of the LoRA strategies: with the update count of the full split, they came within 0.06 of their full-split Adaptation. For sequential fine-tuning, the lower forgetting and the kept control task with few volumes are consistent with an effect of the number of updates rather than of the amount of data: with 1,080 updates on 5 volumes, it forgot as much and lost the control task as completely as with the full split, and the control-task drop of joint fine-tuning also rose with the number of updates. With 5 volumes per task, sequential fine-tuning reached about the Final Performance of shared LoRA on the full split; we read this as a higher Final Performance of a less trained model, not as a benefit of less data. These conclusions rest on the separation at $k = 5$, with one draw of volumes, one run per configuration and a learning rate that was not tuned; for $k = 10$ and $k = 20$, data and updates remain confounded.

defense: data and updates separated only at k = 5; one support set; one seed; learning rate untuned

6.6 Task Order

This experiment asks whether the results depend on the order in which the structures are learned. Sequential fine-tuning, task-specific LoRA ($r = 4$) and shared LoRA ($r = 1024$) were trained in the original and in the reverse task order on the full training split (Section 6.1.2). Table 6.9 gives all metrics and Figure 6.7 the main ones.

For sequential fine-tuning, no effect of the task order can be detected. In the reverse order, Adaptation was 0.003 higher $[-0.005, 0.010]$, Final Forgetting 0.002 lower $[-0.009, 0.005]$ and Final Performance 0.004 higher $[-0.002, 0.010]$; in both orders, it for-

got almost all earlier tasks (Final Forgetting 0.782 and 0.780) and lost the control task after the first task (DSC 0.001 and 0.000). Only Transfer differed, 0.014 instead of 0.001 (+0.013, $[0.012, 0.014]$), which is small in absolute terms. Unlike shared LoRA, sequential fine-tuning learned the first task well in both orders, lateral tibial cartilage with 0.86 and lateral meniscus with 0.83.

For task-specific LoRA, the task order changes nothing in the training of a task: every task starts from the pretrained model with a new adapter, trained on the same data with the same settings, and the random seed is reset at the start of every task (Section 5.4.2). What does change is the state of the random-number generator when the adapter is created, because the adapter is initialised before the seed is reset, so the adapter of each task receives a different random initialisation in the two orders; in addition, the training allows non-deterministic GPU operations. The two runs therefore show how much a single run varies with its initialisation. Five of the six tasks reached the same DSC within 0.01, while lateral meniscus reached 0.40 in the original and 0.34 in the reverse order, so that Adaptation differs by 0.006 (0.485 against 0.478). The paired bootstrap interval of this difference, $[0.004, 0.008]$, does not contain zero, although the difference comes only from the initialisation and the non-determinism of training. This illustrates the limit stated in Section 6.1.4: the intervals reflect the choice of test volumes, not the variation between training runs, and differences of this size between single runs should not be read as effects of a factor.

For shared LoRA, the reverse order led to better retention. Final Forgetting was 0.124 instead of 0.289 and the control-task drop 0.067 instead of 0.237, and Final Performance rose from 0.233 to 0.345, while Adaptation was 0.032 lower (0.439 against 0.471); the paired intervals of these four differences are about ± 0.01 to ± 0.02 wide. In both orders, the first task was learned poorly: lateral tibial cartilage reached 0.29 right after training in the original order and lateral meniscus 0.01 in the reverse order, whereas the same structures reached 0.51 and 0.40 when trained later in the other order. At the end of the reverse sequence, every structure is still segmented to some degree, from 0.13 for lateral meniscus to 0.51 for lateral tibial cartilage, whereas in the original order three of the six structures end below 0.05. Transfer is about the same in both orders (0.169 and 0.174).

In the original order, most of the forgetting and of the control-task drop arose in the last step. Up to the fifth task, the DSC on the control task stayed between 0.955 and 0.980; training on lateral meniscus lowered it to 0.742, and in the same step lateral tibial cartilage fell from 0.22 to 0.02, medial tibial cartilage from 0.35 to 0.02 and patellar cartilage from

defense: paired intervals exclude zero for a pure re-initialisation difference (task-specific LoRA original vs reverse); training variability not covered

0.29 to 0.04. In the reverse order, the last task, lateral tibial cartilage, lowered the control task only from 0.967 to 0.912, and femoral cartilage, which lost most in this step, from 0.65 to 0.50. Whether lateral meniscus causes this loss as a structure or through its position at the end of the sequence cannot be separated with two orders.

The training loss shows how the first task differs from the later ones. On the first task, the loss starts at about 1.0, its value for an empty prediction, and falls slowly: in the original order to 0.69 after ten epochs, in the reverse order only to 0.96. The second task starts much lower, at 0.81 and 0.86 in the first epoch, and falls to 0.56 in both orders, although in the reverse order the first task was hardly learned at all. With $r = 4$, by contrast, the loss of the first task falls to 0.74 within three epochs and to 0.38 after ten.

For sequential fine-tuning, the task order changed no metric beyond the bootstrap interval except Transfer, by 0.013, and it forgot almost everything in both orders. For task-specific LoRA, the task order has no effect beyond the variation between two runs, as its design implies. For shared LoRA, the task order has a strong effect: in the reverse order, the adapter retained more of its performance on the earlier tasks and on the control task (Final Forgetting 0.124 against 0.289, control-task drop 0.067 against 0.237), at an Adaptation lower by 0.032. With two orders, we cannot determine which property of the order causes this difference in retention, and among the strategies tested, a strong effect of the order appeared only for shared LoRA. In both orders, the first task was learned poorly, and the training loss shows that its adapter had not left its initial state after ten epochs; a possible explanation, which connects this observation with the results of Sections 6.3 to 6.5, is discussed in Section 7.2.

6.7 Computational Cost

This section reports the measured cost of the strategies, which complements the complexity analysis of Section 5.5; the hardware is described in Section 6.1.3. Table 6.10 gives, for the runs of the comparison of the strategies, the number of trainable parameters, the time per epoch, the total time of the run, the evaluation time per volume and the peak GPU memory, all taken from the logs of the runs.

The two LoRA strategies and the prompt-tuning strategy took 18.1 to 19.8 minutes per epoch, 26 to 33 % less than sequential fine-tuning with 26.8 minutes, which agrees with their backward pass ending at the prompt decoder instead of reaching the image encoder (Section 5.5). The earlier run of sequential fine-tuning with the same configuration took

23.7 minutes per epoch; we did not investigate this difference on the shared cluster. The distillation-based strategy took 37.1 minutes per epoch, 1.4 times as long as sequential fine-tuning, because every iteration adds a forward pass of the teacher and its transfer from CPU to GPU memory. A complete run, including its evaluations, took 20.8 and 22.6 hours for the two LoRA strategies, 24.0 hours for the prompt-tuning strategy, 32.5 hours for sequential fine-tuning and 47.1 hours for the distillation-based strategy, whose run includes three evaluations; all but the LoRA runs needed a second job. The peak GPU memory was 20.3 to 20.6 GiB for the parameter-efficient strategies, 31.4 GiB for sequential fine-tuning and 37.9 GiB for the distillation-based strategy, close to the 40 GB of the GPU.

For shared LoRA, the number of trainable parameters grows linearly with the rank, from 0.2 million at $r = 4$ to 100.7 million at $r = 2048$, while the time per epoch only rises from 18.9 to 21.0 minutes and the peak memory from 20.4 to 20.8 GiB. Replay increased the mean time per epoch of shared LoRA with $r = 1024$ from 19.8 to 26.4 minutes, because the replayed samples, up to 55 in the last task, add iterations to every epoch (Section 5.4.3). In the few-shot settings, an epoch of sequential fine-tuning took 3.1 to 6.5 minutes for $k = 5$ to $k = 20$, of which the validation on the 23 validation volumes, which does not depend on k, is a large part. The equal-updates runs, with 216 short epochs per task, took 18.4 to 24.3 hours of training, about as long as the full-data runs of the same strategies. Joint fine-tuning took about 2.3 hours per epoch with its 648 training volumes, about 23 hours for its ten epochs. After T tasks, task-specific LoRA stores six adapters of 0.2 million parameters each, which is negligible compared with the model.

Evaluation took 7.7 to 8.8 seconds per volume in the runs of the task sequence, so that the 1,036 volume inferences of one evaluation of the protocol (Section 5.2) took about 2.2 to 2.5 hours. The forward pass has the same cost for all strategies (Section 5.5); the measured times also include loading the volumes and encoding the text prompts, and we did not investigate the differences between runs.

All strategies fit on a single 40 GB GPU. The parameter-efficient strategies are the cheapest to train, with about a fifth less time per epoch and a third less memory than sequential fine-tuning, and the distillation-based strategy is the most expensive in both time and memory. At inference, the strategies cost the same.

Table 6.1: Overview of the experiments. Each configuration was trained once. Training volumes per task: 108 is the full training split. Objective: number of the research objective in Section 1.3. All runs use the original task order unless stated otherwise.

Experiment	Strategies and configurations	Training volumes per task	Objective	Section
Comparison of the strategies	Seq. fine-tuning; Task-specific LoRA ($r = 4$); Shared LoRA ($r = 1024$); Distillation-based; Prompt tuning	108	2	6.2
LoRA rank	Task-specific LoRA ($r = 4, 8$); Shared LoRA ($r = 4, 32, 128, 256, 512, 1024, 2048$)	108	4	6.3
Diagnosis of the patellar failure	Task-specific LoRA ($r = 4, 16$), patellar cartilage only; rank, learning rate, seed or number of epochs changed	108	4	6.4
Amount of labelled data	Seq. fine-tuning; Task-specific LoRA ($r = 4$); Shared LoRA ($r = 1024$)	5, 10, 20, 108	3	6.5
Task order	Seq. fine-tuning in original and reverse order; Task-specific LoRA ($r = 4$) in original and reverse order; Shared LoRA ($r = 1024$) in original and reverse order	108	4	6.6
Replay	Shared LoRA ($r = 1024$) with and without replay	108	4	6.2
Gradient scaling of the prompt-tuning strategy	Prompt tuning without per-task gradient scaling	108	4	6.2
Amount of labelled data with the update count of the full data	Seq. fine-tuning, 216 epochs per task; Task-specific LoRA ($r = 4$), 216 epochs per task; Shared LoRA ($r = 1024$), 216 epochs per task	5	3	6.5
Upper reference: joint training	Joint fine-tuning	5, 10, 20, 108	2, 3	6.2

Table 6.2: Training and evaluation settings shared by all runs (Section 5.3). Generated from the run configurations; code defaults checked against the code.

Setting	Symbol	Value
Optimiser		SGD, Nesterov
Initial learning rate	η_0	10^{-4}
Momentum	μ	0.99
Weight decay	ω	$3 \cdot 10^{-5}$
Learning-rate schedule		polynomial, exponent 1, per iteration, restarted for every task
Epochs per task	E	10; 216 in the equal-updates runs (validation every 24 epochs)
Batch size		1
Patch size (voxels)		$192 \times 192 \times 192$
Foreground-centred patches	π_{fg}	0.85
Loss	$\mathcal{L}_{\text{seg}}$	binary cross-entropy + Dice, weights 1:1
Deep supervision		off
Data augmentation		none
Random seed (training, split)		42, 42
Sliding-window step		0.5 of the patch size
Gaussian weighting		standard deviation 1/8 of the patch size
Probability threshold		0.5

Table 6.3: Parameters of the strategies (Section 5.4). Sequential fine-tuning has no parameters of its own. Bold: rank used in the comparison of the strategies; scaling factor $\gamma_r = \alpha/\sqrt{r}$ listed per rank. The weight ensemble of the distillation-based strategy is reported as an additional evaluation (Section 5.4.4); the run of the prompt-tuning strategy without gradient scaling has the same parameters with the scale inactive (Section 5.4.5).

Strategy	Parameter	Symbol	Value
Task-specific LoRA	Rank	r	**4**, 8
	Scaling constant	α	16
	Scaling factor	γ_r	8, 5.66
	Dropout (low-rank path)		0.05
	Adapted projections		$\boldsymbol{W}_q, \boldsymbol{W}_v$, cross-attention
Shared LoRA	Rank	r	4, 32, 128, 256, 512, **1024**, 2048
	Scaling constant	α	32
	Scaling factor	γ_r	16, 5.66, 2.83, 2, 1.41, 1, 0.71
	Dropout (low-rank path)		0.1
	Adapted projections		$\boldsymbol{W}_q, \boldsymbol{W}_v$, cross-attention
Shared LoRA + replay	Rank (otherwise as shared LoRA)	r	1024
	Replay fraction	ρ	0.1
	Maximum memory size	$M_{\max}$	200
Distillation-based	Distillation weight	λ	0.01
	Ensemble interval (iterations)	I_{we}	10
Prompt tuning	Learned prompt tokens per layer	m	2
	Prompted decoder layers	L_p	3
	Gradient-scale lower bound		0.25

Table 6.4: Results of the comparison of the strategies (full training split, original task order, parameters after the last epoch). Mean DSC; ↑: higher is better, ↓: lower is better. For task-specific LoRA, Final Forgetting and the control-task drop are zero by construction (Section 5.2). Joint fine-tuning trains one model on all six tasks at once; Adaptation, Final Forgetting and Transfer are not defined for it (–).

Strategy	Adapt.↑	Forg.↓	Ctrl. drop↓	Final↑	Transf.↑
Seq. fine-tuning	0.781	0.782	0.979	0.130	0.001
Task-specific LoRA ($r = 4$)	0.485	0.000	0.000	0.485	0.129
Shared LoRA ($r = 1024$)	0.471	0.289	0.237	0.233	0.174
Prompt tuning	0.580	0.510	0.865	0.155	0.043
Distillation-based	0.747	0.734	0.979	0.135	0.002
Shared LoRA + replay ($r = 1024$)	0.472	0.046	0.138	0.450	0.175
Joint fine-tuning (upper reference)	–	–	0.979	0.852	–
Pretrained model	0.114	0.000	0.000	0.114	0.137

Table 6.5: Additional evaluations and component ablations (full training split, original task order). Evaluation: parameters after the last epoch (main evaluation), the checkpoint with the lowest validation loss, or the weight ensemble of the distillation-based strategy (Sections 5.3 and 5.4.4). "No scaling": the earlier run of the prompt-tuning strategy with the per-task gradient scale inactive, evaluated only at the best validation checkpoint (Section 5.4.5). Mean DSC; ↑: higher is better, ↓: lower is better.

Strategy	Evaluation	Adapt.↑	Forg.↓	Ctrl. drop↓	Final↑	Transf.↑
Seq. fine-tuning	last epoch	0.781	0.782	0.979	0.130	0.001
Seq. fine-tuning	best validation	0.723	0.712	0.979	0.130	0.001
Prompt tuning	last epoch	0.580	0.510	0.865	0.155	0.043
Prompt tuning	best validation	0.576	0.505	0.863	0.155	0.039
Prompt tuning (no scaling)	best validation	0.608	0.568	0.979	0.135	0.043
Distillation-based	last epoch	0.747	0.734	0.979	0.135	0.002
Distillation-based	best validation	0.745	0.732	0.979	0.135	0.002
Distillation-based	weight ensemble	0.376	0.354	0.979	0.124	0.007
Pretrained model		0.114	0.000	0.000	0.114	0.137

Table 6.6: Results of the LoRA rank experiment (full training split, original task order). Mean DSC; ↑: higher is better, ↓: lower is better. For task-specific LoRA, Final Forgetting and the control-task drop are zero by construction (Section 5.2).

Strategy	Rank r	Adapt.↑	Forg.↓	Ctrl. drop↓	Final↑	Transf.↑
Task-specific LoRA	4	0.485	0.000	0.000	0.485	0.129
Task-specific LoRA	8	0.404	0.001	0.000	0.404	0.172
Shared LoRA	4	0.607	0.546	0.978	0.152	0.045
Shared LoRA	32	0.570	0.499	0.978	0.154	0.114
Shared LoRA	128	0.524	0.384	0.957	0.204	0.170
Shared LoRA	256	0.503	0.337	0.832	0.222	0.173
Shared LoRA	512	0.488	0.309	0.574	0.230	0.175
Shared LoRA	1024	0.471	0.289	0.237	0.233	0.174
Shared LoRA	2048	0.462	0.284	0.091	0.233	0.175
Pretrained model		0.114	0.000	0.000	0.114	0.137

Table 6.7: Task-specific LoRA trained on patellar cartilage alone, changing one setting of the sequence configuration per run ("none": settings of the sequence). Seed: training seed; the data split is the same in all runs. Updates: training iterations (108 volumes per epoch). DSC on the 24 patellar test volumes right after training, with the 95 % bootstrap interval.

Changed setting	Rank r	η_0	Seed	Epochs	Updates	DSC [95 % interval]
none	4	10^{-4}	42	10	1,080	0.000 [0.000, 0.000]
Rank	16	10^{-4}	42	10	1,080	0.000 [0.000, 0.000]
Learning rate	4	$3 \cdot 10^{-4}$	42	10	1,080	0.000 [0.000, 0.000]
Seed	4	10^{-4}	1337	10	1,080	0.000 [0.000, 0.000]
Epochs	4	10^{-4}	42	50	5,400	0.839 [0.811, 0.864]

Table 6.8: Results of the few-shot experiment (original task order, parameters after the last epoch). Vol. (updates): training volumes per task and updates per task; 108 is the full training split, and 5 (1,080) are the equal-updates runs. Joint fine-tuning trains one stage on $6k$ volumes, so it has no updates per task; Adaptation, Final Forgetting and Transfer are not defined for it (–). Mean DSC; ↑: higher is better, ↓: lower is better.

Strategy	Vol. (updates)	Adapt.↑	Forg.↓	Ctrl. drop↓	Final↑	Transf.↑
Seq. fine-tuning	5 (50)	0.545	0.382	0.010	0.227	0.092
Seq. fine-tuning	5 (1,080)	0.712	0.714	0.979	0.117	0.023
Seq. fine-tuning	10 (100)	0.616	0.522	0.305	0.180	0.035
Seq. fine-tuning	20 (200)	0.616	0.573	0.979	0.138	0.022
Seq. fine-tuning	108 (1,080)	0.781	0.782	0.979	0.130	0.001
Task-specific LoRA ($r = 4$)	5 (50)	0.114	0.000	0.000	0.114	0.137
Task-specific LoRA ($r = 4$)	5 (1,080)	0.429	0.000	0.000	0.429	0.059
Task-specific LoRA ($r = 4$)	10 (100)	0.115	0.000	0.000	0.115	0.137
Task-specific LoRA ($r = 4$)	20 (200)	0.116	0.000	0.000	0.116	0.138
Task-specific LoRA ($r = 4$)	108 (1,080)	0.485	0.000	0.000	0.485	0.129
Shared LoRA ($r = 1024$)	5 (50)	0.115	0.000	−0.001	0.115	0.137
Shared LoRA ($r = 1024$)	5 (1,080)	0.438	0.296	0.334	0.192	0.175
Shared LoRA ($r = 1024$)	10 (100)	0.116	0.000	−0.001	0.117	0.138
Shared LoRA ($r = 1024$)	20 (200)	0.140	−0.009	0.004	0.165	0.140
Shared LoRA ($r = 1024$)	108 (1,080)	0.471	0.289	0.237	0.233	0.174
Joint fine-tuning	5 (–)	–	–	0.733	0.600	–
Joint fine-tuning	10 (–)	–	–	0.894	0.585	–
Joint fine-tuning	20 (–)	–	–	0.979	0.753	–
Joint fine-tuning	108 (–)	–	–	0.979	0.852	–
Pretrained model		0.114	0.000	0.000	0.114	0.137

Table 6.9: Results of the task-order experiment (full training split). The pretrained-model reference is computed for each task order. Mean DSC; ↑: higher is better, ↓: lower is better.

Strategy	Task order	Adapt.↑	Forg.↓	Ctrl. drop↓	Final↑	Transf.↑
Seq. fine-tuning	Original	0.781	0.782	0.979	0.130	0.001
Seq. fine-tuning	Reverse	0.784	0.780	0.979	0.134	0.014
Task-specific LoRA ($r = 4$)	Original	0.485	0.000	0.000	0.485	0.129
Task-specific LoRA ($r = 4$)	Reverse	0.478	0.002	0.000	0.478	0.132
Shared LoRA ($r = 1024$)	Original	0.471	0.289	0.237	0.233	0.174
Shared LoRA ($r = 1024$)	Reverse	0.439	0.124	0.067	0.345	0.169
Pretrained model	Original	0.114	0.000	0.000	0.114	0.137
Pretrained model	Reverse	0.114	0.000	0.000	0.114	0.137

Table 6.10: Measured cost of the strategies in the comparison (full training split, original task order, one NVIDIA A100 40 GB). Trainable param.: parameters set as trainable, in millions; of the 440.03 million of sequential fine-tuning and the distillation-based strategy, 339.29 million receive gradients (Section 5.4.1). Epoch: mean wall-clock time of one training epoch including validation. Wall-clock: total time of all jobs of the run, including the evaluation of the protocol and restarts after the 24-hour limit. Evaluation: median time per test volume, including data loading and text encoding. Peak GPU mem.: maximum over all jobs of the run.

Strategy	Trainable param. (M)	Epoch (min)	Wall-clock (h)	Evaluation (s/volume)	Peak GPU mem. (GiB)
Seq. fine-tuning	440.03	26.8	32.5	8.8	31.4
Task-specific LoRA ($r = 4$)	0.20	18.1	20.8	7.9	20.3
Shared LoRA ($r = 1024$)	50.33	19.8	22.6	8.5	20.6
Prompt tuning	12.60	18.9	24.0	8.0	20.3
Distillation-based	440.03	37.1	47.1	8.7	37.9

Chapter 7

Discussion

This chapter interprets the results of Chapter 6 across the experiments; the conclusion of each experiment is given in its own section and is not repeated here. Section 7.1 answers the research question objective by objective, Section 7.2 discusses a possible common cause of several results, Section 7.3 states the limitations and threats to validity, and Section 7.4 discusses how far the results generalise and how the work could be extended.

7.1 Answer to the Research Question

The research question asks how continual-learning strategies trade adaptation against catastrophic forgetting when a free-text promptable 3D segmentation model is adapted under few-shot conditions. In our runs, the strategies that update all parameters adapted well and forgot almost everything, the LoRA strategies retained much and adapted less, and the prompt-tuning strategy lay between them; replay brought shared LoRA closest to combining both without the task identity, and joint fine-tuning showed that the loss of pretrained ability does not need a task sequence. With few labelled volumes and ten epochs per task, only sequential fine-tuning learned, but with the update count of the full split, the LoRA strategies learned from 5 volumes as well. We answer the objectives in the order of the experiments and end with the protocol.

The second objective compares the strategies in their adaptation and forgetting. Sequential fine-tuning and the distillation-based strategy reached an Adaptation of 0.781 and 0.747 but ended with a Final Performance of 0.130 and 0.135, barely above the 0.114 of the pretrained model, and lost the control task and femoral cartilage, which the pretrained model segmented (Section 6.2). After six tasks, these strategies leave a model that

author: confirm this overall answer; it condenses the conclusions of Sections 6.2 and 6.5

segments little beyond the last structure, so the universality that motivates adapting a universal model is lost rather than extended. Joint fine-tuning, with the same number of updates on all six tasks at once, reached a Final Performance of 0.852 but also lost the control task completely, so the loss of pretrained ability comes from fine-tuning all parameters on the knee structures, not from the task sequence; the loss of the earlier tasks, in contrast, comes from the sequence. The prompt-tuning strategy lay between the two groups, with an Adaptation of 0.580, a Final Forgetting of 0.510 and a control-task drop of 0.865, although its pretrained weights are frozen and it trains a quarter of the parameters of shared LoRA. Its learned prompt tokens and its meta-net act on the query, keys and values of every text prompt (Section 5.4.5), so a frozen backbone alone does not protect the outputs for other text prompts; shared LoRA also acts on every text prompt, however, and retained much more, so the number of trainable parameters and freezing the backbone do not by themselves explain retention in our runs, and we did not test which property does. The LoRA strategies retained more at an Adaptation about 0.30 lower, a gap that the shared budget of ten epochs per task probably widens (Section 7.2). Distillation from the pretrained model reduced forgetting only slightly in the configuration we evaluated, by 0.048 in Final Forgetting, with an untuned weight $\lambda = 0.01$, so this result concerns that configuration, not distillation in general. Replay was the only addition that moved shared LoRA towards both goals at once: with a memory of earlier tasks only, its Final Forgetting was 0.046 instead of 0.289, its Final Performance 0.450 instead of 0.233, and its control-task drop 0.138 instead of 0.237, at the same Adaptation (Section 6.2). In MedPEFT-CL, a framework for continual learning of a 2D text-prompted segmentation model with LoRA, replay likewise gave the largest gain of the components added one at a time [Gao+26a, Table 8]. In answer to the second objective, no strategy combined high adaptation with retention: the two strategies with the highest Adaptation, sequential fine-tuning and the distillation-based strategy, had a Final Forgetting of 0.782 and 0.734, task-specific LoRA gave the highest Final Performance but needs the task identity at inference, and shared LoRA with replay came within 0.034 of it without that requirement; all strategies stayed at least 0.36 below the Final Performance of joint fine-tuning (0.852).

check: MedPEFT-CL Table 8 numbers read from arXiv v1; bib cites the WACV 2026 version

The third objective concerns the amount of labelled data. With 5 and 10 volumes per task, neither LoRA strategy learned any task, and with 20 only shared LoRA began to, whereas sequential fine-tuning learned every task in every few-shot setting (Section 6.5). With fewer volumes, sequential fine-tuning adapted less and forgot less: with 5 volumes, it kept the control task and reached a Final Performance of 0.227, against 0.130 with the full

split. Because the number of epochs was fixed, fewer volumes also meant fewer updates, 50 instead of 1,080 per task. The equal-updates runs separate the two at $k = 5$: with 1,080 updates on 5 volumes, both LoRA strategies learned the tasks, and reducing the data from 108 to 5 volumes at the same number of updates lowered the Adaptation by only 0.032 to 0.069 for the three strategies, while sequential fine-tuning forgot again as much as with the full split (Section 6.5). The control-task drop of joint fine-tuning, which rose from 0.733 to 0.979 with the number of updates, points the same way. That LoRA can learn from 10 labelled volumes of a 3D model when the training budget is larger has also been shown by Silva-Rodríguez et al. (Section 7.2). In answer to the third objective, at $k = 5$ the amount of labelled data had a small effect on Adaptation once the number of updates was held fixed, and the number of updates decided whether the parameter-efficient strategies learned and how much sequential fine-tuning forgot; this rests on one draw of volumes and one run per configuration, and for $k = 10$ and $k = 20$ data and updates remain confounded.

The fourth objective asks how the task order, the LoRA rank and the components of the strategies influence adaptation and forgetting. The task order mattered strongly only for shared LoRA. For task-specific LoRA, the order changed nothing beyond the variation between two runs, as its design implies, and for sequential fine-tuning no effect of the order could be detected, with differences of at most 0.004 outside Transfer, whereas for shared LoRA the reverse order reduced Final Forgetting from 0.289 to 0.124 and the control-task drop from 0.237 to 0.067, at an Adaptation lower by 0.032 (Section 6.6). In the original order, most of this forgetting arose in the last step, when lateral meniscus was trained. The metrics of shared LoRA after the sequence are therefore a property of the strategy together with the order, not of the strategy alone, and with two orders we cannot determine whether lateral meniscus causes the loss as a structure or through its position. The Beyond Forgetting benchmark found that the task order affects the results of several strategies over ten random orders of 2D prostate segmentation tasks from different centres [Wan+26a, Sec. 4.4]; we observe the same for a free-text promptable 3D model with tasks that are structures, but with two orders only and for one of the three strategies we trained in both orders.

For shared LoRA, the rank set a trade-off between adaptation and retention: from $r = 4$ to $r = 2048$, Final Forgetting fell from 0.546 to 0.284 and the control-task drop from 0.978 to 0.091, while Adaptation fell from 0.607 to 0.462 (Section 6.3). Because α was fixed, the rank and the scaling factor γ_r changed together, so we report this as a trade-off observed in this configuration, not as a rule for choosing the rank, and whether the

capacity or the scale of the adapter causes it remains open (Section 7.2). For task-specific LoRA, $r = 8$ brought no gain over $r = 4$, with one run per rank. Of the components of the strategies, replay lowered the Final Forgetting of shared LoRA from 0.289 to 0.046 at the same Adaptation and also lowered its control-task drop; the per-task gradient scaling of the prompt-tuning strategy lowered its Final Forgetting by 0.063 and its control-task drop by 0.117 at an Adaptation lower by 0.032; and the weight ensemble of the distillation-based strategy, in our configuration, halved the Adaptation without protecting the control task (Section 6.2).

In answer to the fourth objective, the task order, the rank and replay all shifted the balance between adaptation and retention of shared LoRA strongly, and gradient scaling shifted the prompt-tuning strategy towards retention, whereas task-specific LoRA did not depend on the order, by design, and did not gain from a higher rank, and sequential fine-tuning forgot almost everything in both orders.

The first objective was an evaluation protocol for this setting (Section 5.2). The protocol measures adaptation, forgetting within the task sequence and the loss of pretrained ability separately, with few-shot settings and two task orders, and two of its parts changed how the results read. The control task showed a loss that Final Forgetting does not capture: between $r = 1024$ and $r = 2048$, the Final Forgetting of shared LoRA differs by less than 0.01, whereas the control-task drop falls from 0.237 to 0.091 (Section 6.3). The reference row of the pretrained model showed that the LoRA strategies with 5 and 10 volumes per task did not change the model at all: their Adaptation of 0.114 to 0.116 equals that of the pretrained model, 0.114, and would otherwise read as weak learning (Section 6.5). The joint reference showed that the loss of the control task under full fine-tuning occurs without a task sequence, which the sequential runs alone could not show (Section 6.2). In answer to the first objective, the protocol separates these effects as intended, within the limits stated in Section 7.3: its control task is a single structure, its joint reference covers all six tasks together only, and its intervals cover only the choice of test volumes. It requires a pretrained free-text promptable model, a text prompt and labelled volumes for each task, and a held-out control task; whether it transfers to other models without changes was not tried (Section 7.4).

7.2 Training Budget as a Common Factor

Four results of Chapter 6 show the same pattern: under the shared configuration of ten epochs per task, a LoRA adapter often did not move the prediction away from that of the pretrained model. Shared LoRA learned the first task poorly in both task orders, whereas the same structures were learned when they came later in the other order (Section 6.6). Task-specific LoRA did not learn patellar cartilage in ten epochs with any rank, learning rate or seed we tried, but did in fifty (Section 6.4). With 5 and 10 volumes per task, that is, 50 and 100 updates, neither LoRA strategy learned any task (Section 6.5). In the rank experiment, the Adaptation that shared LoRA lost at higher ranks came mostly from the first task (Section 6.3).

We propose a hypothesis that connects these results: an adapter that starts from zero needs many updates before it changes the output of the model, and ten epochs were not always enough. The hypothesis rests on how the adapter is initialised. Because $\boldsymbol{B}$ starts at zero (Section 5.4.2), the update $\gamma_r \boldsymbol{B}\boldsymbol{A}$ is zero at the start, and the model predicts exactly what the pretrained model predicts. The gradient with respect to $\boldsymbol{A}$ contains $\boldsymbol{B}$ as a factor and is therefore also zero in the first step, so at first only $\boldsymbol{B}$ changes, and the update grows from zero only as both factors move away from their initial values. For a structure that the pretrained model does not segment, the update has to grow large enough to turn an empty prediction into a non-empty one, and with a learning rate that decays to zero within the ten epochs of a task (Section 5.3), training can end before this point is reached. In shared LoRA, every task after the first starts from an adapter that has already moved. In the reverse order, the training loss of the second task started at 0.86, although the first task had hardly been learned (Section 6.6). This suggests that later tasks benefit from the adapter having moved rather than from what it learned about the earlier structure, but our runs cannot separate the two. The equal-updates runs test one part of the hypothesis. With the update count of the full split on 5 volumes, both LoRA strategies learned the tasks and came within 0.06 of their Adaptation with the full split (Section 6.5), so their failure with 50 updates per task came from the number of updates, as the hypothesis predicts. They do not test the role of the zero initialisation, because sequential fine-tuning, which starts every task from trained weights, also adapted more with more updates, and task-specific LoRA still did not learn patellar cartilage with 1,080 updates, in line with the 50 epochs it needed in the diagnosis (Section 6.4). No run tests the zero initialisation directly, so we present it as consistent with the observations, not as a finding.

author: confirm this reading of the equal-updates runs for

Two observations do not follow from the hypothesis. First, it does not explain why the rank matters: with $r = 4$, the training loss of the first task fell to 0.74 within three epochs, whereas with $r = 1024$ it had fallen only to 0.69 after ten (Section 6.6). Because α was fixed, the scaling factor γ_r changes with the rank, and the rank experiment does not show whether the capacity or the scale of the adapter causes the slower start (Section 6.3). Second, task-specific LoRA, which starts every task from zero, learned most structures within ten epochs, for example lateral tibial cartilage with a DSC of 0.64 and medial meniscus with 0.57 right after training; only patellar cartilage stayed at zero (Section 6.3). How quickly an adapter leaves the empty prediction therefore also depends on the structure, which the hypothesis does not predict. Neither does it explain why shared LoRA retained more of the earlier tasks and of the control task in the reverse order (Section 6.6).

A result of Silva-Rodríguez et al. is in line with the hypothesis, without testing it. In their few-shot adaptation of a 3D CT segmentation model to 16 structures that were new to the model, LoRA with $r = 4$ in the encoder reached a mean DSC of 38.49 % with 10 labelled volumes [Sil+25, Table 4], while our LoRA strategies learned no task with 10 volumes. Their adaptation ran for up to 200 epochs with early stopping, with AdamW and a learning rate chosen for each adaptation method by a grid search [Sil+25, Sec. 4.3.2–4.3.3]; ours ran for 10 epochs, with stochastic gradient descent and one learning rate for all strategies. The model, the data, the classification head and the place of the adapter differ as well, so the comparison shows only that LoRA can learn from 10 volumes with a larger budget, not that the budget is what limits it in our runs.

For the research question, the hypothesis matters because our comparison measures the strategies under one training budget, not each at its best setting (Section 5.3). The patellar diagnosis shows one case in which this budget limits the Adaptation of task-specific LoRA, and the equal-updates runs show that the few-shot results of both LoRA strategies were limited in the same way at $k = 5$. More updates also increased the forgetting of the strategies that share parameters across tasks: at $k = 5$, shared LoRA forgot as much with 1,080 updates as with the full split, and sequential fine-tuning lost the control task, which it kept with 50 updates (Section 6.5). A longer budget would therefore probably raise adaptation and forgetting together, and how the trade-off looks under a longer budget for the full split remains open. The answers in Section 7.1 hold for the budget of ten epochs per task.

7.3 Limitations and Threats to Validity

Three limitations affect the conclusions most: the shared training configuration, the single run per configuration and the narrow scope of the study. We discuss them first, then the control task, and then the remaining threats to internal, construct and external validity.

First, all strategies were trained with one configuration, and neither the learning rate nor the number of epochs was tuned per strategy (Section 5.3). Section 7.2 argues that this budget limits at least task-specific LoRA, so the comparison describes the strategies under ten epochs per task, not at their best settings. Three experiments also change a second factor together with the one they study. The few-shot settings reduce the number of updates together with the number of volumes, the shared-LoRA rank experiment changes the scaling factor γ_r together with the rank, and the 50-epoch patellar run changes the learning-rate schedule together with the number of updates (Sections 6.3 to 6.5). In these experiments, a difference cannot be attributed to the studied factor alone; the equal-updates runs separate volumes and updates only for $k = 5$ and three strategies.

Second, each configuration was trained once, on one random split of each dataset (Section 4.3), and each few-shot setting uses one draw of training volumes (Section 6.1.2). Our bootstrap intervals cover only the choice of test volumes, not the variation between training runs or between draws of training volumes (Section 6.1.4). The two task-specific LoRA runs show how large the variation between runs can be: they differ only in the initialisation of the adapters and in non-deterministic GPU operations, yet they reached 0.40 and 0.34 on lateral meniscus, and the paired interval of their difference in Adaptation excludes zero (Section 6.6). A second example are the earlier run of sequential fine-tuning and its rerun with the same configuration: the rerun was continued inside the last task and therefore drew its training samples in a different order from that point on (Section 6.1.3), and at the same checkpoint, the two evaluation matrices agree up to the fifth task but differ by up to 0.037 in single entries after the sixth. Differences of a few hundredths between single runs, especially on single tasks, should therefore not be read as effects of a strategy or a factor. The trend over the rank and the effect of the task order on shared LoRA are many times larger than the difference in Adaptation between these two runs (0.006), but one pair of runs gives only one example of the variation, and the variation between draws of training volumes in the few-shot settings is unknown. The intervals are also not adjusted for the 152 comparisons we read (Section 6.1.4).

author: confirm using this pair as an example of training variation

Third, the scope is narrow. We adapted one model, VoxTell, to six structures of one knee MRI dataset, in two task orders, with one fixed text prompt per structure that was

used for both training and evaluation (Section 4.1). The distillation-based and the prompt-tuning strategies were run only with the full training split and in the original order, and the replay strategy only with the full split (Section 6.1.1), so the few-shot part of the research question is answered for sequential fine-tuning and the two LoRA strategies only, with joint fine-tuning as their reference. Whether the results depend on the wording of the text prompt was not tested. In SEER, VoxTell kept the highest DSC of five promptable models under rephrased requests, but its worst rephrasing still lowered the DSC on a pelvic-bone dataset from 97.59 % with the class-name prompt to 79.34 % [Zha+26, Table 1] (Section 2.1). Our evaluation uses only the wording seen in training, so it cannot show whether the adapted models still respond to other wordings of the same structure.

The control task measures the loss of pretrained ability with a single structure: the lung in four CT test volumes of AeroPath, which the pretrained model segments with a DSC of 0.979 (Section 4.2). With four test volumes, its DSC is less certain than that of the SKM-TEA tasks with 24, and the origin of its reference labels is not documented (Section 4.2). One large, high-contrast structure in another modality cannot show how the strategies affect the many other structures and modalities that VoxTell was trained on, and unlike the control set of SPU, the control task is not sampled from the pretraining distribution (Section 3.3). That replay lowered the control-task drop although its memory holds no control-task samples arose in the last task of one task order, so it may depend on the order, and its cause was not tested (Section 6.2).

Further threats to internal validity come from differences between the runs other than the strategy. All strategies are evaluated at the last epoch; only the additional evaluation selects a checkpoint, by a validation loss computed on one random patch per volume, and this selection lowered the Adaptation of sequential fine-tuning (Sections 5.3 and 6.2). The prompt-tuning run without gradient scaling was evaluated only at that checkpoint, so the scaling ablation compares both runs at the selected checkpoint. The two LoRA strategies were run with different values of α and of the dropout rate (Section 5.4.2), no strategy used data augmentation (Section 5.3), the code version was recorded only for the reruns, and the software versions were not logged per run (Section 6.1.3).

Two threats concern what the measures and the strategies represent. The joint reference was trained on all six tasks together only, so Adaptation cannot be turned into the intransigence measure of Chaudhry et al., whose reference for a task is a model trained jointly on the tasks up to it (Sections 3.3 and 5.2). The distillation-based and prompt-tuning strategies are adaptations: the first distils on current-task data and updates its weight

ensemble every 10 instead of every 100 iterations, and the second learns prompt tokens in the decoder together with a meta-net whose gradients it scales as well (Sections 5.4.4 and 5.4.5), so their results do not describe the original methods. Our distillation teacher is the pretrained model, as in ZSCL; PRIMED, in continual learning of a medical 2D classification model, reached a higher Last accuracy with the model of the previous task as teacher than with the initial model (82.1 against 78.2 %) [Che+26c, Table 4a], and we did not test this choice. For task-specific LoRA, Final Forgetting and the control-task drop are zero by construction (Section 5.2); its costs are the task identity it needs at inference and one stored adapter per task, which the metrics of the protocol do not measure.

Finally, the external validity is limited by the data and by the strategies that were not evaluated. We used our own random split of SKM-TEA instead of the official one, so our numbers are not comparable with SKM-TEA benchmarks. The SKM-TEA files store no orientation, so the volumes reach the model in their stored axis order, which may differ from the orientation of the knee volumes in VoxTell's training data, and 18 of the 155 scans are stored with the left–right axis reversed (Section 4.4). On our test volumes, the pretrained model segmented femoral cartilage with a DSC of 0.684 but none of the other five structures (Section 6.2), whereas the VoxTell authors report 46.45 % for femoral cartilage and at most 14.78 % for the others [Rok+26, Suppl. Table 9]. The test volumes, the orientation and possibly the checkpoint version differ between the two evaluations, and we did not test which of these causes the difference. The model also sees only the first qDESS echo, although the annotators used both echoes to separate neighbouring cartilage and meniscus voxels [Des+21, App. A.4]. Other strategies, such as elastic weight consolidation, mixture-of-experts adapters or the selection of adapters by the similarity of text-prompt embeddings as in MedCRP-CL [Gao26], were not evaluated (Section 5.4).

defense: pretrained DSC differs from VoxTell Suppl. Table 9; orientation (identity affine), test volumes and checkpoint version are candidate causes, none

7.4 Generalisation and Extensions

The numbers of Chapter 6 belong to VoxTell, to six knee structures and to the training configuration we used, and we do not expect them to carry over to other models or data. What may carry over is the explanation proposed in Section 7.2. It rests only on the standard initialisation of LoRA, in which the update starts at zero [Hu+22, Sec. 4.1], so other promptable segmentation models adapted with LoRA start from the same point; whether they also need more updates than a short schedule provides is a hypothesis that

our runs do not test. For other anatomical regions and for CT tasks, we have no evidence either way.

The results also say nothing about structures that the model has never seen. The three cartilages other than femoral cartilage are in VoxTell's training vocabulary, whereas the menisci are not (Section 4.1), but in our evaluation the pretrained model segmented only femoral cartilage, with a DSC of 0.684, and none of the other five structures (Section 6.3). Our tasks are therefore structures that the model rarely saw or does not segment in this data, not structures of a kind that is absent from its training data, such as a new modality.

For use in practice, the strategies differ in what they assume at inference. Task-specific LoRA needs the task identity to select an adapter (Section 5.2). In our setting, the text prompt names the structure, so the identity is available as long as each text prompt maps to one trained adapter. This holds for a closed set of structures with fixed text prompts, but not for open use: a rephrased text prompt or a new structure has no adapter of its own, so it needs either the pretrained model or a rule that chooses an adapter, and our protocol defines only the first, for the control task. Each task also adds an adapter, about 0.2 million parameters at $r = 4$ (Section 5.4.2), so the memory grows with the number of tasks. The strategies with one parameter set for all tasks need no such mapping, but in our runs all of them forgot part of the earlier tasks or of the control task, from shared LoRA with replay, which forgot least, to sequential fine-tuning and the distillation-based strategy, which forgot almost everything (Section 6.2).

Four extensions follow directly from the open points of this chapter. First, the hypothesis of Section 7.2 can be tested by training the first task of shared LoRA at $r = 1024$ for more epochs, by starting a task from an adapter trained on another structure instead of from zero, and by varying α or the learning rate at high rank, which would also separate the capacity from the scale of the adapter. Second, repeating each configuration with several seeds and, in the few-shot settings, with several draws of training volumes would give the variation between runs that our intervals do not contain, and extending the equal-updates runs to $k = 10$ and $k = 20$ and to the other strategies would separate data and updates where they remain confounded (Section 7.3). Third, evaluating the adapted models with rephrased text prompts, for instance the clinical styles used by SEER [Zha+26], would show whether adaptation with one wording per structure changes how the model responds to other wordings. Fourth, more task orders would show whether the large loss when lateral meniscus is trained last in the original order comes from the structure or from its posi-

author: confirm the added extension (equal-updates runs for k = 10, 20 and

tion in the sequence (Section 6.6). Beyond these, applying the protocol to other free-text promptable models would show which of our observations depend on VoxTell.

Chapter 8

Summary

A universal segmentation model is valuable because it segments many structures from a free-text prompt, but it still fails on structures that were rare in its training data, and labelled volumes of such structures are scarce. This thesis studied what happens when such a model is adapted to new structures one after another with few labelled volumes per structure: how much it learns of each new structure, how much it forgets of the earlier ones, and how much of its pretrained ability it keeps. We adapted VoxTell [Rok+26] to six structures of the SKM-TEA knee MRI dataset [Des+21], four cartilages and two menisci, which the pretrained model segmented with a mean DSC of 0.114, and used the lung in chest CT volumes of AeroPath [Stø+23] as a control task that is never trained on.

The strategies we compared are based on existing methods: sequential fine-tuning, LoRA [Hu+22] with one adapter per task or one shared adapter, replay, ZSCL [Zhe+23] and CPE-CLIP [DAl+23]. Our own work is threefold. First, we designed an evaluation protocol for this setting, which evaluates the model on every task and on a held-out control task after each task, reports the pretrained model and joint fine-tuning as references, and covers few-shot settings with 5, 10 and 20 labelled volumes per task and two task orders (Section 5.2). Second, we adapted the strategies to a free-text promptable 3D model, with LoRA on the cross-attention of the prompt decoder, distillation of a voxel-wise similarity distribution at the coarsest decoder stage instead of ZSCL's image–text distribution, and learned prompt tokens in the cross-attention together with a meta-net that shifts the text query instead of CPE-CLIP's prompts in the encoders (Section 5.4). We compared them under one shared training configuration in 38 runs, with joint fine-tuning as an upper reference (Chapter 6). Third, we analysed the factors that affect adaptation and forgetting in this setting, separating the amount of data from the number of updates, and diagnosed

the failure of task-specific LoRA on patellar cartilage, from which we proposed a hypothesis that connects several results (Section 7.2). All numbers below come from these runs.

The most important result is that no strategy combined high adaptation with retention (Section 6.2). Sequential fine-tuning and the distillation-based strategy reached an Adaptation of 0.781 and 0.747, but after six tasks their Final Performance of 0.130 and 0.135 was barely above that of the pretrained model, and the DSC on the control task fell from 0.979 to almost zero. Joint fine-tuning with the same number of updates on all six tasks at once reached a Final Performance of 0.852, at least 0.36 above every strategy, but its DSC on the control task also fell to almost zero, so this loss comes from fine-tuning all parameters on the knee structures, not from the task sequence. The prompt-tuning strategy, whose pretrained weights stay frozen, lay between the groups and still lost most of the control task, so freezing the backbone did not by itself preserve the other abilities of the model. The two LoRA strategies adapted less, with an Adaptation of 0.485 and 0.471, and retained much more; task-specific LoRA, which forgets nothing by construction but needs the task identity at inference, reached the highest Final Performance, 0.485, and shared LoRA with replay came within 0.034 of it without the task identity. The second result concerns few labelled volumes (Section 6.5). With ten epochs per task and 5 or 10 volumes, neither LoRA strategy learned any task, whereas sequential fine-tuning learned every task and, with 5 volumes, forgot less and kept the control task. With the update count of the full split on 5 volumes, the LoRA strategies learned as well, and reducing the data from 108 to 5 volumes at the same number of updates lowered the Adaptation by only 0.03 to 0.07, so at this setting the number of updates decided the outcome more than the amount of data.

The factors we varied changed the balance mainly for shared LoRA (Sections 6.2, 6.3 and 6.6). In the reverse task order, its Final Forgetting fell from 0.289 to 0.124 and its control-task drop from 0.237 to 0.067, whereas no effect of the order could be detected for sequential fine-tuning and none beyond run-to-run variation for task-specific LoRA. A higher rank lowered the forgetting of shared LoRA at the cost of Adaptation, from a Final Forgetting of 0.546 and an Adaptation of 0.607 at $r = 4$ to 0.284 and 0.462 at $r = 2048$, with the scaling factor changing together with the rank. Replay, with a memory of earlier tasks only, lowered its Final Forgetting from 0.289 to 0.046 and its control-task drop from 0.237 to 0.138 and raised its Final Performance from 0.233 to 0.450 at the same Adaptation. Per-task gradient scaling moved the prompt-tuning strategy towards retention, and the weight ensemble of the distillation-based strategy halved its Adaptation in our configuration.

The protocol separated effects that a single metric would have mixed (Section 7.1). The control task showed a loss of pretrained ability that Final Forgetting did not capture: between $r = 1024$ and $r = 2048$, the Final Forgetting of shared LoRA differed by less than 0.01, whereas its control-task drop fell from 0.237 to 0.091. The pretrained-model reference showed that the few-shot LoRA runs had not changed the model at all, which their Adaptation alone would have shown as weak learning, and the joint reference showed that the loss of the control task does not require a task sequence.

Several results point to the training budget as a common factor. Task-specific LoRA stayed at an empty prediction on patellar cartilage for ten epochs with every rank, learning rate and seed we tried, and reached a DSC of 0.839 with fifty (Section 6.4); shared LoRA learned its first task poorly in both orders; and the LoRA strategies learned nothing with 50 to 100 updates per task but learned from the same 5 volumes with 1,080 (Section 6.5). We proposed that an adapter that starts from zero needs many updates before it changes the output of the model; the runs with equal updates support the role of the number of updates but do not test the initialisation (Section 7.2). The conclusions are further limited by one run per configuration, one training configuration for all strategies, one model, one dataset and one text prompt per structure, a control task of a single structure, and a separation of data and updates at $k = 5$ only (Section 7.3).

Within this setting, the results suggest three points for adapting such a model. A parameter-efficient strategy needs enough updates per task. For some structures, the updates of ten epochs were not enough, and the few-shot settings provided considerably fewer. More updates, however, also increased the forgetting of the strategies that share parameters across tasks. Where the other abilities of the model matter, sequential fine-tuning, the distillation-based strategy and the prompt-tuning strategy lost them in our runs, so a frozen backbone alone is no safeguard. For a closed set of structures with fixed text prompts, task-specific LoRA keeps every earlier structure by construction; without the task identity, replay brought shared LoRA close to it in our single run and is worth testing further, but one run does not support a recommendation.

Future work follows from these limits (Section 7.4). The training-budget hypothesis can be tested by training the first task of shared LoRA for more epochs, by starting a task from an adapter trained on another structure, and by varying the scaling factor or the learning rate at high rank. Repeated runs with several seeds and several draws of training volumes would give the variation between runs that our intervals do not cover, and runs with equal updates for 10 and 20 volumes and for the other strategies would separate data

author: check whether the re-run of replay changes this recommendation

author: confirm, as in Section 7.4

and updates where they remain confounded. Rephrased text prompts would show whether adaptation with one wording changes how the model responds to others, more task orders would show whether the loss at the end of the original order comes from the structure or from its position, and applying the protocol to other free-text promptable models would show which observations depend on VoxTell.

Appendix A

Additional Experimental Details and Results

This appendix complements Chapter 6. Section A.1 lists the volumes of the few-shot training sets and of the test sets (Sections 4.3 and 6.1.2), and Section A.2 gives the evaluation matrix of every run reported in Chapter 6, from which all metrics and all per-task values quoted there are computed.

A.1 Training and Test Volumes

This section belongs to Sections 4.3 and 6.1.2. Table A.1 lists the SKM-TEA volumes of the few-shot training sets and the test volumes of both datasets. The few-shot sets are taken from the training logs of the runs: every run with the same number of volumes k used the same set for all six tasks, and the sets are nested. The test volumes are the same in every run.

A.2 Evaluation Matrices of All Runs

This section belongs to Chapter 6. For every reported run, the table gives the mean DSC on the test volumes of each task (rows, in training order) after each stage (columns): *Pre.* is the pretrained model, and the following columns are the stages after training on the task named in the header. The diagonal entries, right after a task is trained, are set in bold; the last row is the control task. Task abbreviations: LT lateral tibial cartilage, MT medial tibial cartilage, Fe femoral cartilage, Pa patellar cartilage, MM medial meniscus, LM lateral

Table A.1: Few-shot training volumes and test volumes. SKM-TEA volumes by their identifier, AeroPath volumes by their number. The full training split contains 108 SKM-TEA volumes (Section 4.3).

Set	Volumes
SKM-TEA training, $k = 5$	MTR_038, MTR_074, MTR_097, MTR_201, MTR_225
SKM-TEA training, $k = 10$	the 5 volumes above and MTR_118, MTR_154, MTR_188, MTR_207, MTR_219
SKM-TEA training, $k = 20$	the 10 volumes above and MTR_001, MTR_037, MTR_056, MTR_120, MTR_156, MTR_167, MTR_179, MTR_209, MTR_224, MTR_243
SKM-TEA test (24 volumes)	MTR_005, MTR_015, MTR_016, MTR_018, MTR_045, MTR_046, MTR_049, MTR_052, MTR_066, MTR_079, MTR_098, MTR_104, MTR_105, MTR_107, MTR_110, MTR_115, MTR_129, MTR_130, MTR_190, MTR_191, MTR_199, MTR_215, MTR_227, MTR_248
AeroPath test (4 volumes)	1, 3, 6, 12

meniscus, Ctrl control task (lung in AeroPath). The runs are grouped by the experiment in which they first appear in Table 6.1; a run that belongs to several experiments is listed once. Joint fine-tuning has a single stage, *Joint*, after which all six structures are evaluated. All matrices are computed with the parameters after the last epoch, except that of the prompt-tuning run without gradient scaling, which was evaluated only at the checkpoint with the lowest validation loss (Section 6.2).

Comparison of the strategies

Table A.2: Evaluation matrix: Sequential fine-tuning (full training split, original task order); run `exp_20261022_171452_a9f4ca56`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.856**	0.000	0.002	0.000	0.000	0.014
MT	0.000	0.000	**0.843**	0.002	0.000	0.010	0.000
Fe	0.684	0.001	0.001	**0.841**	0.192	0.006	0.004
Pa	0.000	0.000	0.000	0.003	**0.632**	0.006	0.019
MM	0.000	0.000	0.007	0.001	0.000	**0.775**	0.002
LM	0.000	0.005	0.000	0.001	0.000	0.001	**0.740**
Ctrl	0.979	0.001	0.000	0.000	0.000	0.000	0.000

Table A.3: Evaluation matrix: Task-specific LoRA ($r = 4$, full training split, original task order); run `exp_20260816_155157_668e608d`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.635**	0.635	0.635	0.635	0.635	0.635
MT	0.000	0.163	**0.527**	0.527	0.527	0.527	0.527
Fe	0.684	0.440	0.470	**0.774**	0.774	0.774	0.774
Pa	0.000	0.002	0.002	0.000	**0.000**	0.000	0.000
MM	0.000	0.012	0.042	0.000	0.000	**0.572**	0.572
LM	0.000	0.033	0.006	0.000	0.000	0.020	**0.401**
Ctrl	0.979	0.979	0.979	0.979	0.979	0.979	0.979

Table A.4: Evaluation matrix: Shared LoRA ($r = 1024$, full training split, original task order); run `exp_20260926_095539_90e8b4e6`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.294**	0.311	0.174	0.156	0.216	0.025
MT	0.000	0.185	**0.490**	0.326	0.165	0.355	0.018
Fe	0.684	0.648	0.550	**0.719**	0.609	0.644	0.555
Pa	0.000	0.024	0.004	0.008	**0.443**	0.291	0.039
MM	0.000	0.037	0.061	0.030	0.066	**0.477**	0.357
LM	0.000	0.020	0.026	0.008	0.011	0.071	**0.402**
Ctrl	0.979	0.980	0.975	0.976	0.974	0.955	0.742

Table A.5: Evaluation matrix: Prompt-tuning strategy (full training split, original task order); run `exp_20261022_180923_7bcbb6cc`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.711**	0.407	0.165	0.162	0.286	0.043
MT	0.000	0.095	**0.528**	0.160	0.104	0.137	0.005
Fe	0.684	0.065	0.043	**0.730**	0.492	0.235	0.010
Pa	0.000	0.003	0.000	0.012	**0.472**	0.203	0.010
MM	0.000	0.003	0.016	0.014	0.018	**0.503**	0.327
LM	0.000	0.010	0.002	0.006	0.004	0.212	**0.534**
Ctrl	0.979	0.041	0.071	0.226	0.078	0.155	0.115

Table A.6: Evaluation matrix: Distillation-based strategy (full training split, original task order); run `exp_20261022_171452_65011743`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.841**	0.000	0.001	0.005	0.000	0.007
MT	0.000	0.000	**0.831**	0.003	0.003	0.013	0.000
Fe	0.684	0.001	0.001	**0.823**	0.255	0.005	0.002
Pa	0.000	0.000	0.000	0.008	**0.472**	0.000	0.000
MM	0.000	0.000	0.011	0.001	0.000	**0.713**	0.000
LM	0.000	0.006	0.000	0.001	0.000	0.000	**0.801**
Ctrl	0.979	0.001	0.000	0.003	0.002	0.000	0.000

LoRA rank

Table A.7: Evaluation matrix: Task-specific LoRA ($r = 8$, full training split, original task order); run `exp_20260822_110320_530ca92a`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.515**	0.515	0.515	0.515	0.515	0.515
MT	0.000	0.273	**0.516**	0.516	0.516	0.516	0.516
Fe	0.684	0.601	0.488	**0.748**	0.748	0.748	0.748
Pa	0.000	0.006	0.002	0.000	**0.000**	0.000	0.000
MM	0.000	0.021	0.077	0.000	0.000	**0.526**	0.526
LM	0.000	0.040	0.008	0.000	0.000	0.033	**0.118**
Ctrl	0.979	0.979	0.979	0.979	0.979	0.979	0.979

Table A.8: Evaluation matrix: Shared LoRA ($r = 4$, full training split, original task order); run `exp_20260924_202757_88afb75d`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.693**	0.380	0.022	0.051	0.022	0.005
MT	0.000	0.104	**0.520**	0.065	0.004	0.053	0.001
Fe	0.684	0.053	0.096	**0.783**	0.547	0.069	0.004
Pa	0.000	0.000	0.000	0.031	**0.510**	0.005	0.000
MM	0.000	0.002	0.011	0.039	0.012	**0.581**	0.348
LM	0.000	0.011	0.001	0.021	0.001	0.062	**0.554**
Ctrl	0.979	0.004	0.001	0.330	0.014	0.226	0.001

Table A.9: Evaluation matrix: Shared LoRA ($r = 32$, full training split, original task order); run `exp_20260926_095635_1dd1adb4`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.650**	0.427	0.140	0.100	0.195	0.010
MT	0.000	0.175	**0.515**	0.158	0.038	0.201	0.004
Fe	0.684	0.458	0.183	**0.751**	0.504	0.439	0.018
Pa	0.000	0.003	0.000	0.014	**0.492**	0.179	0.016
MM	0.000	0.024	0.025	0.082	0.029	**0.517**	0.381
LM	0.000	0.031	0.002	0.012	0.003	0.103	**0.493**
Ctrl	0.979	0.708	0.193	0.880	0.106	0.080	0.001

Table A.10: Evaluation matrix: Shared LoRA ($r = 128$, full training split, original task order); run `exp_20260816_124851_bc388139`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.453**	0.316	0.102	0.111	0.242	0.015
MT	0.000	0.241	**0.531**	0.164	0.058	0.289	0.010
Fe	0.684	0.633	0.417	**0.732**	0.630	0.639	0.358
Pa	0.000	0.014	0.001	0.008	**0.484**	0.230	0.024
MM	0.000	0.034	0.044	0.071	0.034	**0.490**	0.363
LM	0.000	0.029	0.009	0.002	0.004	0.101	**0.455**
Ctrl	0.979	0.969	0.791	0.937	0.672	0.434	0.022

Table A.11: Evaluation matrix: Shared LoRA ($r = 256$, full training split, original task order); run `exp_20260817_172554_da49178a`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.361**	0.309	0.142	0.121	0.247	0.014
MT	0.000	0.223	**0.522**	0.223	0.085	0.343	0.012
Fe	0.684	0.646	0.468	**0.726**	0.634	0.664	0.478
Pa	0.000	0.023	0.002	0.008	**0.478**	0.243	0.014
MM	0.000	0.042	0.056	0.053	0.045	**0.489**	0.373
LM	0.000	0.026	0.014	0.004	0.005	0.089	**0.443**
Ctrl	0.979	0.979	0.945	0.968	0.918	0.772	0.148

Table A.12: Evaluation matrix: Shared LoRA ($r = 512$, full training split, original task order); run `exp_20260924_163047_f4aa9c98`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.310**	0.290	0.137	0.143	0.192	0.020
MT	0.000	0.201	**0.511**	0.264	0.145	0.321	0.014
Fe	0.684	0.647	0.516	**0.721**	0.619	0.640	0.537
Pa	0.000	0.029	0.003	0.010	**0.461**	0.251	0.020
MM	0.000	0.042	0.058	0.038	0.061	**0.494**	0.361
LM	0.000	0.022	0.023	0.006	0.010	0.074	**0.427**
Ctrl	0.979	0.980	0.965	0.973	0.965	0.874	0.405

Table A.13: Evaluation matrix: Shared LoRA ($r = 2048$, full training split, original task order); run `exp_20260927_213503_efa7a48c`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.280**	0.326	0.208	0.185	0.230	0.038
MT	0.000	0.180	**0.473**	0.342	0.208	0.346	0.023
Fe	0.684	0.648	0.571	**0.720**	0.630	0.648	0.550
Pa	0.000	0.026	0.003	0.005	**0.447**	0.326	0.047
MM	0.000	0.040	0.047	0.023	0.081	**0.464**	0.352
LM	0.000	0.019	0.025	0.008	0.014	0.068	**0.388**
Ctrl	0.979	0.980	0.977	0.978	0.978	0.968	0.889

Diagnosis of the patellar failure

Table A.14: Evaluation matrix: Task-specific LoRA ($r = 4$, patellar cartilage only, settings of the sequence); run `exp_20260824_040750_012f67c8`.

Task	Pre.	Pa
Pa	0.000	**0.000**
Ctrl	0.979	0.979

Table A.15: Evaluation matrix: Task-specific LoRA ($r = 16$, patellar cartilage only); run `exp_20260828_041835_89e6de63`.

Task	Pre.	Pa
Pa	0.000	**0.000**
Ctrl	0.979	0.979

Table A.16: Evaluation matrix: Task-specific LoRA ($r = 4$, patellar cartilage only, learning rate $3 \cdot 10^{-4}$); run `exp_20260828_045913_2e306fe6`.

Task	Pre.	Pa
Pa	0.000	**0.000**
Ctrl	0.979	0.979

Table A.17: Evaluation matrix: Task-specific LoRA ($r = 4$, patellar cartilage only, seed 1337); run `exp_20260901_110801_d8b133c2`.

Task	Pre.	Pa
Pa	0.000	**0.000**
Ctrl	0.979	0.979

Table A.18: Evaluation matrix: Task-specific LoRA ($r = 4$, patellar cartilage only, 50 epochs); run `exp_20260905_220334_6697f1c2`.

Task	Pre.	Pa
Pa	0.000	**0.839**
Ctrl	0.979	0.979

Amount of labelled data

Table A.19: Evaluation matrix: Sequential fine-tuning (5 volumes per task, original task order); run `exp_20261023_212327_a67fb8c9`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.620**	0.513	0.197	0.371	0.129	0.013
MT	0.000	0.000	**0.487**	0.163	0.127	0.185	0.007
Fe	0.684	0.465	0.292	**0.713**	0.366	0.472	0.421
Pa	0.000	0.000	0.000	0.000	**0.338**	0.023	0.036
MM	0.000	0.000	0.013	0.022	0.003	**0.593**	0.362
LM	0.000	0.075	0.015	0.133	0.005	0.118	**0.521**
Ctrl	0.979	0.964	0.961	0.967	0.974	0.970	0.969

Table A.20: Evaluation matrix: Sequential fine-tuning (10 volumes per task, original task order); run `exp_20261023_234659_f041f82f`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.777**	0.405	0.004	0.066	0.036	0.033
MT	0.000	0.002	**0.534**	0.010	0.013	0.017	0.003
Fe	0.684	0.111	0.091	**0.743**	0.241	0.014	0.009
Pa	0.000	0.000	0.000	0.008	**0.361**	0.012	0.003
MM	0.000	0.004	0.025	0.014	0.001	**0.596**	0.351
LM	0.000	0.018	0.001	0.022	0.001	0.243	**0.683**
Ctrl	0.979	0.908	0.772	0.691	0.655	0.435	0.674

Table A.21: Evaluation matrix: Sequential fine-tuning (20 volumes per task, original task order); run `exp_20261024_001927_861cfa24`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.675**	0.190	0.002	0.025	0.002	0.008
MT	0.000	0.006	**0.689**	0.003	0.001	0.003	0.000
Fe	0.684	0.032	0.011	**0.793**	0.233	0.005	0.004
Pa	0.000	0.000	0.000	0.007	**0.410**	0.001	0.006
MM	0.000	0.000	0.017	0.001	0.000	**0.496**	0.180
LM	0.000	0.002	0.001	0.001	0.000	0.367	**0.632**
Ctrl	0.979	0.587	0.005	0.033	0.000	0.000	0.000

Table A.22: Evaluation matrix: Task-specific LoRA ($r = 4$, 5 volumes per task, original task order); run `exp_20260926_120018_3ab58c90`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.000**	0.000	0.000	0.000	0.000	0.000
MT	0.000	0.000	**0.000**	0.000	0.000	0.000	0.000
Fe	0.684	0.685	0.685	**0.685**	0.685	0.685	0.685
Pa	0.000	0.000	0.000	0.000	**0.000**	0.000	0.000
MM	0.000	0.000	0.000	0.000	0.000	**0.000**	0.000
LM	0.000	0.000	0.000	0.000	0.000	0.000	**0.000**
Ctrl	0.979	0.979	0.979	0.979	0.979	0.979	0.979

Table A.23: Evaluation matrix: Task-specific LoRA ($r = 4$, 10 volumes per task, original task order); run `exp_20260926_184314_43a15eeb`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.000**	0.000	0.000	0.000	0.000	0.000
MT	0.000	0.000	**0.000**	0.000	0.000	0.000	0.000
Fe	0.684	0.686	0.685	**0.688**	0.688	0.688	0.688
Pa	0.000	0.000	0.000	0.000	**0.000**	0.000	0.000
MM	0.000	0.000	0.000	0.000	0.000	**0.000**	0.000
LM	0.000	0.000	0.000	0.000	0.000	0.000	**0.000**
Ctrl	0.979	0.979	0.979	0.979	0.979	0.979	0.979

Table A.24: Evaluation matrix: Task-specific LoRA ($r = 4$, 20 volumes per task, original task order); run `exp_20260926_105629_ba181db6`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.000**	0.000	0.000	0.000	0.000	0.000
MT	0.000	0.000	**0.000**	0.000	0.000	0.000	0.000
Fe	0.684	0.689	0.688	**0.693**	0.693	0.693	0.693
Pa	0.000	0.000	0.000	0.000	**0.000**	0.000	0.000
MM	0.000	0.000	0.000	0.000	0.000	**0.000**	0.000
LM	0.000	0.000	0.000	0.000	0.000	0.000	**0.000**
Ctrl	0.979	0.979	0.979	0.979	0.979	0.979	0.979

Table A.25: Evaluation matrix: Shared LoRA ($r = 1024$, 5 volumes per task, original task order); run `exp_20260930_164451_55aada20`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.000**	0.000	0.000	0.000	0.000	0.000
MT	0.000	0.000	**0.000**	0.000	0.000	0.000	0.000
Fe	0.684	0.686	0.688	**0.690**	0.690	0.691	0.692
Pa	0.000	0.000	0.000	0.000	**0.000**	0.000	0.000
MM	0.000	0.000	0.000	0.000	0.000	**0.000**	0.000
LM	0.000	0.000	0.000	0.000	0.000	0.000	**0.000**
Ctrl	0.979	0.980	0.980	0.980	0.980	0.980	0.980

Table A.26: Evaluation matrix: Shared LoRA ($r = 1024$, 10 volumes per task, original task order); run `exp_20260930_165220_b0c6c426`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.000**	0.001	0.001	0.001	0.001	0.002
MT	0.000	0.000	**0.000**	0.000	0.000	0.001	0.002
Fe	0.684	0.690	0.692	**0.698**	0.698	0.699	0.699
Pa	0.000	0.000	0.000	0.000	**0.000**	0.000	0.000
MM	0.000	0.000	0.000	0.000	0.000	**0.000**	0.001
LM	0.000	0.000	0.000	0.000	0.000	0.000	**0.000**
Ctrl	0.979	0.980	0.980	0.980	0.980	0.981	0.981

Table A.27: Evaluation matrix: Shared LoRA ($r = 1024$, 20 volumes per task, original task order); run `exp_20260930_163605_91573278`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.002**	0.005	0.005	0.012	0.020	0.028
MT	0.000	0.000	**0.016**	0.016	0.039	0.092	0.117
Fe	0.684	0.695	0.699	**0.705**	0.701	0.692	0.683
Pa	0.000	0.000	0.002	0.002	**0.067**	0.081	0.087
MM	0.000	0.000	0.001	0.001	0.003	**0.047**	0.073
LM	0.000	0.000	0.000	0.000	0.000	0.001	**0.003**
Ctrl	0.979	0.980	0.980	0.981	0.980	0.978	0.976

Task order

Table A.28: Evaluation matrix: Sequential fine-tuning (full training split, reverse task order); run `exp_20261022_171520_2dba6ff1`.

Task	Pre.	LM	MM	Pa	Fe	MT	LT
LM	0.000	**0.830**	0.000	0.000	0.001	0.000	0.004
MM	0.000	0.000	**0.826**	0.000	0.001	0.011	0.000
Pa	0.000	0.000	0.000	**0.650**	0.001	0.000	0.000
Fe	0.684	0.001	0.002	0.190	**0.822**	0.001	0.004
MT	0.000	0.000	0.006	0.000	0.002	**0.780**	0.001
LT	0.000	0.006	0.000	0.000	0.002	0.002	**0.796**
Ctrl	0.979	0.000	0.000	0.000	0.001	0.000	0.000

Table A.29: Evaluation matrix: Task-specific LoRA ($r = 4$, full training split, reverse task order); run `exp_20260822_123645_5b18bc03`.

Task	Pre.	LM	MM	Pa	Fe	MT	LT
LM	0.000	**0.343**	0.343	0.343	0.343	0.343	0.343
MM	0.000	0.185	**0.572**	0.572	0.572	0.572	0.572
Pa	0.000	0.008	0.002	**0.000**	0.000	0.000	0.000
Fe	0.684	0.695	0.539	0.000	**0.774**	0.774	0.774
MT	0.000	0.068	0.080	0.000	0.000	**0.527**	0.527
LT	0.000	0.041	0.001	0.000	0.000	0.078	**0.654**
Ctrl	0.979	0.979	0.979	0.979	0.979	0.979	0.979

Table A.30: Evaluation matrix: Shared LoRA ($r = 1024$, full training split, reverse task order); run `exp_20260930_101141_5d80eb69`.

Task	Pre.	LM	MM	Pa	Fe	MT	LT
LM	0.000	**0.014**	0.073	0.072	0.065	0.073	0.127
MM	0.000	0.038	**0.469**	0.303	0.330	0.411	0.328
Pa	0.000	0.001	0.008	**0.440**	0.406	0.212	0.176
Fe	0.684	0.693	0.670	0.609	**0.722**	0.646	0.501
MT	0.000	0.018	0.188	0.106	0.061	**0.477**	0.427
LT	0.000	0.017	0.012	0.090	0.035	0.118	**0.510**
Ctrl	0.979	0.972	0.978	0.975	0.977	0.967	0.912

Replay

Table A.31: Evaluation matrix: Shared LoRA ($r = 1024$, full training split, original task order, replay $\rho = 0.1$); run `exp_20261023_014438_55e6abcf`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.287**	0.386	0.383	0.305	0.347	0.385
MT	0.000	0.183	**0.473**	0.405	0.261	0.421	0.379
Fe	0.684	0.647	0.539	**0.713**	0.628	0.678	0.699
Pa	0.000	0.023	0.004	0.006	**0.435**	0.425	0.412
MM	0.000	0.041	0.062	0.031	0.078	**0.497**	0.400
LM	0.000	0.020	0.030	0.020	0.021	0.075	**0.427**
Ctrl	0.979	0.980	0.970	0.975	0.968	0.931	0.842

Gradient scaling of the prompt-tuning strategy

Table A.32: Evaluation matrix: Prompt-tuning strategy (full training split, original task order, gradient scaling inactive, best validation checkpoint); run `exp_20260816_161201_b39ea60e`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.707**	0.364	0.109	0.055	0.164	0.017
MT	0.000	0.102	**0.549**	0.119	0.025	0.090	0.001
Fe	0.684	0.088	0.033	**0.777**	0.408	0.006	0.001
Pa	0.000	0.002	0.000	0.015	**0.434**	0.033	0.002
MM	0.000	0.008	0.019	0.012	0.007	**0.555**	0.162
LM	0.000	0.009	0.002	0.005	0.001	0.166	**0.629**
Ctrl	0.979	0.108	0.019	0.060	0.001	0.002	0.000

Amount of labelled data with the update count of the full split

Table A.33: Evaluation matrix: Sequential fine-tuning (5 volumes per task, original task order, 216 epochs per task); run `exp_20261023_195739_6044ad8d`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.761**	0.014	0.001	0.000	0.000	0.004
MT	0.000	0.083	**0.673**	0.001	0.000	0.007	0.000
Fe	0.684	0.000	0.006	**0.797**	0.079	0.002	0.002
Pa	0.000	0.000	0.000	0.002	**0.756**	0.000	0.000
MM	0.000	0.000	0.007	0.002	0.000	**0.676**	0.086
LM	0.000	0.003	0.000	0.001	0.000	0.122	**0.610**
Ctrl	0.979	0.005	0.001	0.003	0.000	0.013	0.000

Table A.34: Evaluation matrix: Task-specific LoRA ($r = 4$, 5 volumes per task, original task order, 216 epochs per task); run `exp_20261023_195940_ea9221f2`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.601**	0.601	0.601	0.601	0.601	0.601
MT	0.000	0.121	**0.528**	0.528	0.528	0.528	0.528
Fe	0.684	0.089	0.230	**0.751**	0.751	0.751	0.751
Pa	0.000	0.002	0.002	0.000	**0.000**	0.000	0.000
MM	0.000	0.011	0.033	0.001	0.000	**0.466**	0.466
LM	0.000	0.001	0.000	0.000	0.000	0.017	**0.229**
Ctrl	0.979	0.979	0.979	0.979	0.979	0.979	0.979

Table A.35: Evaluation matrix: Shared LoRA ($r = 1024$, 5 volumes per task, original task order, 216 epochs per task); run `exp_20261023_200332_9840ab20`.

Task	Pre.	LT	MT	Fe	Pa	MM	LM
LT	0.000	**0.245**	0.172	0.060	0.130	0.036	0.010
MT	0.000	0.171	**0.428**	0.260	0.151	0.089	0.008
Fe	0.684	0.653	0.595	**0.721**	0.635	0.656	0.384
Pa	0.000	0.038	0.007	0.006	**0.453**	0.310	0.031
MM	0.000	0.035	0.055	0.020	0.087	**0.438**	0.374
LM	0.000	0.016	0.018	0.002	0.011	0.035	**0.346**
Ctrl	0.979	0.979	0.971	0.977	0.978	0.951	0.645

Upper reference: joint fine-tuning

Table A.36: Evaluation matrix: Joint fine-tuning (5 volumes per task); run `exp_20261024_002524_abc243ff`.

Task	Pre.	Joint
LT	0.000	0.631
MT	0.000	0.546
Fe	0.684	0.698
Pa	0.000	0.461
MM	0.000	0.600
LM	0.000	0.661
Ctrl	0.979	0.246

Table A.37: Evaluation matrix: Joint fine-tuning (10 volumes per task); run `exp_20261024_014447_afb34098`.

Task	Pre.	Joint
LT	0.000	0.550
MT	0.000	0.470
Fe	0.684	0.775
Pa	0.000	0.498
MM	0.000	0.582
LM	0.000	0.637
Ctrl	0.979	0.086

Table A.38: Evaluation matrix: Joint fine-tuning (20 volumes per task); run `exp_20261024_051245_f9067927`.

Task	Pre.	Joint
LT	0.000	0.810
MT	0.000	0.771
Fe	0.684	0.782
Pa	0.000	0.601
MM	0.000	0.779
LM	0.000	0.773
Ctrl	0.979	0.000

Table A.39: Evaluation matrix: Joint fine-tuning (full training split); run `exp_20261022_171655_6a6e9cb5`.

Task	Pre.	Joint
LT	0.000	0.872
MT	0.000	0.847
Fe	0.684	0.873
Pa	0.000	0.834
MM	0.000	0.842
LM	0.000	0.845
Ctrl	0.979	0.000

List of Abbreviations

CT computed tomography

CTA computed tomography angiography

MRI magnetic resonance imaging

PET positron emission tomography

qDESS quantitative double-echo steady-state

DICOM Digital Imaging and Communications in Medicine

HDF5 Hierarchical Data Format version 5

NIfTI Neuroimaging Informatics Technology Initiative

IQR interquartile range

DSC Dice similarity coefficient

SAM Segment Anything Model

IoU intersection over union

LoRA low-rank adaptation

EWC elastic weight consolidation

MLP multilayer perceptron

List of Figures

1.1 Setting of this thesis 3

3.1 Architecture of VoxTell 28
3.2 Entries of the evaluation matrix used by each metric 33

4.1 Example volumes with their labels 39

5.1 VoxTell as used in this thesis and the parts changed by the strategies . . . 58

6.1 Metrics of the strategies 64
6.2 Evaluation matrices of the strategies 65
6.3 Control task over the task sequence 66
6.4 Metrics of shared LoRA over the rank 69
6.5 Training loss of the patellar diagnosis runs 71
6.6 Metrics over the number of training volumes 72
6.7 Metrics in the original and the reverse task order 74

List of Tables

2.1 Setting of this thesis compared with the closest works 19

2.2 Works reviewed in this chapter, grouped by section. Works cited only for a definition, a method or a single finding are not listed. Results are given as reported by the authors, in the paper's metric, with the main comparison in parentheses; settings are named in the paper's own terms. FT: fine-tuning; seq.: sequential; TIL, DIL, CIL: task-, domain- and class-incremental learning (Section 3.2); MTIL: the eleven-dataset benchmark of ZSCL; FSCIL: few-shot class-incremental learning. 21

4.1 Datasets used in this thesis 38

5.1 Overview of the continual-learning strategies 51

5.2 Computational cost of the strategies 57

6.1 Overview of the experiments 78

6.2 Training and evaluation settings shared by all runs 79

6.3 Parameters of the strategies 80

6.4 Results of the comparison of the strategies 81

6.5 Additional evaluations and component ablations 81

6.6 Results of the LoRA rank experiment 82

6.7 Diagnosis of the patellar failure 82

6.8 Results of the few-shot experiment 83

6.9 Results of the task-order experiment 83

6.10 Measured cost of the strategies 84

A.1 Few-shot training volumes and test volumes 102

Bibliography

[Baw+18] C. Baweja, B. Glocker, and K. Kamnitsas. Towards continual learning in medical imaging, 2018. arXiv: 1811.02496 [cs.CV]. URL: https://arxiv.org/abs/1811.02496 (cited on p. 15).

[Ber+23] S. Bera, V. Ummadi, D. Sen, S. Mandal, and P. K. Biswas. Memory replay for continual medical image segmentation through atypical sample selection. In H. Greenspan, A. Madabhushi, P. Mousavi, S. Salcudean, J. Duncan, T. Syeda-Mahmood, and R. Taylor, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023. 26th International Conference, Vancouver, BC, Canada, October 8–12, 2023, Proceedings, Part IV* (October 8–12, 2023), pages 513–522, Cham. Springer Nature Switzerland, 2023. ISBN: 978-3-031-43901-8 (cited on pp. 15, 22).

[But+23] V. I. Butoi, J. J. Gonzalez Ortiz, T. Ma, M. R. Sabuncu, J. Guttag, and A. V. Dalca. Universeg: universal medical image segmentation. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 21381–21394, 2023. DOI: 10.1109/ICCV51070.2023.01960 (cited on p. 11).

[Cer+20] F. Cermelli, M. Mancini, S. Rota Bulò, E. Ricci, and B. Caputo. Modeling the background for incremental learning in semantic segmentation. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 13–19, 2020), pages 9230–9239, Seattle, WA, USA. IEEE, June 2020. DOI: 10.1109/CVPR42600.2020.00925 (cited on p. 15).

[Cha+18] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. S. Torr. *Riemannian walk for incremental learning: understanding forgetting and intransigence*. In *Computer Vision – ECCV 2018*. Springer International Publishing, 2018, pages 556–572. ISBN: 9783030012526. DOI: 10.1007/978-3-030-01252-6_33. URL: http://dx.doi.org/10.1007/978-3-030-01252-6_33 (cited on pp. 6, 32, 34, 45).

[Che+23] J. Cheng, J. Ye, Z. Deng, J. Chen, T. Li, H. Wang, Y. Su, Z. Huang, J. Chen, L. Jiang, H. Sun, J. He, S. Zhang, M. Zhu, and Y. Qiao. Sam-med2d, 2023. arXiv: 2308.16184 [cs.CV]. URL: https://arxiv.org/abs/2308.16184 (cited on pp. 1, 9).

[Che+26a] C. Che, Z. Wang, P. Yang, C. Wang, H. Ma, and Z. Shi. LoRA in LoRA: towards parameter-efficient architecture expansion for continual visual instruction tuning. In *Proceedings of the AAAI Conference on Artificial Intelligence* (January 20–27, 2026), volume 40 of number 24, pages 19978–19986, Singapore. Association for the Advancement of Artificial Intelligence, March 2026. DOI: 10.1609/aaai.v40i24.39082 (cited on pp. 13, 14, 21).

[Che+26b] C.-S. Chen, X. Zhang, G.-Y. Chen, Q. Xie, F. Zhang, and E.-J. Kuo. Few-shot continual learning for 3d brain mri with frozen foundation models, 2026. arXiv: 2602.23533 [eess.IV]. URL: https://arxiv.org/abs/2602.23533 (cited on pp. 4, 12, 17, 19, 22).

[Che+26c] Z. Chen, Y. Gao, M. Han, Y. Liu, Z. Chen, D. Yang, and L. Zhang. Forging a dynamic memory: retrieval-guided continual learning for generalist medical foundation models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 3–7, 2026), pages 32309–32321, Denver, CO, USA, June 2026. URL: https://openaccess.thecvf.com/content/CVPR2026/html/Chen_Forging_a_Dynamic_Memory_Retrieval-Guided_Continual_Learning_for_Generalist_Medical_CVPR_2026_paper.html (cited on pp. 13, 14, 21, 93).

[Çiç+16] Ö. Çiçek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger. 3d u-net: learning dense volumetric segmentation from sparse annotation. In S. Ourselin, L. Joskowicz, M. R. Sabuncu, G. Unal, and W. Wells, editors, *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2016*, pages 424–432, Cham. Springer International Publishing, 2016. ISBN: 978-3-319-46723-8 (cited on p. 26).

[DAl+23] M. D'Alessandro, A. Alonso, E. Calabrés, and M. Galar. Multimodal parameter-efficient few-shot class incremental learning. In *2023 IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)* (October 2–6, 2023), pages 3385–3395, Paris, France. IEEE, October 2023. DOI: 10.1109/

iccvw60793.2023.00364. URL: http://dx.doi.org/10.1109/ICCVW60793.2023.00364 (cited on pp. 5, 14, 22, 31, 32, 55, 97).

[De +22] M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars. A continual learning survey: defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385, 2022. DOI: 10.1109/TPAMI.2021.3057446 (cited on pp. 2, 3, 7, 12, 30, 35, 52, 53).

[Des+21] A. D. Desai, A. M. Schmidt, E. B. Rubin, C. M. Sandino, M. S. Black, V. Mazzoli, K. J. Stevens, R. Boutin, C. Re, G. E. Gold, B. Hargreaves, and A. Chaudhari. SKM-TEA: a dataset for accelerated MRI reconstruction with dense image labels for quantitative clinical evaluation. In *Thirty-fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 2)*, 2021. URL: https://openreview.net/forum?id=YDMFgD_qJuA (cited on pp. 1, 2, 5, 10, 37, 38, 40, 93, 97).

[Dic45] L. R. Dice. Measures of the amount of ecologic association between species. *Ecology*, 26(3):297–302, July 1945. DOI: 10.2307/1932409 (cited on p. 32).

[Dis+25] T. Dissanayake, Y. George, D. Mahapatra, S. Sridharan, C. Fookes, and Z. Ge. Few-shot learning for medical image segmentation: a review and comparative study. *ACM Comput. Surv.*, 58(1), September 2025. ISSN: 0360-0300. DOI: 10.1145/3746224. URL: https://doi.org/10.1145/3746224 (cited on pp. 10, 11).

[Du+24] Y. Du, F. Bai, T. Huang, and B. Zhao. SegVol: universal and interactive volumetric medical image segmentation. In A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang, editors, *Advances in Neural Information Processing Systems* (December 10–15, 2024), volume 37, pages 110746–110783, Vancouver, BC, Canada. Curran Associates, Inc., 2024. DOI: 10.52202/079017-3516. URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/c7c7cf10082e454b9662a686ce6f1b6f-Paper-Conference.pdf (cited on pp. 9, 21).

[Efr+93] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*, number 57 in Monographs on Statistics and Applied Probability. Chapman & Hall, New York, 1993. ISBN: 978-0-412-04231-7. DOI: 10.1007/978-1-4899-4541-9 (cited on p. 62).

[Gao+26a] Z. Gao and P. Morel. MedPEFT-CL: dual-phase parameter-efficient continual learning with medical semantic adapter and bidirectional memory consolidation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)* (March 6–10, 2026), pages 7699–7708, Tucson, AZ, USA. IEEE, March 2026. DOI: 10.1109/WACV61042.2026.00743 (cited on pp. 4, 18, 19, 23, 86).

[Gao+26b] Z. Gao and P. Morel. Prompt-aware adaptive elastic weight consolidation for continual learning in medical vision-language models. In J. Lokoč, L. Peška, J. Zahálka, S. Rudinac, M. Kastner, J. Chen, M.-C. Hu, J. Wu, and U. Sharma, editors, *MultiMedia Modeling. 32nd International Conference on Multimedia Modeling, MMM 2026, Prague, Czech Republic, January 29–31, 2026, Proceedings, Part I* (January 29–31, 2026), volume 16412 of *Lecture Notes in Computer Science*, pages 349–363, Singapore. Springer Nature Singapore, 2026. DOI: 10.1007/978-981-95-6950-2_25 (cited on pp. 4, 18, 19, 23).

[Gao26] Z. Gao. MedCRP-CL: continual medical image segmentation via Bayesian nonparametric semantic modality discovery. In T. Zhang, M. Dudik, M. Jaggi, A. Agarwal, S. Li, D. Schuurmans, J. Zhu, F. Berkenkamp, H. Dong, and A. Bietti, editors, *Proceedings of the 43rd International Conference on Machine Learning* (July 6–11, 2026), volume 306 of *Proceedings of Machine Learning Research*, pages 33385–33403, Seoul, South Korea. PMLR, 2026. URL: https://proceedings.mlr.press/v306/gao26s.html (cited on pp. 4, 18, 19, 22, 93).

[Gon+23] C. González, A. Ranem, D. Pinto dos Santos, A. Othman, and A. Mukhopadhyay. Lifelong nnu-net: a framework for standardized medical continual learning. *Scientific Reports*, 13(1):9381, 2023 (cited on pp. 15, 16, 22).

[Guo+25] D. Guo, Z. Ji, Y. Su, D. Zheng, H. Guo, P. Wang, K. Yan, Y. Wang, Q. Yu, Z. Li, M. Xu, J. Zhang, H. Li, J. Ge, T.-Y. Ho, B.-S. Huang, T. Ai, K. Zhao, N. Shen, Q. Wang, Y. Bian, T. Wu, P. Du, H. Zhang, F.-M. Kong, A. L. Yuille, C. H. Tan, C. Miao, P. J. Pickhardt, S. Yan, R. M. Summers, L. Lu, D. Jin, and X. Ye. A continual learning-driven model for accurate and generalizable segmentation of clinically comprehensive and fine-grained whole-body anatomies in ct, 2025. arXiv: 2503.12698 [eess.IV]. URL: https://arxiv.org/abs/2503.12698 (cited on pp. 3, 16).

[Hin+15] G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network, 2015. arXiv: 1503.02531 [stat.ML]. URL: https://arxiv.org/abs/1503.02531 (cited on p. 30).

[Hof+23] E. Hofstad, D. Bouget, A. Pedersen, K.-H. Støverud, T. Langø, and H. O. Leira. AeroPath: an airway segmentation benchmark dataset with challenging pathology. Version 1. Licence: Creative Commons Attribution 4.0 International. Zenodo, November 3, 2023. DOI: 10.5281/zenodo.10069289. URL: https://zenodo.org/records/10069289 (visited on 10/19/2026) (cited on p. 39).

[Hu+22] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022. URL: https://openreview.net/forum?id=nZeVKeeFYf9 (cited on pp. 5, 11, 30, 51, 52, 93, 97).

[Ise+21] F. Isensee, P. F. Jaeger, S. A. A. Kohl, J. Petersen, and K. H. Maier-Hein. nnU-Net: a self-configuring method for deep learning-based biomedical image segmentation. *Nature Methods*, 18(2):203–211, February 2021. ISSN: 1548-7105. DOI: 10.1038/s41592-020-01008-z. URL: https://doi.org/10.1038/s41592-020-01008-z (cited on pp. 1, 26, 41, 49).

[Ise+24] F. Isensee, T. Wald, C. Ulrich, M. Baumgartner, S. Roy, K. Maier-Hein, and P. F. Jäger. nnU-Net revisited: a call for rigorous validation in 3D medical image segmentation. In M. G. Linguraru, Q. Dou, A. Feragen, S. Giannarou, B. Glocker, K. Lekadir, and J. A. Schnabel, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2024. 27th International Conference, Marrakesh, Morocco, October 6–10, 2024, Proceedings, Part IX* (October 6–10, 2024), volume 15009 of *Lecture Notes in Computer Science*, pages 488–498, Cham. Springer Nature Switzerland, 2024. DOI: 10.1007/978-3-031-72114-4_47 (cited on p. 27).

[Ji+23] Z. Ji, D. Guo, P. Wang, K. Yan, L. Lu, M. Xu, Q. Wang, J. Ge, M. Gao, X. Ye, and D. Jin. Continual segment: towards a single, unified and non-forgetting continual segmentation model of 143 whole-body organs in CT scans. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)* (October 1–6, 2023), pages 21083–21094, Paris, France. IEEE, October 2023. DOI: 10.1109/ICCV51070.2023.01933 (cited on pp. 3, 15, 16).

[Kal23] D. Kalajdzievski. A rank stabilization scaling factor for fine-tuning with lora, 2023. arXiv: 2312.03732 [cs.CL]. URL: https://arxiv.org/abs/2312.03732 (cited on pp. 30, 51, 70).

[Kir+17] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, March 2017. ISSN: 1091-6490. DOI: 10.1073/pnas.1611835114. URL: http://dx.doi.org/10.1073/pnas.1611835114 (cited on pp. 15, 50).

[Kir+23] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, T. Xiao, S. Whitehead, A. C. Berg, W.-Y. Lo, P. Dollar, and R. Girshick. Segment anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 4015–4026, October 2023 (cited on p. 9).

[Li+18] Z. Li and D. Hoiem. Learning without forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(12):2935–2947, December 2018. DOI: 10.1109/TPAMI.2017.2773081 (cited on pp. 17, 30).

[Li+24] Z. Li, Y. Li, Q. Li, P. Wang, D. Guo, L. Lu, D. Jin, Y. Zhang, and Q. Hong. LViT: language meets vision transformer in medical image segmentation. *IEEE Transactions on Medical Imaging*, 43(1):96–107, January 2024. DOI: 10.1109/TMI.2023.3291719 (cited on p. 9).

[Li+25] Y. Li, G. Pang, W. Suo, C. Jing, Y. Xi, L. Liu, H. Chen, G. Liang, and P. Wang. CoLeCLIP: open-domain continual learning via joint task prompt and vocabulary learning. *IEEE Transactions on Neural Networks and Learning Systems*, 36(8):15137–15151, August 2025. DOI: 10.1109/TNNLS.2025.3547882 (cited on p. 14).

[Li+26] W. Li, J. Zhang, G. Wang, J. Ning, Y. Chen, G. Yang, and L. Gu. C^{2}GR: coupled comprehensive generative replay for a continually learnable universal segmentation model, 2026. arXiv: 2606.23473 [cs.CV]. URL: https://arxiv.org/abs/2606.23473 (cited on pp. 16, 22).

[Liu+22] P. Liu, X. Wang, M. Fan, H. Pan, M. Yin, X. Zhu, D. Du, X. Zhao, L. Xiao, L. Ding, X. Wu, and S. K. Zhou. Learning incrementally to segment multiple organs in a CT image. In L. Wang, Q. Dou, P. T. Fletcher, S. Speidel, and

S. Li, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2022. 25th International Conference, Singapore, September 18–22, 2022, Proceedings, Part IV* (September 18–22, 2022), volume 13434 of *Lecture Notes in Computer Science*, pages 714–724, Cham. Springer Nature Switzerland, 2022. DOI: 10.1007/978-3-031-16440-8_68 (cited on p. 15).

[Liu+23a] J. Liu, Y. Zhang, J.-N. Chen, J. Xiao, Y. Lu, B. A. Landman, Y. Yuan, A. Yuille, Y. Tang, and Z. Zhou. Clip-driven universal model for organ segmentation and tumor detection. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)* (October 1–6, 2023), pages 21095–21107, Paris, France. IEEE, October 2023. DOI: 10.1109/iccv51070.2023.01934. URL: http://dx.doi.org/10.1109/ICCV51070.2023.01934 (cited on pp. 10, 21).

[Liu+23b] X. Liu, H. A. Shih, F. Xing, E. Santarnecchi, G. El Fakhri, and J. Woo. Incremental learning for heterogeneous structure segmentation in brain tumor MRI. In H. Greenspan, A. Madabhushi, P. Mousavi, S. Salcudean, J. Duncan, T. Syeda-Mahmood, and R. Taylor, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023. 26th International Conference, Vancouver, BC, Canada, October 8–12, 2023, Proceedings, Part II* (October 8–12, 2023), volume 14221 of *Lecture Notes in Computer Science*, pages 46–56, Cham. Springer Nature Switzerland, 2023. DOI: 10.1007/978-3-031-43895-0_5 (cited on p. 15).

[Liu+24] J. Liu, Y. Zhang, K. Wang, M. C. Yavuz, X. Chen, Y. Yuan, H. Li, Y. Yang, A. Yuille, Y. Tang, and Z. Zhou. Universal and extensible language-vision models for organ segmentation and tumor detection from abdominal computed tomography. *Medical Image Analysis*, 97:103226, October 2024. DOI: 10.1016/j.media.2024.103226 (cited on pp. 15, 16, 19, 22).

[Liu+26a] A. Liu, X. R. Cao, Y. Shen, Y. Lu, X. Li, Q. Chen, and J. Chen. Medsam3: delving into segment anything with medical concepts, 2026. arXiv: 2511.19046 [cs.CV]. URL: https://arxiv.org/abs/2511.19046 (cited on p. 10).

[Liu+26b] Y. Liu, Q. Hong, L. Huang, A. Gomez-Villa, D. Goswami, T. Peng, X. Liu, J. van de Weijer, and Y. Tian. Continual learning for vlms: a survey and

taxonomy beyond forgetting, 2026. arXiv: 2508.04227 [cs.CV]. URL: https://arxiv.org/abs/2508.04227 (cited on pp. 5, 12, 13, 50).

[Ly+26] S. T. Ly and H. V. Nguyen. Self-auditing parameter-efficient fine-tuning for few-shot 3D medical image segmentation. In G. Yang, E. Adeli, M. de Bruijne, B. W. Papież, S. Speidel, P. Tiwari, G. Zheng, M. Yaqub, Q. Dou, and I. Rekik, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2026. 29th International Conference, Strasbourg, France, September 27 – October 1, 2026, Proceedings, Part III* (September 27–October 1, 2026), volume 16880 of *Lecture Notes in Computer Science*, pages 559–569, Cham. Springer Nature Switzerland, 2026. DOI: 10.1007/978-3-032-38069-2_53 (cited on pp. 11, 12).

[Ma+24] J. Ma, Y. He, F. Li, L. Han, C. You, and B. Wang. Segment anything in medical images. *Nature Communications*, 15(1), January 2024. ISSN: 2041-1723. DOI: 10.1038/s41467-024-44824-z. URL: http://dx.doi.org/10.1038/s41467-024-44824-z (cited on pp. 1, 9).

[Mic+19] U. Michieli and P. Zanuttigh. Incremental learning techniques for semantic segmentation. In *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)* (October 27–28, 2019), pages 3205–3212, Seoul, Korea (South). IEEE, October 2019. DOI: 10.1109/ICCVW.2019.00400 (cited on p. 15).

[Ozd+18] F. Ozdemir, P. Fuernstahl, and O. Goksel. *Learn the new, keep the old: extending pretrained models with new anatomy and images*. In *Medical Image Computing and Computer Assisted Intervention – MICCAI 2018. 21st International Conference, Granada, Spain, September 16–20, 2018, Proceedings, Part IV*. A. F. Frangi, J. A. Schnabel, C. Davatzikos, C. Alberola-López, and G. Fichtinger, editors. Springer International Publishing, Cham, 2018, pages 361–369. ISBN: 9783030009373. DOI: 10.1007/978-3-030-00937-3_42. URL: http://dx.doi.org/10.1007/978-3-030-00937-3_42 (cited on p. 15).

[Rad+21] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever. Learning transferable visual models from natural language supervision. In M. Meila and T. Zhang, editors, *Proceedings of the 38th International Conference on*

Machine Learning (July 18–24, 2021), volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763, Virtual. PMLR, 2021. URL: https://proceedings.mlr.press/v139/radford21a.html (cited on pp. 9, 12).

[Ran+22] A. Ranem, C. Gonzalez, and A. Mukhopadhyay. Continual hippocampus segmentation with transformers. In *2022 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)* (June 19–20, 2022), pages 3710–3719, New Orleans, LA, USA. IEEE, June 2022. DOI: 10.1109/CVPRW56347.2022.00415 (cited on p. 16).

[Reb+17] S.-A. Rebuffi, A. Kolesnikov, G. Sperl, and C. H. Lampert. Icarl: incremental classifier and representation learning. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5533–5542, 2017. DOI: 10.1109/CVPR.2017.587 (cited on p. 53).

[Rok+26] M. Rokuss, M. Langenberg, Y. Kirchhoff, F. Isensee, B. Hamm, C. Ulrich, S. Regnery, L. Bauer, E. Katsigiannopulos, T. Norajitra, and K. Maier-Hein. VoxTell: free-text promptable universal 3D medical image segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 3–7, 2026), pages 37538–37557, Denver, CO, USA, June 2026. URL: https://openaccess.thecvf.com/content/CVPR2026/html/Rokuss_VoxTell_Free-Text_Promptable_Universal_3D_Medical_Image_Segmentation_CVPR_2026_paper.html (cited on pp. 1, 2, 5, 10, 21, 26–29, 39, 40, 47, 48, 58, 93, 97).

[Sil+25] J. Silva-Rodríguez, J. Dolz, and I. Ben Ayed. Towards foundation models and few-shot parameter-efficient fine-tuning for volumetric organ segmentation. *Medical Image Analysis*, 103:103596, July 2025. DOI: 10.1016/j.media.2025.103596 (cited on pp. 11, 12, 21, 90).

[Staa] Stanford MIMI Lab. SKM-TEA tutorial. Google Colab notebook, linked as Tutorial from the SKM-TEA code repository (https://github.com/StanfordMIMI/skm-tea). URL: https://colab.research.google.com/drive/1PluqK77pobD5dXE7zzBLEAeBgaaeGKXa (visited on 10/19/2026) (cited on p. 38).

[Stab] Stanford MIMI Lab. Stanford knee MRI multi-task evaluation (SKM-TEA) dataset. Dataset documentation (DATASET.md) in the SKM-TEA code repository (https://github.com/StanfordMIMI/skm-tea). URL: https://

github.com/StanfordMIMI/skm-tea/blob/main/DATASET.md (visited on 10/19/2026) (cited on pp. 37, 38, 41).

[Stø+23] K.-H. Støverud, D. Bouget, A. Pedersen, H. O. Leira, T. Langø, and E. F. Hofstad. Aeropath: an airway segmentation benchmark dataset with challenging pathology, 2023. arXiv: 2311.01138 [cs.CV]. URL: https://arxiv.org/abs/2311.01138 (cited on pp. 5, 37–39, 97).

[Vas+17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, editors, *Advances in Neural Information Processing Systems* (December 4–9, 2017), volume 30, pages 5998–6008, Long Beach, CA, USA. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf (cited on pp. 26, 27).

[vdVen+25] G. M. van de Ven, N. Soures, and D. Kudithipudi. *Continual learning and catastrophic forgetting*. In *Learning and Memory: A Comprehensive Reference*. Elsevier, 2025, pages 153–168. ISBN: 9780443157554. DOI: 10.1016/b978-0-443-15754-7.00073-0. URL: http://dx.doi.org/10.1016/B978-0-443-15754-7.00073-0 (cited on pp. 12, 29).

[Wan+22] Y. Wang, Z. Huang, and X. Hong. S-Prompts learning with pre-trained transformers: an Occam's razor for domain incremental learning. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems* (November 28–December 9, 2022), volume 35, pages 5682–5695, New Orleans, LA, USA. Curran Associates, Inc., 2022. DOI: 10.52202/068431-0411. URL: https://proceedings.neurips.cc/paper_files/paper/2022/file/25886d7a7cf4e33fd44072a0cd81bf30-Paper-Conference.pdf (cited on p. 14).

[Wan+24] G. Wang, J. Ye, J. Cheng, T. Li, Z. Chen, J. Cai, J. He, and B. Zhuang. SAM-Med3D-MoE: towards a non-forgetting segment anything model via mixture of experts for 3D medical image segmentation. In M. G. Linguraru, Q. Dou, A. Feragen, S. Giannarou, B. Glocker, K. Lekadir, and J. A. Schnabel, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2024. 27th International Conference, Marrakesh, Morocco, Octo-*

ber 6–10, 2024, Proceedings, Part IX (October 6–10, 2024), volume 15009 of *Lecture Notes in Computer Science*, pages 552–561, Cham. Springer Nature Switzerland, 2024. DOI: 10.1007/978-3-031-72114-4_53 (cited on pp. 4, 17, 22).

[Wan+26a] B. Wang, H. Zhou, Y. Gao, and X. Zhuang. Beyond forgetting in continual medical image segmentation: a comprehensive benchmark study, 2026. arXiv: 2605.06160 [cs.CV]. URL: https://arxiv.org/abs/2605.06160 (cited on pp. 16, 22, 87).

[Wan+26b] J. Wang, W. Dai, H. Wang, S. Yang, H. Bi, and J. Sun. Continual alignment for SAM: rethinking foundation models for medical image segmentation in continual learning. In *2026 IEEE/CVF Conference on Computer Vision and Pattern Recognition – Findings Track (CVPRF)* (June 3–7, 2026), pages 7523–7532, Denver, CO, USA. IEEE, June 2026. DOI: 10.1109/CVPRF71909.2026.00722 (cited on pp. 4, 17, 22).

[Wu+25] B. Wu, W. Shi, J. Wang, and M. Ye. Synthetic data is an elegant GIFT for continual vision-language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 10–17, 2025), pages 2813–2823, Nashville, TN, USA. IEEE, June 2025. DOI: 10.1109/CVPR52734.2025.00268 (cited on pp. 12, 14, 21).

[Ye+25] Y. Ye, Y. Wu, X. Luo, H. Zhang, Z. Chen, T. Dang, Y. Zhang, and Y. Xia. MedSeqFT: sequential fine-tuning foundation models for 3D medical image segmentation, 2025. arXiv: 2509.06096 [cs.CV]. URL: https://arxiv.org/abs/2509.06096 (cited on pp. 3, 17, 19, 22).

[Yu+24] J. Yu, Y. Zhuge, L. Zhang, P. Hu, D. Wang, H. Lu, and Y. He. Boosting continual learning of vision-language models via mixture-of-experts adapters. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June 16–22, 2024), pages 23219–23230, Seattle, WA, USA. IEEE, June 2024. DOI: 10.1109/CVPR52733.2024.02191 (cited on p. 14).

[Zan+24] M. Zanella and I. Ben Ayed. Low-rank few-shot adaptation of vision-language models. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)* (June 17–18, 2024), pages 1593–1603, Seattle, WA, USA. IEEE, June 2024. DOI: 10.1109/CVPRW63382.2024.00166 (cited on pp. 11, 21, 51).

[Zha+23] Y. Zhang, X. Li, H. Chen, A. L. Yuille, Y. Liu, and Z. Zhou. Continual learning for abdominal multi-organ and tumor segmentation. In H. Greenspan, A. Madabhushi, P. Mousavi, S. Salcudean, J. Duncan, T. Syeda-Mahmood, and R. Taylor, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2023. 26th International Conference, Vancouver, BC, Canada, October 8–12, 2023, Proceedings, Part II* (October 8–12, 2023), volume 14221 of *Lecture Notes in Computer Science*, pages 35–45, Cham. Springer Nature Switzerland, 2023. DOI: 10.1007/978-3-031-43895-0_4 (cited on pp. 3, 16, 19, 22).

[Zha+24] W. Zhang, P. Janson, R. Aljundi, and M. Elhoseiny. Overcoming generic knowledge loss with selective parameter update. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 24046–24056, June 2024 (cited on pp. 4, 13, 34, 44).

[Zha+25a] H. Zhang, Y. Liang, H. Kuang, L. Cen, Z. Qu, Y. Cen, M. Zeng, and S. Kan. Contrastive regularization over lora for multimodal biomedical image incremental learning. In *Proceedings of the 33rd ACM International Conference on Multimedia*, MM '25, pages 7500–7509, Dublin, Ireland. ACM, October 2025. DOI: 10.1145/3746027.3754917. URL: http://dx.doi.org/10.1145/3746027.3754917 (cited on pp. 14, 22).

[Zha+25b] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou. Qwen3 Embedding: advancing text embedding and reranking through foundation models, 2025. arXiv: 2506.05176 [cs.CL]. URL: https://arxiv.org/abs/2506.05176 (cited on pp. 27, 47).

[Zha+25c] T. Zhao, Y. Gu, J. Yang, N. Usuyama, H. H. Lee, S. Kiblawi, T. Naumann, J. Gao, A. Crabtree, J. Abel, C. Moung-Wen, B. Piening, C. Bifulco, M. Wei, H. Poon, and S. Wang. A foundation model for joint segmentation, detection and recognition of biomedical objects across nine modalities. *Nature Methods*, 22(1):166–176, 2025. ISSN: 1548-7091. DOI: 10.1038/s41592-024-02499-w (cited on p. 10).

[Zha+25d] Z. Zhao, Y. Zhang, C. Wu, X. Zhang, X. Zhou, Y. Zhang, Y. Wang, and W. Xie. Large-vocabulary segmentation for medical images with text prompts. *npj Digital Medicine*, 8(1):566, September 2025. DOI: 10.1038/s41746-025-01964-w (cited on pp. 1, 10, 21).

[Zha+26] T. Zhang, C. Wang, Y. Li, Z. Chen, X. Zhan, and H. Shan. Skill-evolving grounded reasoning for free-text promptable 3D medical image segmentation. In G. Yang, E. Adeli, M. de Bruijne, B. W. Papież, S. Speidel, P. Tiwari, G. Zheng, M. Yaqub, Q. Dou, and I. Rekik, editors, *Medical Image Computing and Computer Assisted Intervention – MICCAI 2026. 29th International Conference, Strasbourg, France, September 27 – October 1, 2026, Proceedings, Part VII* (September 27–October 1, 2026), volume 16884 of *Lecture Notes in Computer Science*, pages 536–545, Cham. Springer Nature Switzerland, 2026. DOI: 10.1007/978-3-032-38085-2_50 (cited on pp. 10, 92, 94).

[Zhe+23] Z. Zheng, M. Ma, K. Wang, Z. Qin, X. Yue, and Y. You. Preventing zero-shot transfer degradation in continual learning of vision-language models. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)* (October 1–6, 2023), pages 19068–19079, Paris, France. IEEE, 2023. DOI: 10.1109/ICCV51070.2023.01752 (cited on pp. 2, 5, 6, 13, 14, 21, 31, 32, 34, 35, 53, 54, 97).

[Zho+22a] K. Zhou, J. Yang, C. C. Loy, and Z. Liu. Conditional prompt learning for vision-language models. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16816–16825, 2022 (cited on pp. 5, 11, 31, 55).

[Zho+22b] K. Zhou, J. Yang, C. C. Loy, and Z. Liu. Learning to prompt for vision-language models. *International Journal of Computer Vision*, 130(9):2337–2348, 2022. DOI: 10.1007/s11263-022-01653-1 (cited on pp. 11, 31).